Compiladores — Questões e Resoluções

Provas do meio de período, provas finais e exercícios — enunciados com resolução

Documento corrigido e reorganizado para estudo: as provas do meio estão todas na Parte I e as provas finais todas na Parte II, com sumário e páginas-divisórias.

Questões repetidas aparecem uma única vez (as demais viram nota de referência) e o texto foi reflowado — sem fim de linha no meio da frase.

Como este documento está organizado

PARTE I — PROVAS DO MEIO DE PERÍODO (MIDTERMS) — 9 seções: Primeira Prova do André · prova do meio (gabarito of, Segunda Prova do André · prova do meio (gabarito ofi, Outra primeira prova: Prova Parcial 1 - 2023.1 · pro, CSE401 midterm inverno 2008, CSE401 midterm inverno 2009, CSE401 midterm inverno 2010, CSE401 midterm outono 2010, Prova au02 (outono de 2002) · prova do meio, Prova au05 (outono de 2005) · prova do meio

PARTE II — PROVAS FINAIS — 6 seções: CSE401 final inverno 2009, CSE401 final inverno 2010, CSE401 final outono 2010, CSEP501 inverno 2008 · prova final, CSEP501 outono 2009 · prova final, CSEP501 outono 2011 · prova final

PARTE III — EXERCÍCIOS DOS SLIDES (COM SOLUÇÕES) — 5 seções: Gramática ambígua de expressões aritméticas (slide C, Dangling else (slide C-grammars), Que linguagem esta gramática gera? (slide E-lr-const, Arrays 2-D: Fortran × C (slide K-codeshape-1), Grafo de interferência e coloração (slide N-backend)

Regras usadas nesta edição

1. Meio x final. Toda prova com rótulo de origem ficou no seu grupo (pastas “Provas mid” e “Provas finals”). As cinco provas sem rótulo (au02, au05, csep501wi08, csep501au09, csep501au11) foram classificadas pelo conteúdo predominante: au02 e au05 são de meio; wi08, au09 e au11 são finais (cobrem o curso inteiro). A 2ª prova do André, embora trate de assuntos de fim de curso, é a segunda prova do professor — por isso está com as provas do meio; o aviso está no topo dela.

2. Repetições. Onde a mesma questão aparece em dois lugares, fica a primeira ocorrência do documento e a segunda é substituída por uma nota com a página exata — você não copia a mesma questão duas vezes e continua sabendo que ela existia naquela prova.

3. Texto. A transcrição foi reflowada: as linhas quebradas no meio da frase foram reunidas, as listas e os rótulos (Passo1, a:, (b), Grafo:) ganharam a quebra de linha que faltava, e blocos de código e figuras ficaram como no original.

Sumário

PARTE I — PROVAS DO MEIO DE PERÍODO (MIDTERMS) 5

1. Provas do Professor André (provas parciais) 6

I.1 — Primeira Prova do André · prova do meio (gabarito oficial, inglês) 6

Questão 1 (16 pontos) 6

Questão 2 (24 pontos) 6

Questão 3 (18 pontos) 8

Questão 4 (16 pontos) 10

I.2 — Segunda Prova do André · prova do meio (gabarito oficial, inglês) 12

Questão 1 (14 pontos) 12

Questão 7 (12 pontos) — A little coloring 13

Questão 7 (16 pontos) — Abstract syntax and semantics 15

Questão 5 (20 pontos) — x86 hacking 17

I.3 — Outra primeira prova: Prova Parcial 1 - 2023.1 · prova do meio 21

Questão 1 — Conjuntos FIRST e FOLLOW 21

Questão 2 — Expressões regulares 21

Questão 3 — Gramática de expressões: máquina LR(0), LR(0)? e SLR? 22

Questão 4 — Por que parsing e verificação semântica em fases separadas? 22

2. Provas oficiais CSE401 — midterms 23

M1 — CSE401 midterm inverno 2008 23

Questão 1 (10 pontos) 23

Questão 2 (8 pontos) 23

Questão 3 (10 pontos) 23

Questão 4 (16 pontos) 24

Questão 5 (14 pontos) 24

Questão 6 (12 pontos) 26

Questão 7 (10 pontos) 26

Questão 8 (10 pontos) 27

M2 — CSE401 midterm inverno 2009 28

Pergunta 1 (16 pontos) 28

Pergunta 2 (16 pontos) 28

Pergunta 3 (16 pontos) 28

Pergunta 4 (16 pontos) 29

Pergunta 5 (20 pontos) 29

Pergunta 6 (16 pontos) 31

M3 — CSE401 midterm inverno 2010 33

Questão 1 (18 pontos) 33

Questão 2 (24 pontos) 33

Questão 3 (16 pontos) 33

Questão 4 (10 pontos) 33

Questão 5 (8 pontos) 33

Questão 6 (8 pontos) 34

Questão 7 (16 pontos) 34

M4 — CSE401 midterm outono 2010 35

Pergunta 1 (20 pontos) 35

Pergunta 2 — Ambiguidade (14 pontos) 35

Pergunta 3 (30 pontos) 37

Pergunta 4 — FIRST/FOLLOW/NULLABLE (20 pontos) 39

Pergunta 5 — As ferramentas de parse (16 pontos) 40

3. Provas complementares de meio 42

O1 — Prova au02 (outono de 2002) · prova do meio 42

Questão 1 (10 pontos) 42

Questão 2 (10 pontos) 42

Questão 3 (10 pontos) 43

Questão 4 (10 pontos) 43
Questão 5 (12 pontos) 44
Questão 6 (20 pontos) 44
O2 — Prova au05 (outono de 2005) · prova do meio 47
Questão 1 (8 pontos) 47
Questão 2 (8 pontos) 47
Questão 3 (5 pontos) 47
Questão 4 (8 pontos) 47
Questão 5 (3 pontos) 48
Questão 6 (8 pontos) 48
Questão 7 (8 pontos) 49
Questão 8 (20 pontos) 49
Questão 9 (16 pontos) 52

PARTE II — PROVAS FINAIS 56

1. Provas oficiais CSE401 — finais 57

F1 — CSE401 final inverno 2009 57
Questão 1 (10 pontos) 57
Questão 2 (8 pontos) 57
Questão 3 (8 pontos) 57
Questão 4 (10 pontos) 58
Questão 5 (8 pontos) 58
Questão 6 (9 pontos) 58
Questão 7 (8 pontos) 59
Questão 8 (15 pontos) 60
Questão 9 (12 pontos) 62
Questão 10 (12 pontos) 63
F2 — CSE401 final inverno 2010 66
Pergunta 1 (14 pontos) 66
Pergunta 2 (6 pontos) 66
Pergunta 3 (6 pontos) 66
Pergunta 4 (10 pontos) 66
Pergunta 7 (12 pontos) 67
F3 — CSE401 final outono 2010 68
Questão 1 (10 pontos) 68
Questão 2 (34 pontos) 68
Questão 3 (32 pontos) 73
Questão 4 (12 pontos) 77
Questão 5 (12 pontos) 78

2. Provas complementares de final 80

O3 — CSEP501 inverno 2008 · prova final 80
Questão 1 (12 pontos) 80
Questão 2 (10 pontos) 80
Questão 3 (15 pontos) 81
Questão 4 (15 pontos) 82
Questão 5 (15 pontos) 83
Questão 6 (8 pontos) 85
Questão 7 (10 pontos) 85
Questão 8 (15 pontos) 87
O4 — CSEP501 outono 2009 · prova final 90
Questão 1 (12 pontos) 90
Questão 2 (14 pontos) 90
Questão 3 (15 pontos) 91
Questão 4 (15 pontos) 93
Questão 5 (15 pontos) 95
Questão 6 (14 pontos) 98
Questão 7 (15 pontos) 100

O5 — CSEP501 outono 2011 · prova final ... 104
Questão 1 (10 pontos — 5 cada) ... 104
Questão 2 (10 pontos) ... 104
Questão 3 (18 pontos) ... 105
Questão 4 (22 pontos) ... 108
Questão 5 (17 pontos) ... 110
Questão 6 (13 pontos) ... 113

PARTE III — EXERCÍCIOS DOS SLIDES (COM SOLUÇÕES) ... 116

Exercícios dos slides ... 117

Exercício 1 — Gramática ambígua de expressões aritméticas (slide C-grammars) ... 117
Questão 1 ... 117
Exercício 2 — Dangling else (slide C-grammars) ... 118
Questão 1 ... 118
Exercício 3 — Que linguagem esta gramática gera? (slide E-lr-construction) ... 119
Questão 1 ... 119
Exercício 4 — Arrays 2-D: Fortran × C (slide K-codeshape-1) ... 120
Questão 1 ... 120
Exercício 5 — Grafo de interferência e coloração (slide N-backend) ... 121
Exercício — Grafo de interferência e coloração (slide N-backend) ... 121

PARTE I — PROVAS DO MEIO DE PERÍODO (MIDTERMS)

Todas as questões de prova do meio do material, reunidas num único bloco: as provas do Prof. André (parciais), as provas oficiais CSE401 de meio de período e as provas complementares de conteúdo de meio.

Se você está copiando questões de prova do meio para o caderno, copie desta parte — ela termina na página em que começa a Parte II.

Nesta parte:

• I.1 — Primeira Prova do André · prova do meio (gabarito oficial, inglês) · I.2 — Segunda Prova do André · prova do meio (gabarito oficial, inglês) · I.3 — Outra primeira prova: Prova Parcial 1 - 2023.1 · prova do meio

• M1 — CSE401 midterm inverno 2008 · M2 — CSE401 midterm inverno 2009 · M3 — CSE401 midterm inverno 2010 · M4 — CSE401 midterm outono 2010

• O1 — Prova au02 (outono de 2002) · prova do meio · O2 — Prova au05 (outono de 2005) · prova do meio

O sumário com a página de cada prova e de cada questão está logo após a página de organização.

1. Provas do Professor André (provas parciais)

As três provas do Prof. André presentes no material. A 1ª e a 2ª vêm com o gabarito oficial resolvido em inglês; a Prova Parcial 1 - 2023.1 veio em português (fotos do gabarito impresso).

I.1 — Primeira Prova do André · prova do meio (gabarito oficial, inglês)

A Primeira Prova do André é um exame de compiladores cujo gabarito resolvido foi distribuído em inglês (estilo CSE401). Reproduzimos abaixo as páginas do gabarito, que contêm enunciado e resolução completa de cada item. Ordem das páginas: Questão 1 (fases do compilador), Questão 2 (gramática de expressões — máquina LR(0), itens (b) e (c)), questão de expressões regulares para constantes de ponto flutuante FORTRAN e Questão 4 (FIRST/FOLLOW e LL(1)).

Questão 1 (16 pontos)

The lexical- (scanner), syntactic- (parser), and semantic-analysis phases of a compiler front-end each process parts of the source program in particular ways and also check certain rules of the language being compiled. For each of the following possible language rules, specify which phase of the compiler should verify that a program conforms to that rule and why that part of the compiler is the best place for that check. If a check could be done equally well in more than one phase of the compiler, briefly discuss the tradeoffs between the alternative implementations. Keep your answers short and to the point.

(a) A function is called with the correct number of arguments.

(b) Underscore characters (_) may appear in the middle of identifiers, but not at the beginning or end (i.e., `this_identifier` is legal, but `_this_one` is not).

(c) Every variable must be declared before it is used in the program (the classic C or Pascal rule).

(d) Assignment statements must end with a semicolon (;).

Resolução

(a) **Semantics.** This requires comparing information about the declaration and call of the function, which cannot be done in the parser or scanner.

(b) **Scanner.** In a normal compiler, only the scanner sees individual input characters. The rest of the compiler deals with tokens or intermediate representations of the program.

(c) **Most likely semantics**, since this can be detected in a pass over the AST at the same time other checks are being performed. It is possible to do this in the parser if the parser records information about identifiers as it sees their declarations and then checks each use to see if the identifier was previously declared.

(d) **Parser.** The scanner can't do this since it doesn't know the context in which the tokens appear. The semicolon is not present in the AST, so the semantics phase can't check for it.

Questão 2 (24 pontos)

Consider the following grammar:

```
0. exp' ::= exp $
1. exp ::= id
2. exp ::= ( exp )
3. exp ::= ( id ) exp
```

(a) (18 points) Draw the LR(0) state machine for this grammar. You do not need to write out the parser tables or first/follow/nullable sets, although you can do that if it helps you to answer the remaining parts of the question on the next page.

(b) (3 points) Is this grammar LR(0)? Why or why not?

(c) (3 points) Is this grammar SLR? Why or why not?

Resolução

(a) Máquina de estados LR(0) (9 estados):

```
S0 (inicial): { exp' ::= . exp $
 exp ::= . id

exp ::= . ( exp )
 exp ::= . ( id ) exp }
 exp -> S2, id -> S3, ( -> S1

S1: { exp ::= ( . exp )
 exp ::= ( . id ) exp
 exp ::= . id
 exp ::= . ( exp )
 exp ::= . ( id ) exp }
 ( -> S1 (auto-loop), exp -> S5, id -> S4

S2: { exp' ::= exp . $ } (aceita em $)
S3: { exp ::= id . } (reduz exp ::= id)

S4 (marcado com *): { exp ::= ( id . ) exp
 exp ::= id . }
 ) -> S7 [conflito deslocar/reduzir]

S5: { exp ::= ( exp . ) } ) -> S6
S6: { exp ::= ( exp ) . } (reduz)

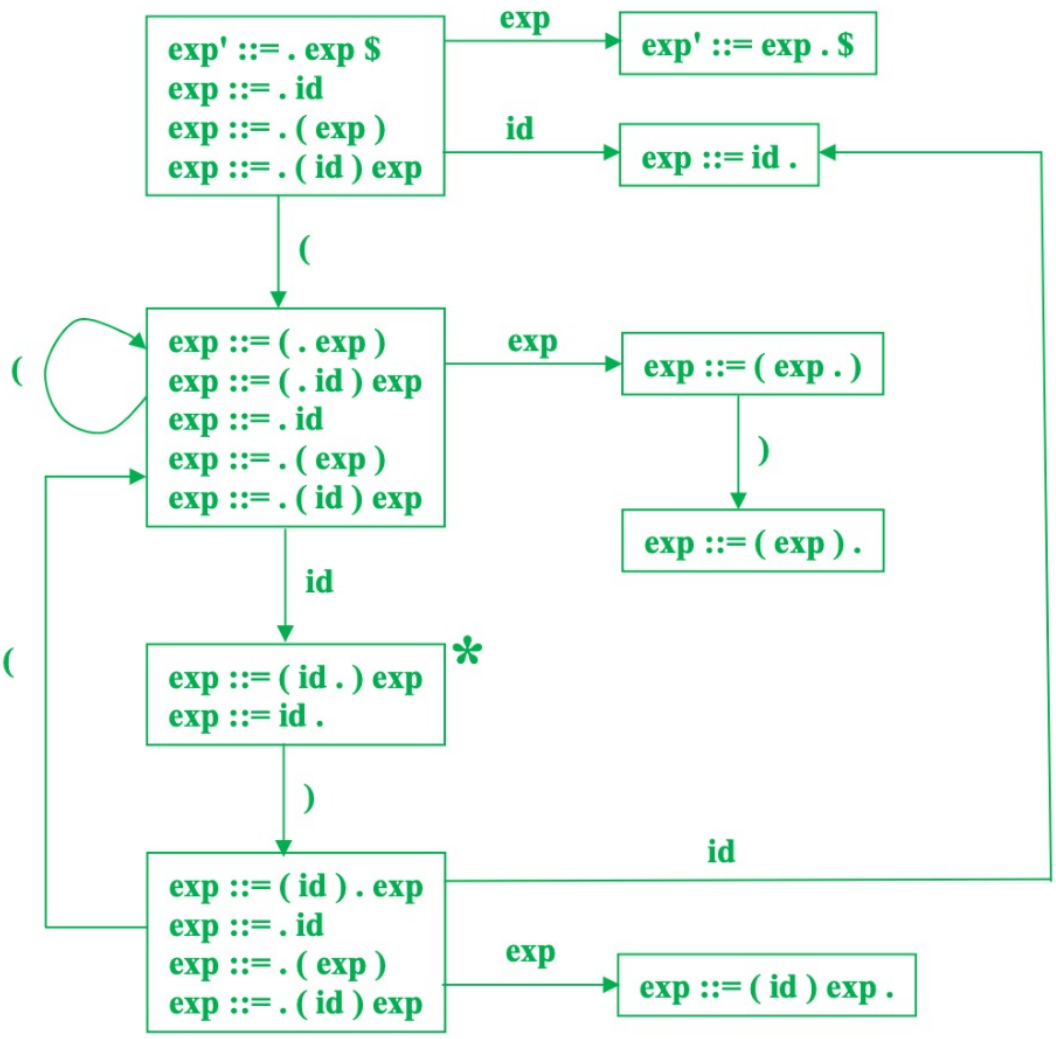
S7: { exp ::= ( id ) . exp
 exp ::= . id
 exp ::= . ( exp )
 exp ::= . ( id ) exp }
 exp -> S8, ( -> S1, id -> S3

S8: { exp ::= ( id ) exp . } (reduz)
```

Diagrama oficial do gabarito:

0. *exp′* ::= *exp* $
1. *exp* ::= `id`
2. *exp* ::= `(` *exp* `)`
3. *exp* ::= `(` `id` `)` *exp*

(a) (18 points) Draw the LR(0) state machine for this grammar. You do not need to write out the parser tables or first/follow/nullable sets, although you can do that if it helps you to answer the remaining parts of the question on the next page.



(b) **No. The state labeled * has a shift-reduce conflict.**

(c) **No.** The grammar would be SLR if we could use the contents of FOLLOW(exp) to resolve the situation. But FOLLOW(exp) contains ')', so that doesn't get rid of the shift-reduce conflict in that state.

Questão 3 (18 pontos)

(Obs.: a página com o enunciado introdutório — a descrição do formato de constantes de ponto flutuante FORTRAN — não está presente no gabarito; a questão começa no item (a), com 9 pontos cada.)

(a) (9 points) Give a regular expression for FORTRAN floating-point constants as described above. You may only use basic regular expression operators (concatenation rs, choice r|s, Kleene star r*) and the additional

operators r+ and r?. You may also specify character sets using the notation [abw-z], and sets excluding specified characters [^aeiou]. Finally, you can name parts of the regular expression, like vowel=[aeiou].

(b) (9 points) Draw a DFA (Deterministic Finite Automata) that recognizes FORTRAN floating-point constants as described in the problem and generated by the regular expression in your answer to part (a).

Resolução

(a) Regex (como escrito no gabarito):

```
dec = [0-9]+ . [0-9]* | [0-9]+
exp = [eEdE] [+-]? [0-9]+
float = {dec} {exp}? | [0-9]+ {exp}
```

(Nota: a classe do expoente foi escrita `[eEdE]` no gabarito — aparente typo, pois as transições do DFA do item (b) usam `eEdD`.)

(b) DFA do gabarito (7 estados; q0 inicial, q3 e q6 finais):

```
q0 -0-9-> q1 q0 -.-> q2
q1 -0-9-> q1 (loop) q1 -.-> q3 q1 -eEdD-> q4
q2 -0-9-> q3
q3 -0-9-> q3 (loop) q3 -eEdD-> q4
q4 -+ - -> q5 (sinal) q4 -0-9-> q6
q5 -0-9-> q6
q6 -0-9-> q6 (loop)
```

Leitura: `q1` acumula dígitos antes do ponto, `q3` depois do ponto (final — mantissa sem expoente), `q4` lê o expoente `e`/`d`, `q5` o sinal opcional, `q6` os dígitos do expoente (final). Página oficial:

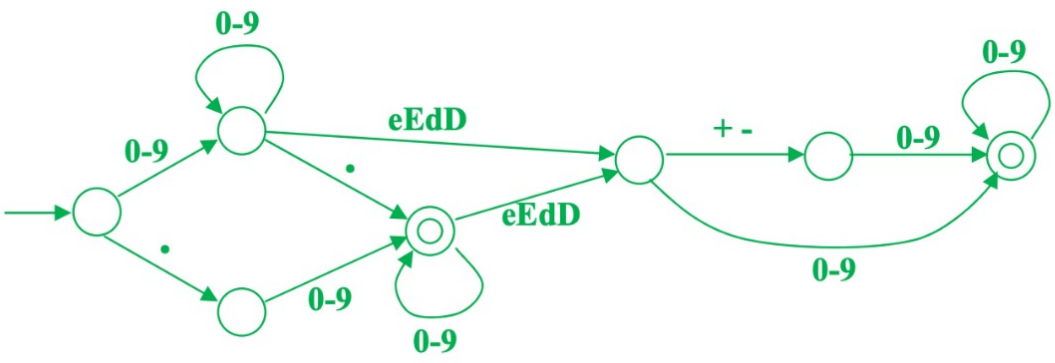
(a) (9 points) Give a regular expression for FORTRAN floating-point constants as described above. You may only use basic regular expression operators (concatenation `rs`, choice `r|s`, Kleene star `r*`) and the additional operators `r+` and `r?`. You may also specify character sets using the notation `[abw-z]`, and sets excluding specified characters `[^aeiou]`. Finally, you can name parts of the regular expression, like `vowel=[aeiou]`.

dec = [0-9]+ . [0-9]* | [0-9]+

exp = [eEdE] [+ -]? [0-9]+

float = {dec} {exp}? | [0-9]+ {exp}

(b) (9 points) Draw a DFA (Deterministic Finite Automata) that recognizes FORTRAN floating-point constants as described in the problem and generated by the regular expression in your answer to part (a).



Questão 4 (16 pontos)

Consider the following grammar, which we previously saw in a homework problem:

```
S ::= ( L ) | x
L ::= L , S | S
```

(a) Complete the following table with the FIRST and FOLLOW information for the productions and non-terminals in this grammar.

(b) Does this grammar have the LL(1) property that makes it suitable for top-down parsing? Briefly justify your answer.

(c) Describe how to fix this grammar, if necessary, to make it suitable for LL(1) parsing. Be specific.

Resolução

(a) FIRST/FOLLOW (como no gabarito):

Production	FIRST	FOLLOW
S ::= (L)	(	

Production	FIRST	FOLLOW
S ::= x	x	
L ::= L , S	(x	,)
L ::= S	(x	

(Nota: o gabarito deixa as células de FOLLOW(S) em branco; o FOLLOW padrão seria FOLLOW(S) = { $,), , } e FOLLOW(L) = {), , }.)

(b) **No. The productions for L have problems. The first one is left-recursive and can't be used directly. Also, the FIRST sets of the two productions for L are the same.**

(c) The standard fix would be to rewrite the productions for L as follows:

```
L ::= S Ltail
Ltail ::= , S Ltail | ε
```

(The last symbol is the Greek letter epsilon ε, i.e. the empty production.)

I.2 — Segunda Prova do André · prova do meio (gabarito oficial, inglês)

A Segunda Prova do André também veio com gabarito resolvido em inglês. Conteúdo: estruturas de dados em tempo de execução (vtables — Questão 1), coloração de grafos (Questão 7), sintaxe abstrata e semântica estática de MiniJava (Questão 7) e "x86 hacking" (Questão 5): quadro de pilha e tradução para linguagem de montagem.

Questão 1 (14 pontos)

Runtime data structures. Suppose we have the following three Java classes:

```
javapublic class Marsupial {
 int weight;
 public void eat() { ... }
 public void speak() { ... }
}

public class Wombat extends Marsupial {
 boolean happy;
 int age;
 public void play() { ... }
 public void sleep() { ... }
 public void speak() { ... }
}

public class Main {
 public static void main(String[] ignored) {
 Marsupial fred = new Marsupial();
 Marsupial matilda = new Wombat();
 Wombat mate = new Wombat();
 fred.eat();
 matilda.speak();
 mate.play();
 }
}
```

On the next page draw a diagram of the runtime data structures for this program after the declarations in the Main method have been processed, as follows:

(a) Draw pictures showing the variables in the program, the objects they refer to, and how the objects and their data members would be organized and laid out in memory.

(b) Add to your diagram from part (a) any mechanisms that support dynamic method binding as in Java (e.g., vtables). You may assume that this class structure is fixed at compile time and no new classes or methods will be added at runtime. You may also ignore constructors.

Resolução

Diagrama do gabarito (texto):

```
fred -> objeto Marsupial { vtable, weight }
matilda -> objeto Wombat { vtable, weight, happy, age }
mate -> objeto Wombat { vtable, weight, happy, age }

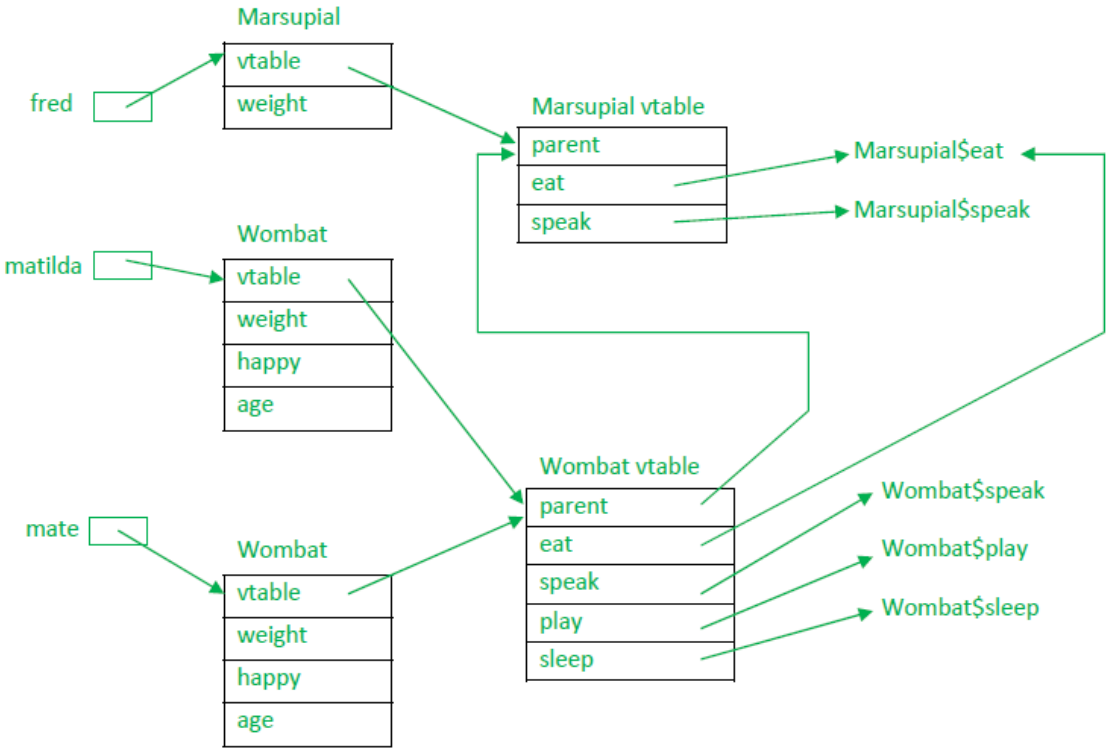
Marsupial vtable: { parent,
 eat -> Marsupial$eat,
 speak -> Marsupial$speak }

Wombat vtable: { parent -> Marsupial vtable,
 eat -> Marsupial$eat, (herdado)
 speak -> Wombat$speak, (override)
 play -> Wombat$play,
 sleep -> Wombat$sleep }
```

Cada objeto guarda, no offset 0, o ponteiro para a vtable da sua classe; os campos seguem na mesma ordem nas classes base e derivadas. `matilda` (estaticamente `Marsupial`) chama `Wombat$speak` pelo dispatch virtual; `mate.play()` usa `Wombat$play`.

Notes (gabarito): There is some freedom of the order of fields in the objects and vtables. But the first fields in Wombat objects must match the fields in Marsupial objects, and the order of methods in the Marsupial vtable must match the order of the methods in the beginning of the Wombat vtable exactly.

Question 1 (cont). Draw your diagram for question 1 below.



Notes: There is some freedom of the order of fields in the objects and vtables. But the first fields in Wombat objects must match the fields in Marsupial objects, and the order of methods in the Marsupial vtable must match the order of the methods in the beginning of the Wombat vtable exactly.

Questão 7 (12 pontos) — A little coloring

Considering the following code fragment:

```
a = read();
b = read();
c = a+b;
if (c > 0) {
 d = b+c;
} else {
 d = c+1;
}
print(d);
```

(a) Draw the control flow graph for the code, keeping the diagram to the left side of the paper.

(b) To the right of the control flow graph, neatly show the live ranges of the variables.

(c) Below, draw the interference graph for the variables. Use the left side of the paper.

(d) To the right of the interference graph, indicate which groups of variables can occupy the same register, based on the information in the interference graph. You do not need to go through the steps of the graph coloring algorithm explicitly, although it may be helpful as a guide to assigning registers. If there is more than one possible answer that uses the minimum number of registers, any of them will be fine.

Resolução

(a) CFG:

```
a = read();
 v
b = read();
 v
c = a + b;
 v
c > 0
 / \
/ \
d = b+c; d = c+1;
 \ /
 \ /
print(d);
```

(b) Live ranges: `a` — de `a = read()` até o uso em `c = a+b`; `b` — de `b = read()` até o uso em `d = b+c` (ramo verdadeiro); `c` — de `c = a+b` até os usos nos dois ramos; `d` — de sua definição até `print(d)`.

(c) Interference graph: arestas `a — b` (ambos vivos em `c = a+b`) e `b — c` (vivos em `d = b+c`); `d` fica isolado.

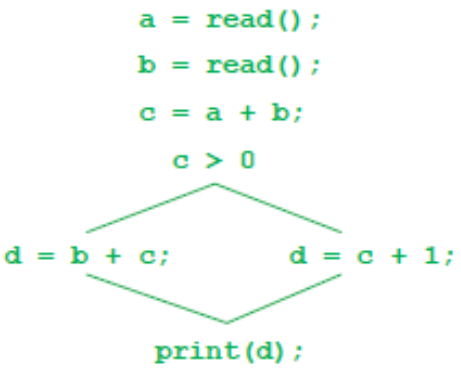
(d) Need two registers. One possibility:

```
r1 = { a, c, d }
r2 = { b }
```

Question 7. (12 points) A little coloring. Considering the following code fragment:

```
a = read();
b = read();
c = a+b;
if (c > 0) {
  d = b+c;
} else {
  d = c+1;
}
print(d);
```

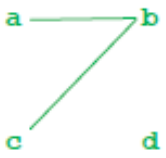
(a) Draw the control flow graph for the code, keeping the diagram to the left side of the paper.





(b) To the right of the control flow graph, neatly show the live ranges of the variables.

(c) Below, draw the interference graph for the variables. Use the left side of the paper.



Need two registers. One possibility:

r1 = { a, c, d }

r2 = { b }

(d) To the right of the interference graph, indicate which groups of variables can occupy the same register, based on the information in the interference graph. You do not need to go through the steps of the graph coloring algorithm explicitly, although it may be helpful as a guide to assigning registers. If there is more than one possible answer that uses the minimum number of registers, any of them will be fine.

Questão 7 (16 pontos) — Abstract syntax and semantics

Abstract syntax and semantics. Here is the abstract syntax for a fragment of a MiniJava program:

```
while
├── &&
│   ├── id: p
│   └── id: q
└── assign
    ├── id: n
    └── int: 17
```

(a) (4 points) What is the Java source code that corresponds to this abstract syntax? i.e., what is the original concrete syntax fragment that the scanner and parser read to produce the above tree, including all necessary punctuation to make it legal Java code?

(b) (12 points) What checks need to be performed in the static semantics/typechecking phase of the compiler to verify that this abstract syntax is, in fact, legal and contains no type errors or other static semantics problems? A good way to show your answer is to annotate the above diagram to show the checks that need to be performed at each location in the AST. Or you can write your answer below.

Resolução

(a)

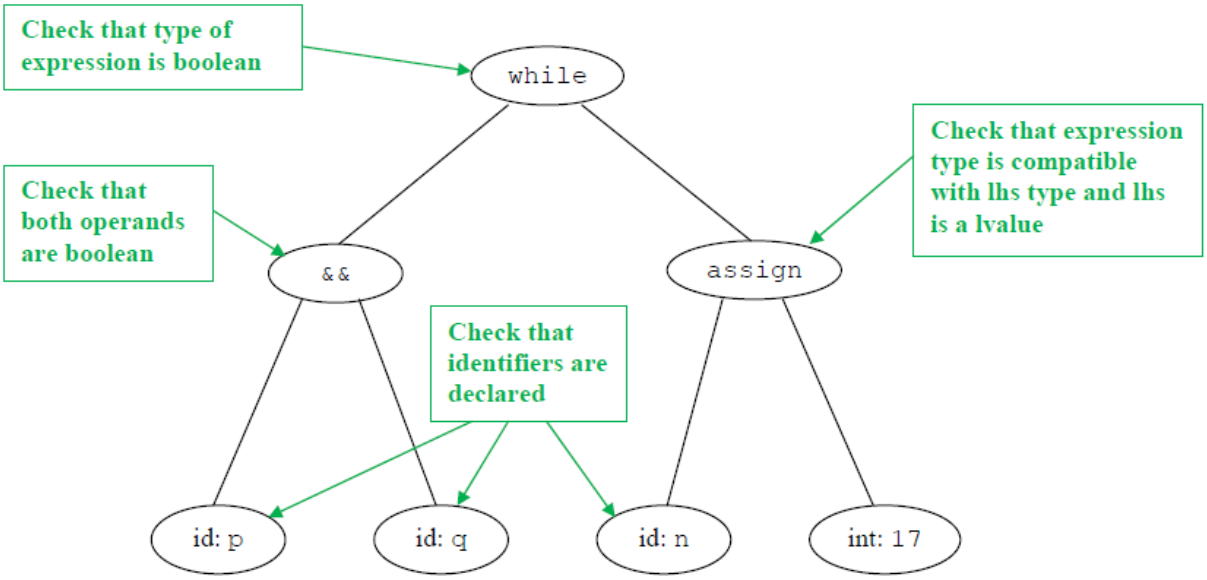
```
while (p && q)
 n = 17;
```

(b) Checks por nó da AST (como anotado no diagrama oficial):

- **while**: check that type of expression is boolean.
- **&&**: check that both operands are boolean.
- **assign**: check that expression type is compatible with lhs type and lhs is an lvalue.
- **id: p, id: q, id: n**: check that identifiers are declared. (`int: 17` não precisa.)

Diagrama oficial anotado:

Question 7. (16 points) Abstract syntax and semantics. Here is the abstract syntax for a fragment of a MiniJava program.



(a) (4 points) What is the Java source code that corresponds to this abstract syntax? i.e., what is the original concrete syntax fragment that the scanner and parser read to produce the above tree, including all necessary punctuation to make it legal Java code?

```
while (p && q)
  n = 17;
```

(b) (12 points) What checks need to be performed in the static semantics/typechecking phase of the compiler to verify that this abstract syntax is, in fact, legal and contains no type errors or other static semantics problems? A good way to show your answer is to annotate the above diagram to show the checks that need to be performed at each location in the AST. Or you can write your answer below.

See diagram above.

Questão 5 (20 pontos) — x86 hacking

x86 hacking. Consider the following C function that returns the sum of a sequence of numbers recursively:

```
c/* return the sum k + (k+1) + (k+2) + ... + n */
int sum(int k, int n) {
 int ans;
 if (k == n) {
 ans = n;
 } else {
 ans = k + sum(k+1, n);
 }
 return ans;
}
```

(a) (6 points) In the space below draw a picture of the stack frame for function `sum` right before executing the return statement at the end of the function. Your picture should show where the parameters and variables are located, as well as any additional items that are part of the stack frame, such as the return address. You should also draw labeled arrows showing where in the stack frame the registers ebp (frame pointer) and esp

(stack pointer) point, and indicate the numeric offset of each parameter and local variable from the frame pointer ebp.

(b) (14 points) Translate the sum function into x86 assembly language. Your code does not need to look like the code generated by your compiler — any clean x86 code will do. However, your code must conform to the standard x86 C language calling conventions. Further, your code must include all of the statements in the original function, including the assignments to the local variable ans and the recursive function call. You may use either the Intel or GNU assembler syntax for your code — just be sure to pick one and not mix them. Note: The standard conventions require that registers ebx, esi, edi, ebp, and esp must be saved and restored if they are used in the body of a function. Code repeated for reference:

```
c/* return the sum k + (k+1) + (k+2) + ... + n */
int sum(int k, int n) {
 int ans;
 if (k == n) { ans = n; } else { ans = k + sum(k+1, n); }
 return ans;
}
```

Resolução

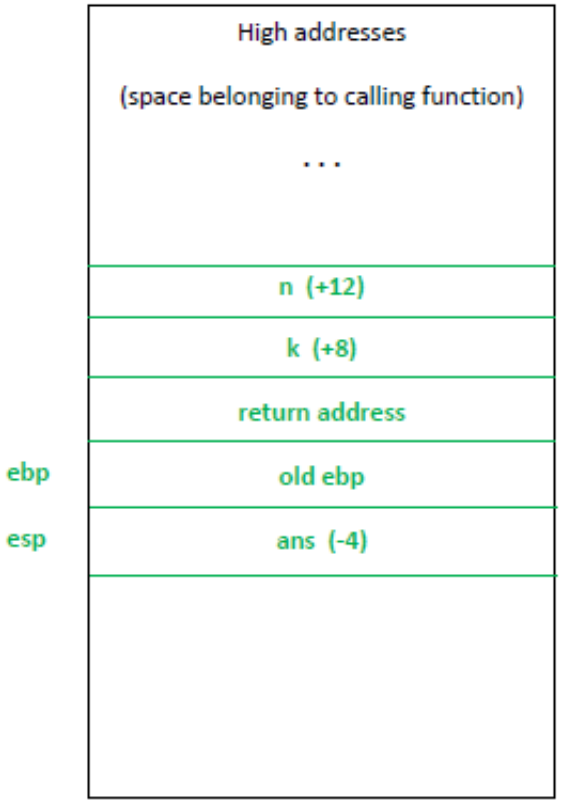
(a) Stack frame logo antes do `return` (ebp/esp como no gabarito):

```
+--------------------------------------+
| High addresses |
| (space belonging to calling function) |
| ... |
|--------------------------------------|
| n (+12) |
|--------------------------------------|
| k (+8) |
|--------------------------------------|
| return address |
|--------------------------------------|
| old ebp <- ebp |
|--------------------------------------|
| ans (-4) <- esp |
| (empty region) |
+--------------------------------------+
```

Question 5. (20 points) x86 hacking. Consider the following C function that returns the sum of a sequence of numbers recursively.

```
/* return the sum k + (k+1) + (k+2) + ... n */
int sum(int k, int n) {
   int ans;
   if (k == n) {
      ans = n;
   } else {
      ans = k + sum(k+1, n);
   }
   return ans;
}
```

(a) (6 points) In the space below draw a picture of the stack frame for function `sum` right before executing the `return` statement at the end of the function. Your picture should show where the parameters and variables are located, as well as any additional items that are part of the stack frame, such as the return address. You should also draw labeled arrows showing where in the stack frame the registers `ebp` (frame pointer) and `esp` (stack pointer) point, and indicate the numeric offset of each parameter and local variable from the frame pointer `ebp`.



(b) Resposta oficial (Intel syntax):

```
asm
;; answer using intel syntax

sum: push ebp ; prologue
 mov ebp,esp
 sub esp,4 ; frame with space for ans
 mov eax,[ebp+12] ; eax = n
 mov edx,[ebp+8] ; edx = k
 cmp eax,edx
```

```
 jne else ; jump if n != k

mov [ebp-4],eax ; ans = n
 jmp exit ; return
else: mov eax,[ebp+12] ; call sum(k+1,n)
 push eax ; push n
 mov eax,[ebp+8]
 inc eax
 push eax ; push k+1
 call sum ; recursive call - result in eax
 add esp,8 ; pop arguments
 add eax,[ebp+8] ; add k to result
 mov [ebp-4],eax ; store ans
exit: mov eax,[ebp-4] ; return ans in eax
 mov esp,ebp ; standard return
 pop ebp
 ret
```

There are obviously many possible solutions to this question. The above is fairly straightforward code, and definitely not the most compact possible.

Although the instructions said to include all of the original code in the solution, many people used eax to hold variable ans without storing it in the stack frame. We let that go when grading.

I.3 — Outra primeira prova: Prova Parcial 1 - 2023.1 · prova do meio

A "Outra primeira prova do André" é a Prova Parcial 1 - 2023.1, em português (fotos do gabarito impresso com anotações do aluno). Abaixo, o enunciado transcrito e a resolução correta, passo a passo, de cada questão. Para a Questão3(a) e (b)/(c) reutilizamos as figuras oficiais do gabarito da Primeira Prova (gramática idêntica).

Questão 1 — Conjuntos FIRST e FOLLOW

```
Considere as seguintes gramáticas:
a. A -> BC
B -> Ax | x
C -> yC | y
b. A -> BC
B -> Ax | x | ε
C -> yC | y
c. A -> BC
B -> Ax | x | ε
C -> yC | y | ε
Dê os conjuntos First e Follow de cada gramática.
```

Resolução

FIRST: fechamento descendente (fixpoint, começando vazio); FOLLOW: se X -> αAβ, FIRST(β)\{ε} vai para FOLLOW(A); se β é anulável, FOLLOW(X) também; símbolo inicial tem $.

```
Gramática a)
| Não-terminal | FIRST | FOLLOW |
| A | {x} | {x, $} |
| B | {x} | {y} |
| C | {y} | {x, $} |

Gramática b) — B anulável, C não
| Não-terminal | FIRST | FOLLOW |
| A | {x, y} | {x, $} |
| B | {x, y, ε} | {y} |
| C | {y} | {x, $} |

Gramática c) — B e C anuláveis
| Não-terminal | FIRST | FOLLOW |
| A | {x, y, ε} | {x, $} |
| B | {x, y, ε} | {x, y, $} |
| C | {y, ε} | {x, $} |
```

a) FIRST(A) = FIRST(B) = {x}: a alternativa `B -> x` quebra a circularidade A <-> B e C não é anulável.

FOLLOW(B) = FIRST(C) = {y}; FOLLOW(C) = FOLLOW(A) = {x, $}.

b) Como B deriva ε, FIRST(A) também recebe FIRST(C) = {y}, logo FIRST(A) = {x, y} e FIRST(B) = {x, y, ε}. O ε de B não muda FOLLOW(B), pois o sucessor de B é C, que não é anulável.

c) Com C anulável, FIRST(A) ganha ε e FOLLOW(B) passa a receber FOLLOW(A), logo FOLLOW(B) =

```
{x, y, $}.
```

Questão 2 — Expressões regulares

2. Dê expressões regulares para os seguintes conjuntos de palavras. Você deve usar somente expressões regulares básicas formadas por caracteres e épsilon (ε), classes de caracteres representando um único caractere ([...] e [^...]), concatenação (xy), escolha (x|y), repetição (x* e x+), e opcional (x?). Você pode também dar nomes a subexpressões (nome = re) e usar parênteses para agrupamento.

a) Identificadores formados conforme: um identificador consiste de um ou mais letras (a-z e A-Z), dígitos (0-9) e underscore (_). Um identificador precisa começar com uma letra e não deve terminar com um

underscore.

b) Identificadores de arquivos remotos da forma usuário@servidor:nome_do_arquivo, conforme: as partes do identificador são formadas por palavras, que são sequências de um ou mais letras e dígitos. A parte usuário contém somente uma palavra. Um servidor consiste de uma ou mais palavras separadas por pontos, como www.google.com ou attu. O nome_do_arquivo consiste de uma ou mais palavras separadas por caracteres de barra (/) com, opcionalmente uma barra no começo e/ou no fim (convenção normal do Unix). A parte usuário@ é opcional e pode ser omitida. Toda a parte usuário@servidor: pode ser omitida, incluindo os dois pontos do final. A parte usuário@ não deve aparecer a não ser que exista a parte servidor:.

Resolução

a)

```
letra = [a-zA-Z]
alnum = [0-9a-zA-Z]
ident = letra ( [0-9a-zA-Z_]* alnum )?
```

Começa com letra; o corpo aceita letras, dígitos e _, mas o grupo final obriga o último símbolo a ser letra ou dígito — assim nenhum identificador termina em _.

b)

```
palavra = [0-9a-zA-Z]+
servidor = palavra ( . palavra )*
caminho = /? palavra ( / palavra )* /?
ident = ( palavra @ )? servidor : caminho | caminho
```

A forma completa é `[usuário@]servidor:caminho`; a segunda alternativa cobre a omissão de toda a parte `usuário@servidor:`. Como `@` só existe dentro de `( palavra @ )?`, que é sempre seguido de `servidor :`, `usuário@` nunca aparece sem `servidor:`.

Questão 3 — Gramática de expressões: máquina LR(0), LR(0)? e SLR?

Questão repetida. Este enunciado e esta resolução são idênticos aos de **Questão 2 (24 pontos)**, de **I.1 — Primeira Prova do André · prova do meio (gabarito oficial, inglês)** (página 6). Não há nada novo aqui — se você já copiou aquela, pode pular esta.

Questão 4 — Por que parsing e verificação semântica em fases separadas?

4. Compiladores quase sempre fazem o parsing e a verificação semântica/type em fases separadas de compilação. Dê duas razões distintas do porque há esta separação em fases. Suas razões podem ser exemplos técnicos de coisas que só podem ser feitas em uma das fases mas não na outra, ou razões de engenharia mostrando que é uma boa prática, ou outros argumentos justificáveis.

Resolução

1) Limite técnico: a análise sintática usa gramática livre de contexto, e regras como declarar antes de usar, compatibilidade de tipos e número/tipos de argumentos não são context-free — exigem tabela de símbolos, escopo e tipos, que o parser (LL/LR) não tem.

2) Engenharia: o parser costuma ser gerado por ferramentas (yacc/CUP/bison) a partir da gramática, enquanto a análise semântica é código escrito à mão; separar as fases mantém cada parte testável e depurável isoladamente e melhora a qualidade dos mensagens de erro.

2. Provas oficiais CSE401 — midterms

Classificação herdada da pasta de origem “Provas mid” (inverno 2008, 2009, 2010 e outono 2010).

M1 — CSE401 midterm inverno 2008

Nesta prova podem ser utilizados os seguintes conjuntos alfabéticos. Alpha ::= a | b | c | d | e | f | g | h | i | j | k | l | m | n | o | p | q | r | s | t | u | v | w | x | y | z | A | B | C | D | E | F | G | H | I | J | K | L | M | N | O | P | Q | R | S | T | U | V | W | X | Y | Z
Num ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

Questão 1 (10 pontos)

NOTA: meta-regras como [] ("zero ou um") não são permitidas nesta questão; você pode usar ε sempre que necessário.

(A) A base de um nome de arquivo é qualquer sequência "corretamente hifenizada" de letras ou dígitos: não começa nem termina com hífen e não tem hífens consecutivos (ex.: i8-a-Hot-dog). A base deve ter pelo menos um caractere. Escreva uma expressão regular para

```
file_name_base ::=
```

(B) Um nome de arquivo é uma ou mais bases, cada uma separada da seguinte por um ponto seguido, opcionalmente, de outro ponto e de uma extensão. Uma extensão tem exatamente três letras. Se o nome contém ponto, então obrigatoriamente há extensão: "a" é nome válido; "a.b" não é; "a.b.doc" é. Escreva uma expressão regular para

```
filename ::=
```

Resolucão

```
file_name_base ::= (Alpha|Num)+ (-(Alpha|Num)+)*
filename ::= file_name_base ( ε | ( . file_name_base )* . Alpha Alpha Alpha )
```

A base usa `(...)+` agrupado antes de cada hífen, o que impede hífen no início, no fim e hífens consecutivos. Em `filename`, a segunda alternativa obriga o ponto final a ser seguido de exatamente três letras, garantindo que "a.b" seja rejeitado e "a.b.doc" seja aceito.

Questão 2 (8 pontos)

No compilador MiniJava, classificamos os tokens em grupos importantes. Dê dois exemplos para cada grupo: palavra-chave, delimitador, operador, token com valor.

Resolucão

```
palavra-chave: if, class
Delimitador: (, )
Operador: +, &&
Token com valor: IDENTIFIER, INT_LITERAL
```

Questão 3 (10 pontos)

Dê (a) a árvore de sintaxe concreta e (b) a árvore de sintaxe abstrata (nós estilo MiniJava) para `(c + a) * b`, usando a gramática:

```
E ::= E + T | T
T ::= T * F | F
F ::= id | ( E )
```

Resolução

(a) Árvore de sintaxe concreta (raiz `E`, todos os símbolos da gramática):

```
E
 |
 T

/ | \
 T * F
 | |
 F id(b)
 |
 ( E )
 /|\
 E + T
 | |
 T F
 | |
 F id(a)
 |
 id(c)
```

(b) Árvore de sintaxe abstrata (só os nós de operadores e identificadores; parênteses não geram nó):

```
MulExpr
 / \
 AddExpr VarExpr(b)
 / \
VarExpr VarExpr(c) VarExpr(a)
```

Erros clássicos: enraizar em `T` em vez de `E`, deixar nós `E`/`T` sobrando na AST, rotular o operador só com `+` em vez de `AddExpr`/`MulExpr` e esquecer o `VarExpr` dos identificadores.

Questão 4 (16 pontos)

```
Dada a gramática S ::= aS | aSbS | c, use exemplos ou definições para justificar sua resposta:
```

a) É recursiva à esquerda? sim / não — Por quê?

b) É adequada para análise top-down (parsing preditivo)? sim / não — Por quê?

c) É ambígua? sim / não — Por quê?

d) É regular? sim / não — Por quê?

Resolução

a) **Não.** `S` nunca aparece como primeiro símbolo do lado direito de nenhuma produção (os lados direitos começam por `a` ou `c`).

b) **Não.** As produções `aS` e `aSbS` têm o prefixo comum `aS`, o que gera conflito em parsers LL(1); seria

```
preciso fatorar (S ::= aS' | c, S' ::= S | bS).
```

c) **Sim.** Há duas árvores de análise diferentes para `aacbc`:

```
1) S -> aSbS, esq: S -> aS -> aac, dir: S -> c (a . aac . b . c)
2) S -> aS, int: S -> aSbS -> acbc (a . acbc)
```

d) **Sim, a linguagem é regular.** Toda cadeia gerada tem a forma `c | a+ c (b a* c)*`: todo `b` é sempre precedido por `c` e seguido de `a* c`, e a cadeia começa por `a` (ou é só `c`). Isso é descrito por uma expressão regular, logo a linguagem é regular. (O argumento de contagem "#a deve ser > #b" não vale:

`acbc` está na linguagem com #a = #b.)

Questão 5 (14 pontos)

Dada a gramática

```
s ::= expr $
expr ::= a | a subs
subs ::= [ expr ] | [ expr ] subs
```

Construa o primeiro par de estados do DFA de um parser LR para essa gramática:

a) Calcule o fechamento do item `s ::= . expr $` no Estado 1.

b) Desenhe e rotule as arestas que saem do Estado 1.

c) Mostre o fechamento completo dos estados alcançáveis por essas arestas.

d) Indique algo especial sobre os estados (conflitos, estados de redução). Não desenhe arestas dos demais estados — 2 a 5 estados no total; não é preciso montar o DFA inteiro.

Resolucão

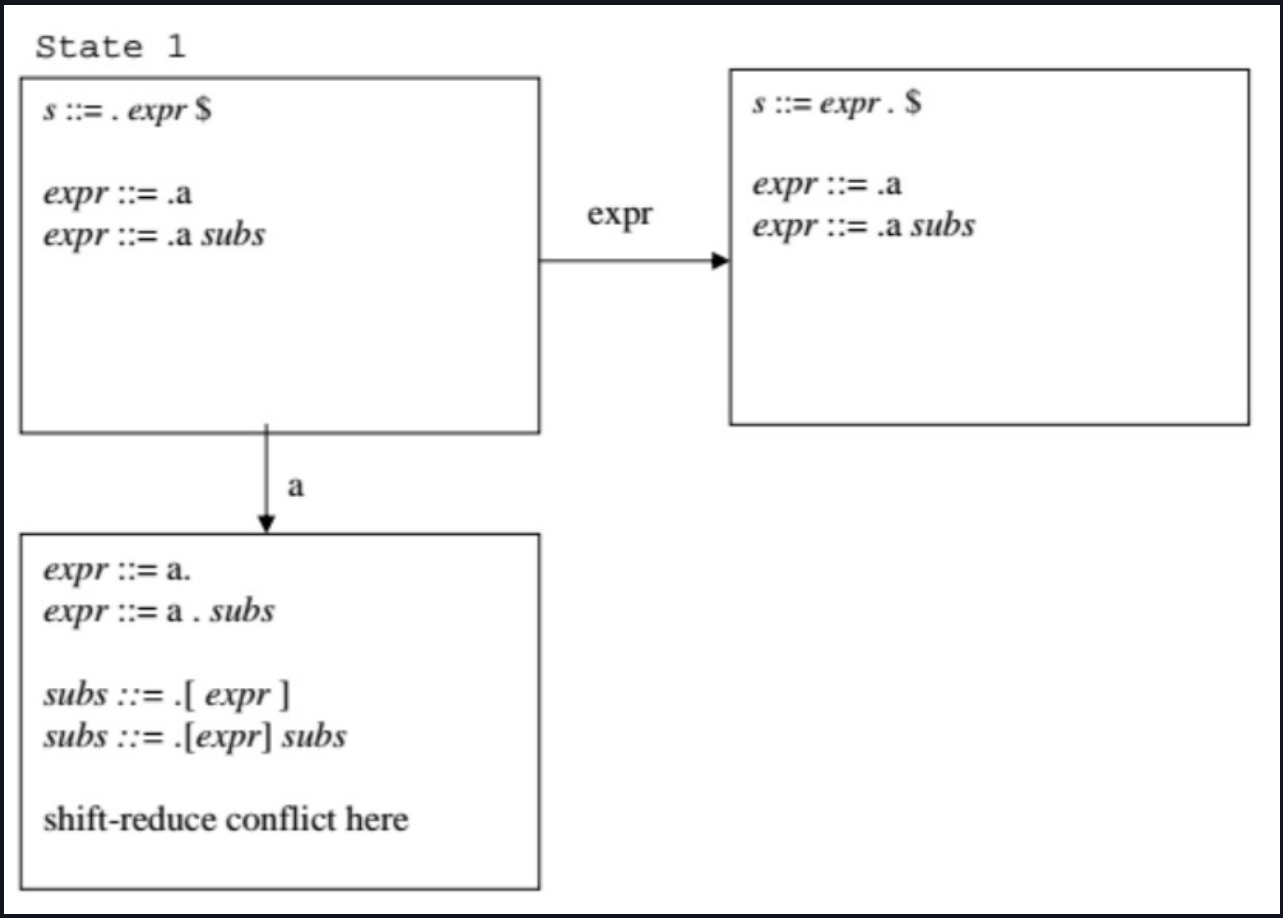
```
Estado 1 (I0 = fecho de s ::= . expr $):
 s ::= . expr $
 expr ::= . a
 expr ::= . a subs

I0 -a-> Estado 2 (fecho de expr ::= a .):
 expr ::= a .
 expr ::= a . subs
 subs ::= . [ expr ]
 subs ::= . [ expr ] subs

Estado 2 -[-- > Estado 3 (fecho de subs ::= [ . expr ]):
 subs ::= [ . expr ]
 subs ::= [ . expr ] subs
 expr ::= . a
 expr ::= . a subs

Estado 3 -a-> estado idêntico ao Estado 2
```

O Estado 1 tem só a aresta `a` (nenhum item permite `[` ali). O Estado 2 é de redução (`expr ::= a .`) e também desloca `[`; o Estado 3 fecha os itens de `expr` novamente.



Questão 6 (12 pontos)

Queira-se adicionar a declaração condicional abaixo ao MiniJava:

```
ifequal ( exp1 , exp2 ) instrucao1 smaller instrucao2 larger instrucao3
```

`instrucao1` executa se exp1 = exp2; `instrucao2`, se exp1 < exp2; `instrucao3`, se exp1 > exp2. `ifequal`, `smaller` e `large` são palavras-chave.

(A) [5] Dê a(s) produção(ões) de gramática livre de contexto para `ifequal` permitindo que as partes `smaller` e `larger` (ou ambas) sejam omitidas. Se ambas aparecem, nessa ordem. Não precisa dar produções de expressões ou de outras declarações (mas `ifequal` deve ser uma declaração).

(B) [5] A gramática da parte (a) é ambígua? Se não, explique; se sim, dê um exemplo.

(C) [2] Ao compilar essa declaração, que regra(s)/condição(ões) o verificador de tipo verifica?

Resolução

(A) Uma solução com produções de ε:

```
stmt ::= ifequal ( exp , exp ) stmt optsmaller optlarger
optsmaller ::= smaller stmt | ε
optlarger ::= larger stmt | ε
```

(Solução alternativa sem ε: quatro produções de `stmt`, uma para cada combinação de partes ausentes/presentes.)

(B) **Sim, é ambígua** — mesmo problema do `else` pendente. Para `ifequal ( exp , exp ) ifequal ( exp , exp ) stmt smaller stmt`, `smaller stmt` pode ser associado ao `ifequal` interno (fechando-o) ou ao externo, dando duas árvores.

(C) Verifica que `exp1` e `exp2` são ambos do tipo `int`.

Questão 7 (10 pontos)

Qual é a diferença entre escopo estático (lexical) e tipagem estática? Dê uma definição para cada um; use um exemplo em pseudocódigo se isso ajudar.

Resolução

Escopo trata da visibilidade de uma variável: qual instância correta vincular a um nome. **Tipagem** trata da determinação do tipo de um valor.

- **Escopo estático/lexical**: qual instância usar é decidido em tempo de compilação, pela estrutura léxica do programa (o que está visível no papel). **Escopo dinâmico**: decidido em tempo de execução, pelo aninhamento das chamadas de procedimento.
- **Tipagem estática**: tipos resolvidos em tempo de compilação. **Tipagem dinâmica**: tipos verificados em tempo de execução. (Tanto estática quanto dinâmica podem ser fortes ou fracas.)

Questão 8 (10 pontos)

Dê um exemplo mostrando a diferença entre equivalência estrutural e equivalência nominal.

Resolução

1) Equivalentes estruturalmente, mas não nominalmente:

```
struct { int x; } s1;
struct { int x; } s2; // mesmo layout: estruturalmente iguais,
 // mas tipos com nomes diferentes
```

2) Equivalentes nominalmente (e também estruturalmente):

```
typedef struct { int x; } S;
S s3;
S s4; // mesmo nome de tipo
```

A equivalência é sobre o **tipo** das variáveis, não sobre os valores armazenados nelas.

M2 — CSE401 midterm inverno 2009

Pergunta 1 (16 pontos)

Questão repetida. Este enunciado e esta resolução são idênticos aos de **Questão 1 (16 pontos)**, de **I.1 — Primeira Prova do André · prova do meio (gabarito oficial, inglês)** (página 6). Não há nada novo aqui — se você já copiou aquela, pode pular esta.

Pergunta 2 (16 pontos)

Dê uma descrição, em português, dos conjuntos de cadeias gerados pelas expressões regulares abaixo. Para crédito total, descreva o conjunto sem simplesmente reescrever a expressão em palavras. Exemplo:

para `(a* b*)*`, uma boa resposta é "todas as cadeias com 0 ou mais a's e b's, em qualquer sequência".

```
(a) (x|y)*x(x|y)

(b) p(p|q)*p
```

(c) Desenhe um DFA que aceita o mesmo conjunto de cadeias da parte (b).

Resolução

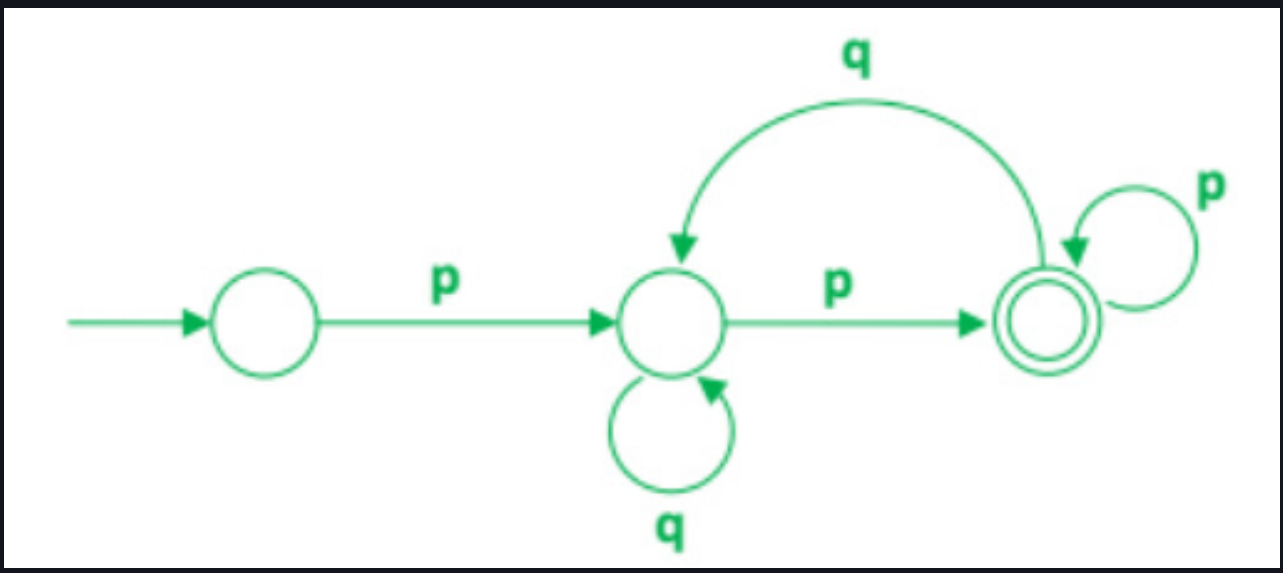
(a) Todas as cadeias de `x` e `y` cujo penúltimo caractere é `x` — ou seja, cadeias terminadas em `xx` ou `xy`. Cadeias de comprimento 1 não são aceitas (a expressão exige pelo menos dois caracteres).

(b) Todas as cadeias de `p` e `q` que começam e terminam com `p`.

(c) DFA com estados `q0` (início, não final), `q1` (final), `q2` (não final) e `q3` (estado morto):

```
q0 -p-> q1 q0 -q-> q3
q1 -p-> q1 q1 -q-> q2
q2 -p-> q1 q2 -q-> q2
q3 -p-> q3 q3 -q-> q3
```

`q1` significa "começou com `p` e termina com `p`"; `q2`, "começou com `p` mas termina com `q`".



Pergunta 3 (16 pontos)

Questão repetida. Este enunciado e esta resolução são idênticos aos de **Questão 2 — Expressões regulares**, de **I.3 — Outra primeira prova: Prova Parcial 1 - 2023.1 · prova do meio** (página 21). Não há nada novo aqui — se você já copiou aquela, pode pular esta.

Pergunta 4 (16 pontos)

Questão repetida. Este enunciado e esta resolução são idênticos aos de **Questão 4 (16 pontos)**, de **I.1 — Primeira Prova do André · prova do meio (gabarito oficial, inglês)** (página 10). Não há nada novo aqui — se você já copiou aquela, pode pular esta.

Pergunta 5 (20 pontos)

A linguagem de programação Pascal incluí ponteiros e registros (estruturas), assim como C. Como em C, x.y indicado na seleção de campo. Mas Pascal teve uma gramática muito mais saudável para as variáveis de ponteiro. O ponteiro de referencia do Pascal foi indicado por uma seta para a direita da variável. A expressão Pascal p ↑ significa o mesmo que * p em C, e a expressão de p- C> F (que é equivalente a (* p) .f), é representado por p .f ↑ em Pascal. Aqui está uma gramática para variáveis Pascal envolvendo seleção de campo e ponteiro de referência. O símbolo id é um terminal que representa um identificador simples. var' é o não-terminal adicional inserido para lidar com o fim-de-arquivo marcado por $.

0. var' ::= var $

1. var ::= var ↑

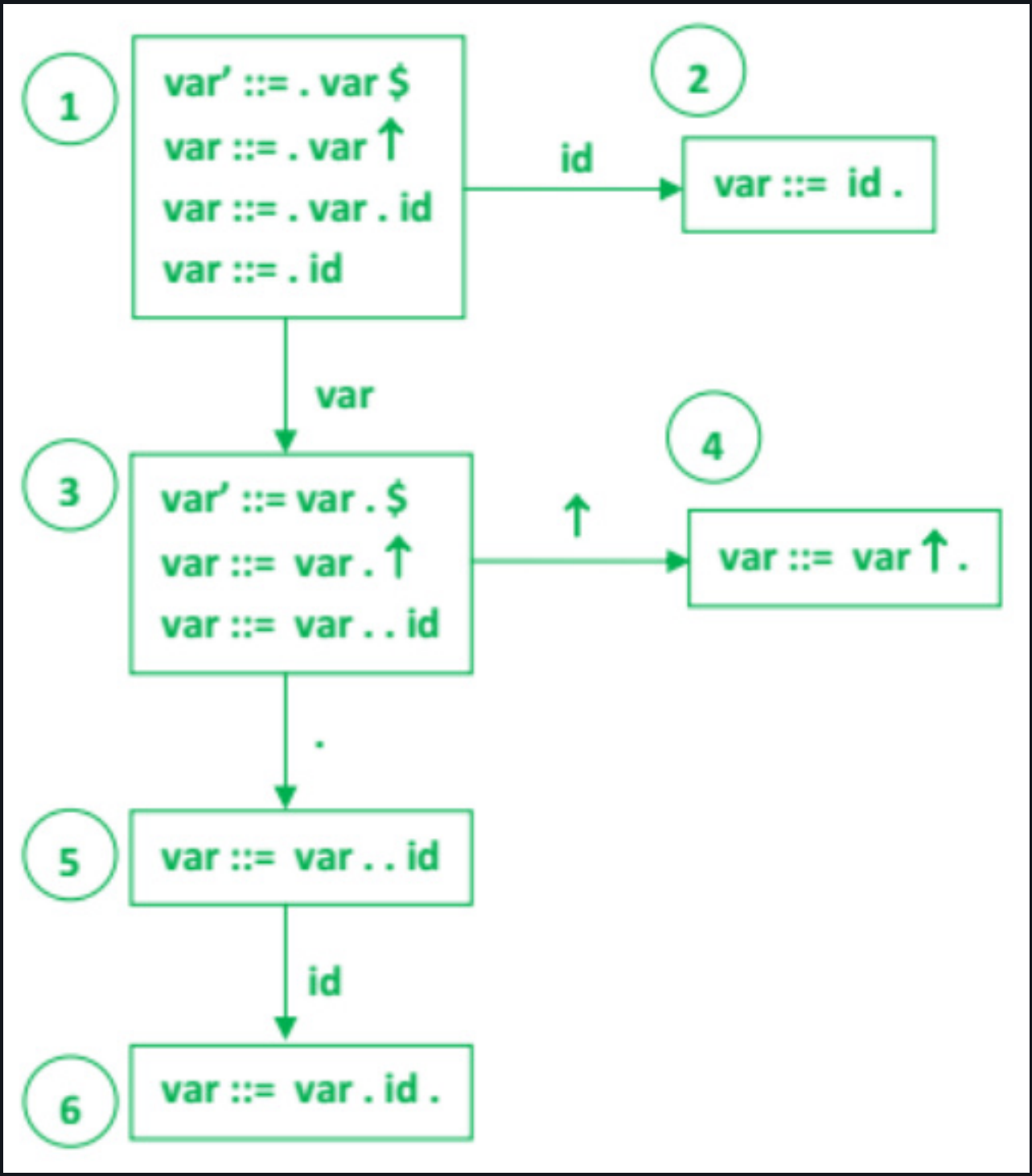
2. var ::= var . id

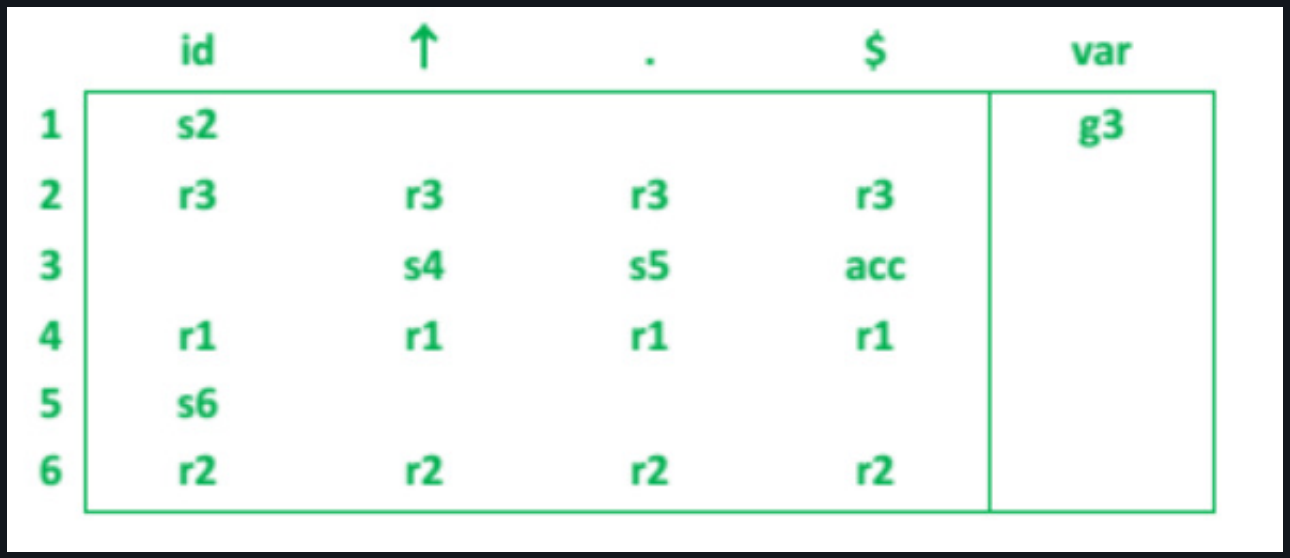
3. var ::= id No restante desta página, e (se necessário) a próxima página, faça o seguinte:

a) Construir o diagrama de estado LR(0) para esta gramática,

b) Construa a tabela de parse LR (0) correspondente ao seu diagrama da parte (a), e

c) Indique se esta gramática é LR (0). Justifique sua resposta. NÃO se esqueça de fazer esta parte!


1
var' ::= . var $
var ::= . var ↑
var ::= . var . id
var ::= . id
id
2
var ::= id .
var
3
var' ::= var . $
var ::= var . ↑
var ::= var . . id
↑
4
var ::= var ↑ .
.
5
var ::= var . . id
id
6
var ::= var . id .



	id	↑	.	$	var
1	s2				g3
2	r3	r3	r3	r3	
3		s4	s5	acc	
4	r1	r1	r1	r1	
5	s6				
6	r2	r2	r2	r2	

Resolução

(a) Diagrama de estados LR(0):

```
I0: var' ::= . var $
 var ::= . var ^
 var ::= . var . id
 var ::= . id
I0 -var-> I1: var' ::= var . $ , var ::= var . ^ , var ::= var . . id
I0 -id-> I2: var ::= id .
I1 -^-> I3: var ::= var ^ .
I1 -.---> I4: var ::= var . . id
I4 -id-> I5: var ::= var . id .
I1 -$-> aceitação
```

(b) Tabela LR(0) (deslocar/goto/reduzir):

Estado	id	^	.	$	var
I0	deslocar I2				goto I1
I1		deslocar I3	deslocar I4	aceitar	
I2	reduzir (3)	reduzir (3)	reduzir (3)	reduzir (3)	
I3	reduzir (1)	reduzir (1)	reduzir (1)	reduzir (1)	
I4	deslocar I5				
I5	reduzir (2)	reduzir (2)	reduzir (2)	reduzir (2)	

(c) **Sim, a gramática é LR(0).** Não há conflitos: `I0` e `I4` só deslocam, `I1` só desloca/aceita e `I2`, `I3`, `I5` são estados de redução puros (nenhum item de deslocar no mesmo estado).

Pergunta 6 (16 pontos)

Java inclui uma expressão condicional apenas como aquela em C, C ++ e outras linguagens. A expressão condicional tem a forma exp0? exp1: exp2. O valor de uma expressão condicional é o valor de exp1 ou exp2 dependendo do valor de exp0. Se exp0 é verdadeira, então o valor da expressão condicional é o valor de exp1, e exp2 não é avaliada. Se exp0 é falsa, então o valor da expressão condicional é o valor da exp2, e exp1 não é avaliado. Abaixo está uma árvore de sintaxe abstrata para a atribuição declaração max = x> y? x: y ;. Descrever a verificação de tipo necessária na fase semântica do compilador para verificar se não há erros de tipo nesta instrução de atribuição. Você pode assumir que os identificadores são declarados em outros lugares e que os seus tipos podem ser recuperados a partir de uma entrada da tabela de símbolos apropriado. Uma boa maneira de mostrar a sua resposta seria escrever ao lado de cada nó que tipo de verificação precisa ser feito naquele nó (se houver), e indicar o tipo da expressão resultante (sub) representado por esse nó (se houver).

Resolução

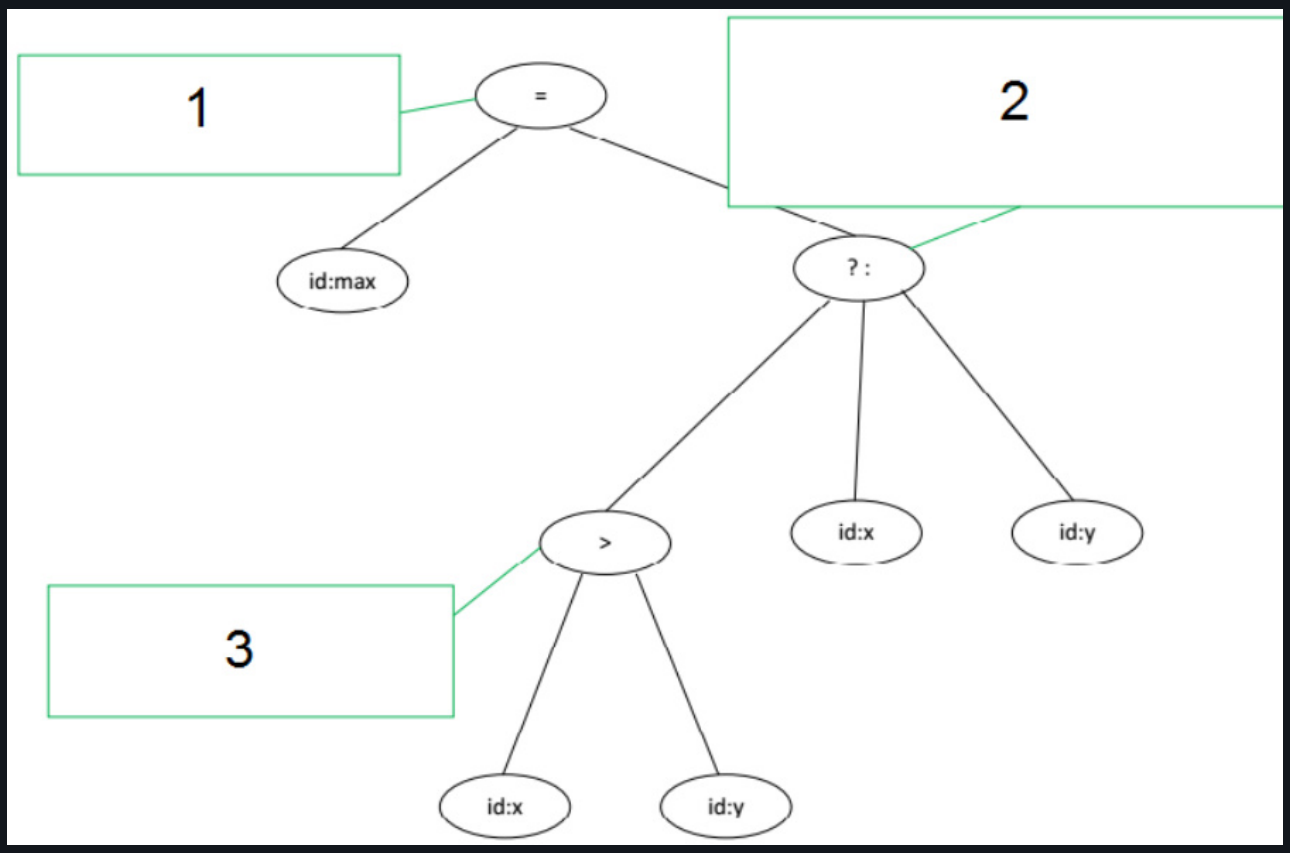
Checagens necessárias (todas locais a cada nó):

1. **Nó `?` (condicional):** o operando de condição (`x > y`) deve ser `boolean`; os dois ramos (`x` e `y`) devem ter o mesmo tipo ou ser coercíveis a um tipo comum `T`. O tipo resultante do nó é `T`.

2. **Nó `>`:** os operandos `x` e `y` devem ter tipos comparáveis por `>`. Resultado: `boolean`.

3. **Atribuição (`max = ...`):** o tipo da expressão à direita (o tipo comum `T` do `?:`) deve ser compatível com o tipo de `max`.

A verificação do nó `?:` é independente de ele estar no lado direito de uma atribuição.



M3 — CSE401 midterm inverno 2010

Questão 1 (18 pontos)

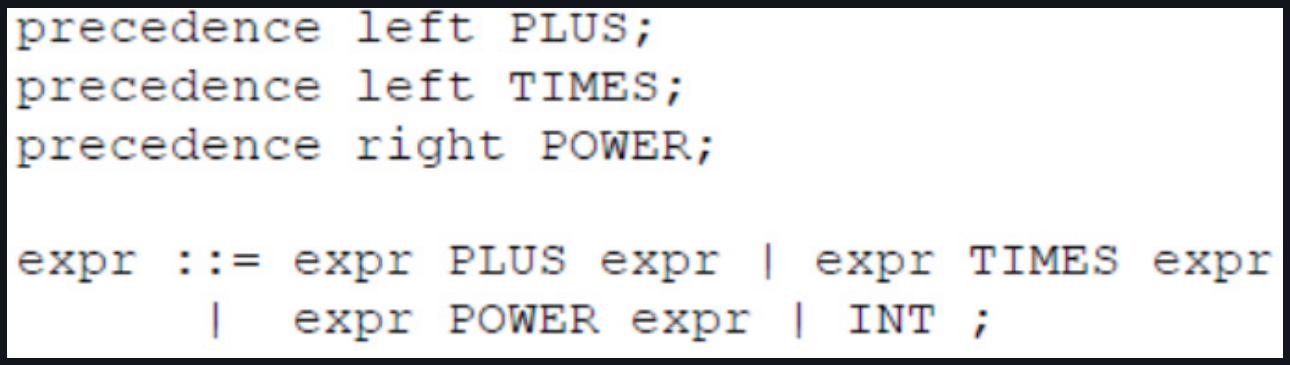
Questão repetida. Este enunciado e esta resolução são idênticos aos de **Questão 3 (18 pontos)**, de **I.1 — Primeira Prova do André · prova do meio (gabarito oficial, inglês)** (página 8). Não há nada novo aqui — se você já copiou aquela, pode pular esta.

Questão 2 (24 pontos)

Questão repetida. Este enunciado e esta resolução são idênticos aos de **Questão 2 (24 pontos)**, de **I.1 — Primeira Prova do André · prova do meio (gabarito oficial, inglês)** (página 6). Não há nada novo aqui — se você já copiou aquela, pode pular esta.

Questão 3 (16 pontos)

Ferramentas de análise. No CUP e ferramentas semelhantes, podemos usar declarações de precedência para resolver as ambiguidades e falhas de precedência em gramáticas. Aqui está uma especificação CUP para expressões inteiras com PLUS (+), TIMES (*), e POWER (^), onde PLUS e TIMES associados à esquerda (ou seja, a+b+c significa (a + b) + c) e POWER (exponenciação) associados à direita (ou seja, a^b^c significa a^(b^c)). Como de costume, TIMES tem precedência maior do que PLUS, e POWER tem a maior precedência. INT é o símbolo terminal para inteiros. Para este problema, dar uma gramática livre de contexto não ambígua, sem declarações de precedência, que gera expressões com a mesma precedência e associatividade como na especificação CUP acima. Você só precisa dar regras gramaticais comuns - eles não precisam ser uma especificação CUP formatada corretamente. (Dica: você pode achar ser útil introduzir não terminais adicionais em sua gramática.)



```
precedence left PLUS;
precedence left TIMES;
precedence right POWER;

expr ::= expr PLUS expr | expr TIMES expr
       |  expr POWER expr | INT ;
```

Resolucão

```
expr ::= term | expr PLUS term
term ::= expon | term TIMES expon
expon ::= INT | INT POWER expon
```

Cada nível da hierarquia implementa uma precedência: `expr` (PLUS, associado à esquerda), `term` (TIMES, associado à esquerda) e `expon` (POWER, associado à direita — a recursão é à direita, depois do `POWER`).

Questão 4 (10 pontos)

Questão repetida. Este enunciado e esta resolução são idênticos aos de **Questão 4 — Por que parsing e verificação semântica em fases separadas?**, de **I.3 — Outra primeira prova: Prova Parcial 1 - 2023.1 · prova do meio** (página 22). Não há nada novo aqui — se você já copiou aquela, pode pular esta.

Questão 5 (8 pontos)

Nós sugerimos o uso de pelo menos duas passagens separadas sobre a AST para verificar semântica e tipos em um programa MiniJava. Por que não fazê-lo em uma passagem? Existe alguma razão técnica pela qual duas passagens são necessárias, ou pelo menos muito útil?

Resolução

Em MiniJava, classes podem ser usadas antes de serem declaradas. A primeira passagem coleta os nomes de todas as classes (e seus membros); a segunda faz a verificação detalhada. Em uma única passagem, teríamos que voltar atrás e reavaliar usos anteriores assim que encontrássemos a declaração de uma classe cujo nome já tinha sido usado.

Questão 6 (8 pontos)

No Java completo, a declaração de um campo de uma classe pode ser precedida por vários modificadores. As possibilidades são: public, protected, private, static, final, transient, e volatile. Estes modificadores podem aparecer em qualquer ordem. Um compilador é necessário para verificar que, em qualquer declaração única nenhum modificador apareça mais de uma vez, e que no máximo um dos modificadores de acesso public, protected ou private é usado. Seria melhor colocar esta verificação no scanner, no parser, ou na parte semântica estática do compilador? Dar uma justificação técnica para a sua resposta.

Resolução

Na **semântica estática**. O scanner não pode, porque não tem contexto para interpretar os tokens. Dá para construir uma gramática que aceite só combinações legais, mas seriam necessárias regras para todas as permutações dos modificadores, deixando o parser enorme e com ações semânticas redundantes. O mais simples é o parser aceitar qualquer sequência de modificadores e a passagem semântica verificar (com um conjunto/contagem) que nenhum se repete e que há no máximo um de

```
public/protected/private.
```

Questão 7 (16 pontos)

Questão repetida. Este enunciado e esta resolução são idênticos aos de **Questão 7 (16 pontos) — Abstract syntax and semantics**, de **I.2 — Segunda Prova do André · prova do meio (gabarito oficial, inglês)** (página 15). Não há nada novo aqui — se você já copiou aquela, pode pular esta.

M4 — CSE401 midterm outono 2010

Pergunta 1 (20 pontos)

Constantes inteiras da linguagem Ada contêm um ou mais dígitos e podem ter sublinhados embutidos:

cada `_` deve ser precedido e seguido de pelo menos um dígito (logo, o número não começa nem termina com `_` e não há `__` consecutivos). Exemplos: `0`, `17`, `017`, `0_01_7`, `1_048_576`.

Números também podem ser escritos em outras bases, precedidos pela base e pelo caractere `#` (todos representam 15231): `15231`, `10#15231`, `16#3B7F`, `16#03B_7F`, `2#00111011_0111111`, `8#35577`. Exemplos do número 5: `5`, `2#00101`, `7#5`, `5#10`.

A base (que precede o `#`) está entre 2 e 16 e é escrita em decimal. Os dígitos são `0-9` e as letras `A-F` (somente maiúsculas). A linguagem exige dígito < base, mas ignore essa regra aqui: qualquer sequência de dígitos/`A-F` é aceita após o `#`; sem base, só `0-9`.

(A) (10 pontos) Dê uma expressão regular para constantes inteiras Ada. Você só pode usar concatenação, escolha (`r|s`), Kleene star (`r*`) e ainda `r+`, `r?`, classes de caracteres (`[ABW-z]`, `[^aeiou]`) e nomes para subexpressões (`vogal = [aeiou]`).

(B) (10 pontos) Desenhe um DFA que reconhece essas constantes, gerado pela expressão regular do item (A).

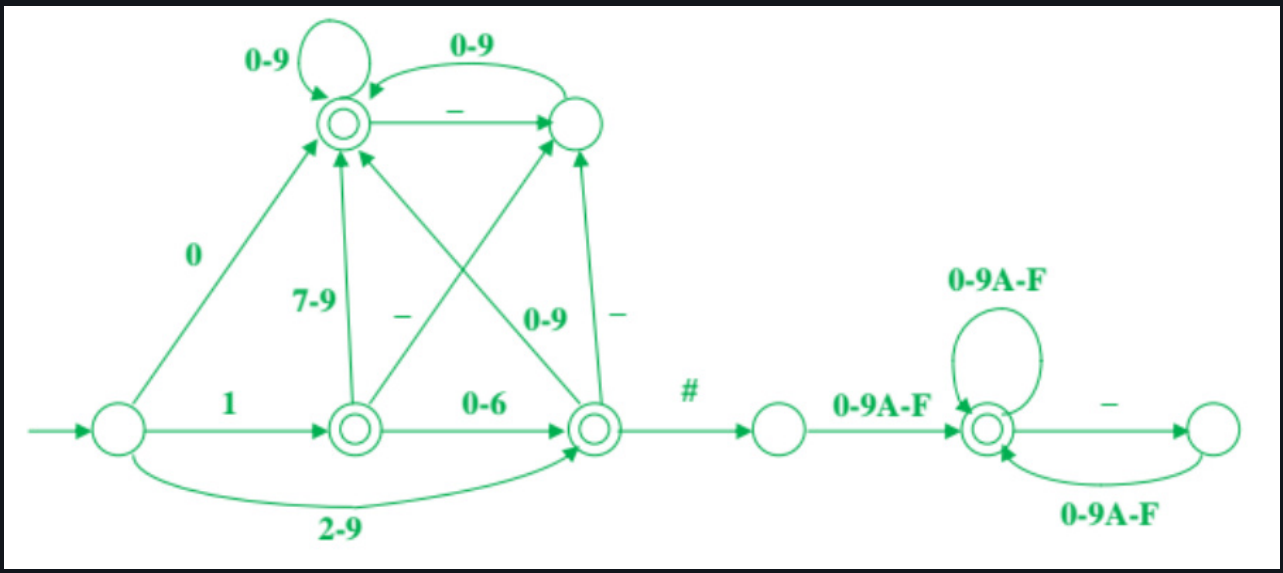
Resolução

(A)

```
decint = [0-9] ( _? [0-9] )*
basedint = ( 1[0-6] | [2-9] ) # [0-9A-F] ( _? [0-9A-F] )*
integer = decint | basedint
```

`(_?[0-9])*` exige dígito antes e depois de cada `_` (impede `_` inicial, final e `__`). A base usa `1[0-6] | [2-9]`, ou seja, 2 a 16 em decimal; após o `#`, os dígitos são `[0-9A-F]` com as mesmas regras de `_`.

(B) DFA correspondente: figura abaixo.



Pergunta 2 — Ambiguidade (14 pontos)

A sintaxe das expressões regulares pode ser definida por gramática livre de contexto. Uma gramática possível, sobre o alfabeto `{a, b}`:

```
R ::= R R (concatenação)
R ::= R | R (| é o operador escolha)
R ::= R * (fecho de Kleene)
R ::= ( R )

R ::= a
R ::= b
```

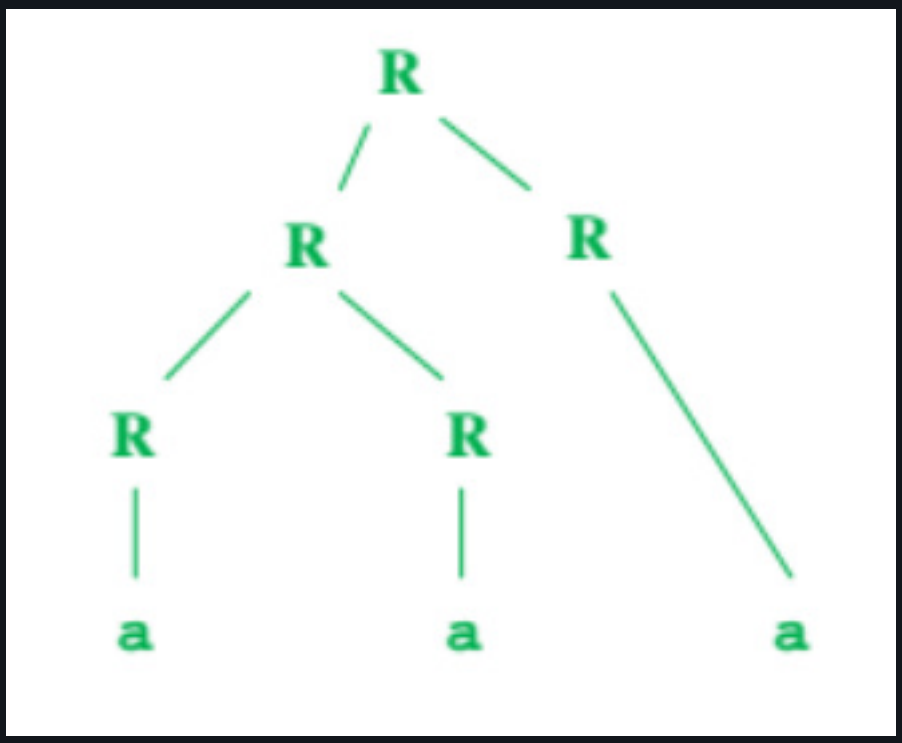
Mostre que essa gramática é ambígua.

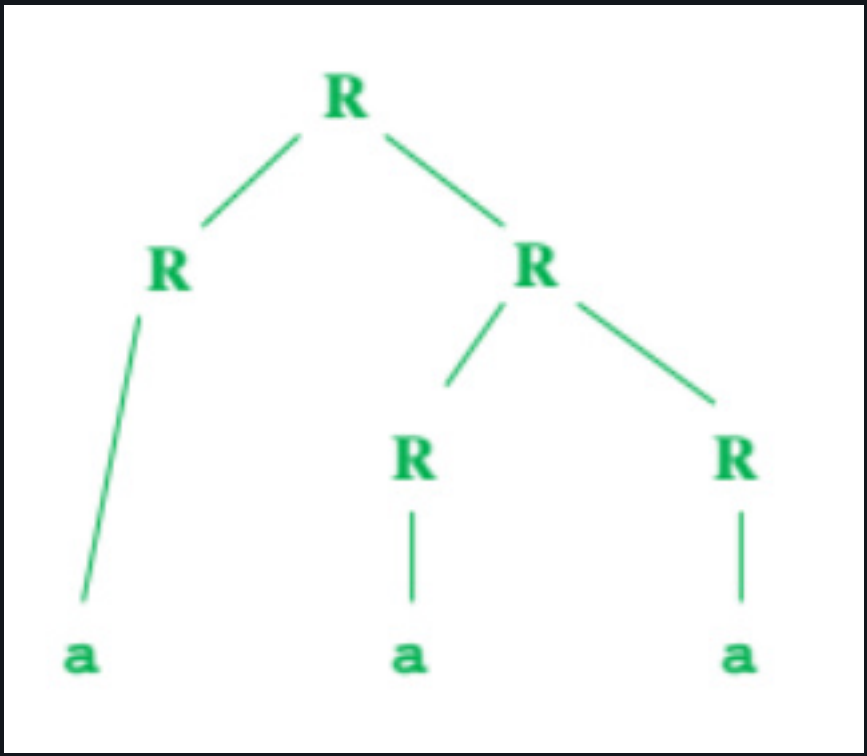
Resolução

Basta uma cadeia com duas derivações diferentes. Para `aaa`:

```
Árvore 1: R => R R => (R R) R => (a a) a = ((aa)a)
Árvore 2: R => R R => R (R R) => a (a a) = (a(aa))
```

Na árvore 1 a concatenação agrupa os dois primeiros `a`; na árvore 2, os dois últimos. Como há duas árvores de análise distintas para a mesma cadeia, a gramática é ambígua. As figuras abaixo mostram as duas árvores.





Pergunta 3 (30 pontos)

Gramática:

```
0. S' ::= S $
1. S ::= x S S
2. S ::= y
```

(A) (12 pontos) Desenhe a máquina de estados LR(0) para essa gramática.

(B) (12 pontos) Construa a tabela do parser LR(0) para essa gramática, a partir da máquina do item (A).

(C) (3 pontos) Esta gramática é LR(0)? Por quê?

(D) (3 pontos) Esta gramática é SLR? Por quê?

Resolução

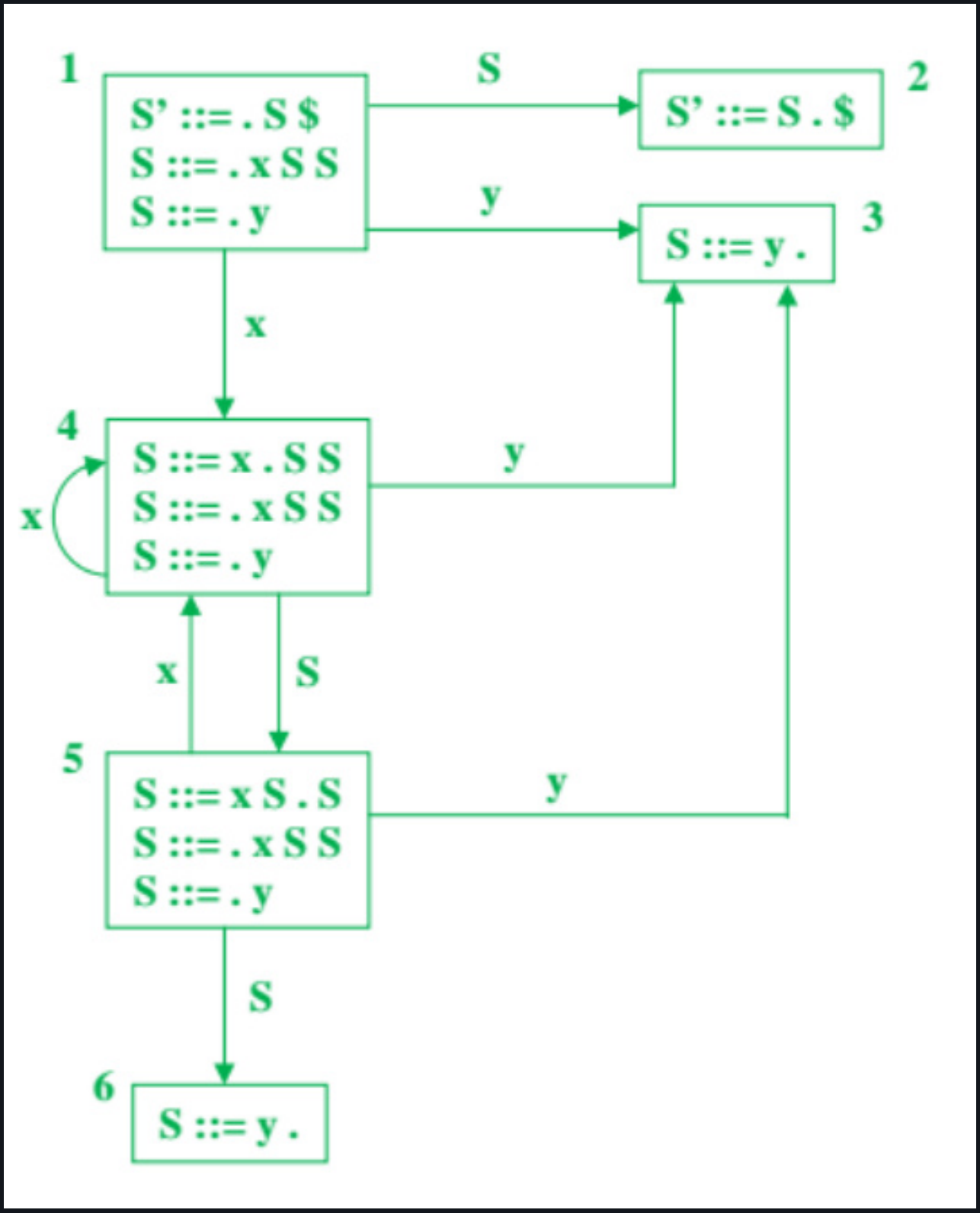
(A) Máquina de estados LR(0): figura abaixo.

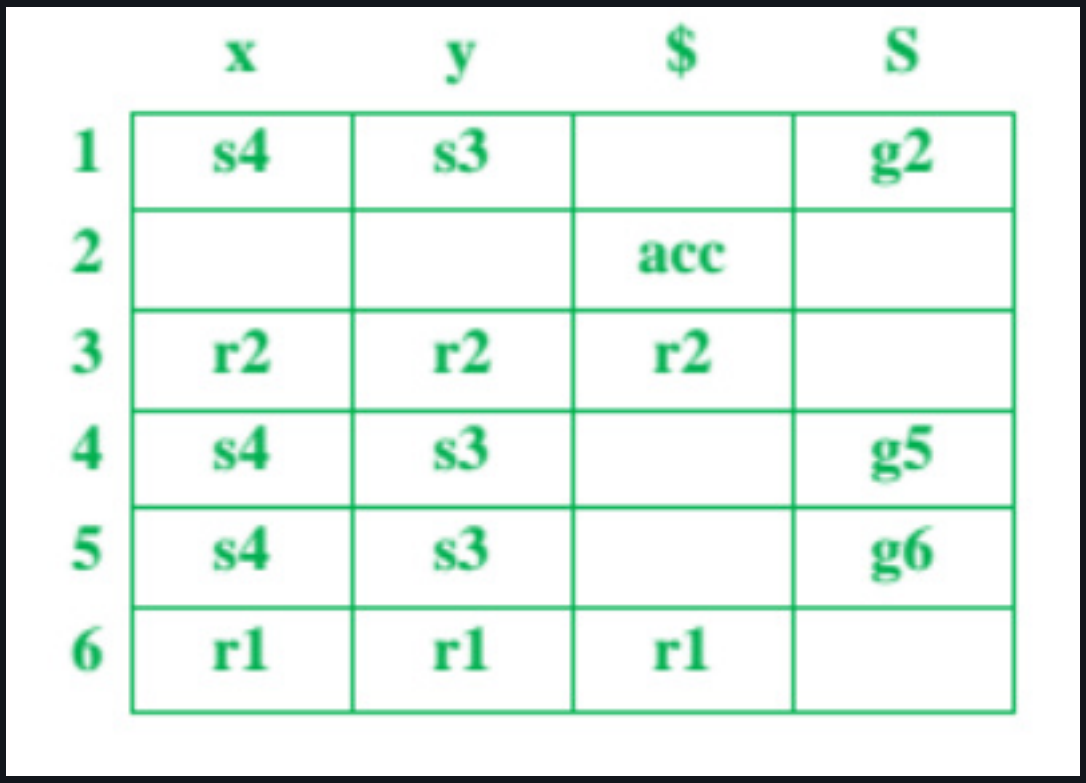
(B) Tabela LR(0): figura abaixo.

(C) **Sim.** Não há conflitos shift-reduce nem reduce-reduce em nenhuma entrada da tabela: os estados de

```
item interno só deslocam (S' ::= x . S S, S ::= x S . S) e os estados de redução (S ::= y ., S
::= x S S .) não têm itens de deslocar.
```

(D) **Sim.** Toda gramática LR(0) também é SLR (o SLR só usa lookahead adicional para decidir reduções; se não há conflito sem lookahead, não há com ele).


1
S' ::= . S $
S ::= . x S S
S ::= . y
S
2
S' ::= S . $
y
3
S ::= y .
x
4
S ::= x . S S
S ::= . x S S
S ::= . y
y
x
x
S
5
S ::= x S . S
S ::= . x S S
S ::= . y
y
S
6
S ::= y .



	x	y	$	S
1	s4	s3		g2
2			acc	
3	r2	r2	r2	
4	s4	s3		g5
5	s4	s3		g6
6	r1	r1	r1	

Pergunta 4 — FIRST/FOLLOW/NULLABLE (20 pontos)

Considere a gramática:

```
S ::= A C B
A ::= B C | B a A
B ::= b C
C ::= C b | C c | ε
```

Complete a tabela com FIRST, FOLLOW e NULLABLE de cada não-terminal.

Resolucão

```
| Não-terminal | NULLABLE | FIRST | FOLLOW |
| S | não | {b} | {$} |
| A | não | {b} | {b, c} |
| B | não | {b} | {$, a, b, c} |
| C | sim | {b, c, ε} | {b, c} |
```

`C` é a única anulável (produção `ε`); `B` começa com `b`, então `A` e `S` também não são anuláveis e seu FIRST é

```
{b}. FOLLOW(A) vem de S ::= A . C B: como C é anulável, FIRST(C)\{ε} = {b,c} e FIRST(B) =
{b}.
FOLLOW(B) recebe $ de S ::= A C B ., {b,c} de A ::= B . C (mais FOLLOW(A) de C anulável) e
{a}
de A ::= B . a A. FOLLOW(C) vem de S ::= A C . B, A ::= B C . e das próprias produções de C.
```

Symbol	NULLABLE	FIRST	FOLLOW
S	F	{ b }	$
A	F	{ b }	{ b, c }
B	F	{ b }	{ a, b, c }
C	T	{ b, c }	{ a, b, c }

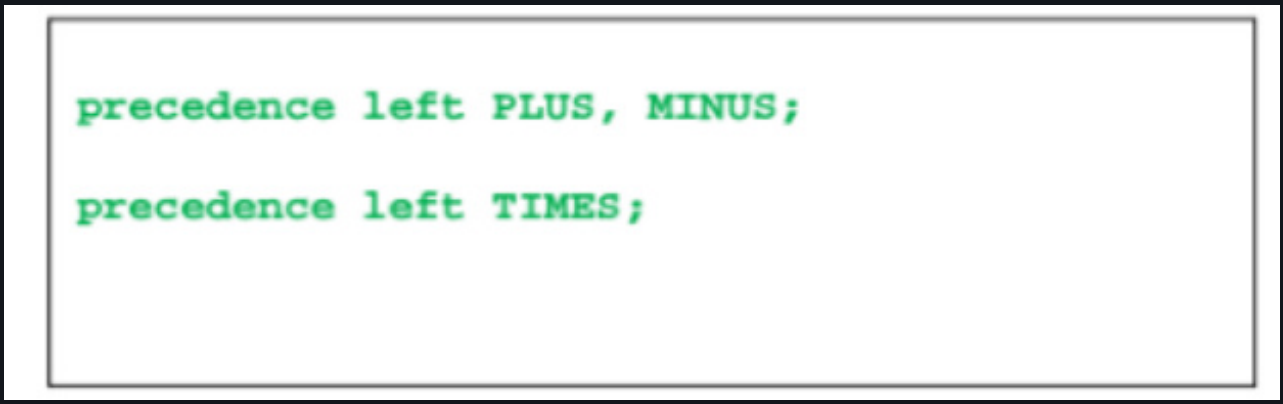
Pergunta 5 — As ferramentas de parse (16 pontos)

Para esse problema que gostaria de traduzir expressões aritméticas de notação infix ordinário para postfix usando CUP. Em notação sufixo uma expressão binário como a * b é escrito como ab *, com os operandos que aparecem na mesma ordem que eles se inicialmente seguido pelo operador. Operadores ajem sobre as duas expressões ou subexpressions imediatamente à sua esquerda e os operadores são executados assim que eles são atingidos digitalização esquerda para a direita. Expressões Postfix são inequívocas e não necessitam de parênteses ou regras de precedência. Aqui estão alguns exemplos:

Infix Postfix Avaliação a-b+c ab-c+ obtém-se, em seguida, c é adicionado ao resultado a-(b+c) abc+- são digitalizados, em seguida, b + c é calculado em primeiro lugar, seguida esse resultado é o operando direito da subtração de a a+b*c abc*+ multiplicar b * c, em seguida, adicionar esse resultado ao a (a+b)*c ab+c* adicionar a + b em primeiro lugar, em seguida, multiplique por c a*b+c ab*c+ multiplicar a * b em primeiro lugar, em seguida, adicione c (a*b)+c ab*c+ Iguais a*b+c*d ab*cd*+ multiplicar a * b, multiplique c * d, em seguida, adicione os dois produtos Na próxima página, preencha a especificação CUP para que o analisador resultante aceitará expressões aritméticas infixas envolvendo identificadores; a operadores +, - e *; e subexpressões entre parênteses, e imprimir a expressão postfix correspondente. (Nota: você não está construindo uma árvore como você fez para a parte analisador do projeto Basta imprimir coisas em locais adequados..) Você deve assumir que as ações semânticas pode chamar uma cópia (s) função que irá imprimir o conteúdo de string s para a saída imediatamente após qualquer saída anterior. A acção semântica para identificador é fornecido como um exemplo. (Nós estamos tomando algumas liberdades com os tipos aqui e estão assumindo que quando um token identificador é impresso a seqüência de identificador correspondente aparece na saída. Isso não é verdade para os operadores PLUS, Times, etc. Você precisa imprimir strings literais apropriadas em os lugares certos.) Você também vai precisar adicionar declarações de precedência para lidar com as ambiguidades na gramática dada. Conclua a especificação CUP abaixo para ler expressões infixas e imprimir o postfix correspondente. (Dica: as respostas podem ser bastante curto e / ou simples - não se assuste se isso acontecer.) / * Terminais (tokens retornado pelo scanner) * / PLUS terminal, subtração, multiplicação, LPAREN, RPAREN, identificador; / * Nonterminals

- / não-terminal vazio Expr; / * Declarações de precedência - acrescentam nada apropriado aqui * / /* Produtos */

```
Expr ::= IDENTIFIER:id {: print(id) :} | Expr:exp1 PLUS Expr:exp2
```



Resolução

Complete a especificação CUP:

```
/* Terminais */
terminal PLUS, MINUS, TIMES, LPAREN, RPAREN, IDENTIFIER;
/* Não-terminais */
Não terminal Expr;
/* Precedência: resolve a ambiguidade */
precedence left PLUS, MINUS;
precedence left TIMES;

/* Produções */
Expr ::= IDENTIFIER:id {: print(id) :}
| Expr:exp1 PLUS Expr:exp2 {: print("+") :}
| Expr:exp1 MINUS Expr:exp2 {: print("-") :}
| Expr:exp1 TIMES Expr:exp2 {: print("*") :}
| LPAREN Expr:exp RPAREN {: /* nada */ :}
;
```

`precedence left` declara `+` e `-` com a mesma precedência, associados à esquerda, e `*` com precedência maior. As ações imprimem o operador **depois** de os operandos terem sido impressos (ordem pós-fixa); parênteses não imprimem nada, pois só alteram a estrutura.

3. Provas complementares de meio

au02 e au05 não trazem rótulo de meio/final na fonte; entraram aqui porque a prova inteira cobra assuntos de meio de período — expressões regulares, scanner, gramáticas, ambiguidade, LR parsing, FIRST/FOLLOW e AST (uma única questão, a 9 de au05, é de quadro de pilha).

O1 — Prova au02 (outono de 2002) · prova do meio

Questão 1 (10 pontos)

Expressões regulares. Descreva o conjunto de strings gerado por cada uma das seguintes expressões regulares.

```
(a) (a | (bc)* d)+

(b) ((0 | 1)* (2 | 3)+) | 0011
```

Resolução

(a) Uma ou mais ocorrências de: ou a string `a`, ou a string `d` precedida de zero ou mais pares `bc`. (Note que `*` tem precedência mais alta, a concatenação vem em seguida e `|` tem a menor precedência; quem errou por confundir níveis de precedência recebeu a maior parte da nota.)

(b) Ou a string `0011`, ou uma string formada por zero ou mais `0`/`1`, seguida de uma string com um ou mais `2`/`3`.

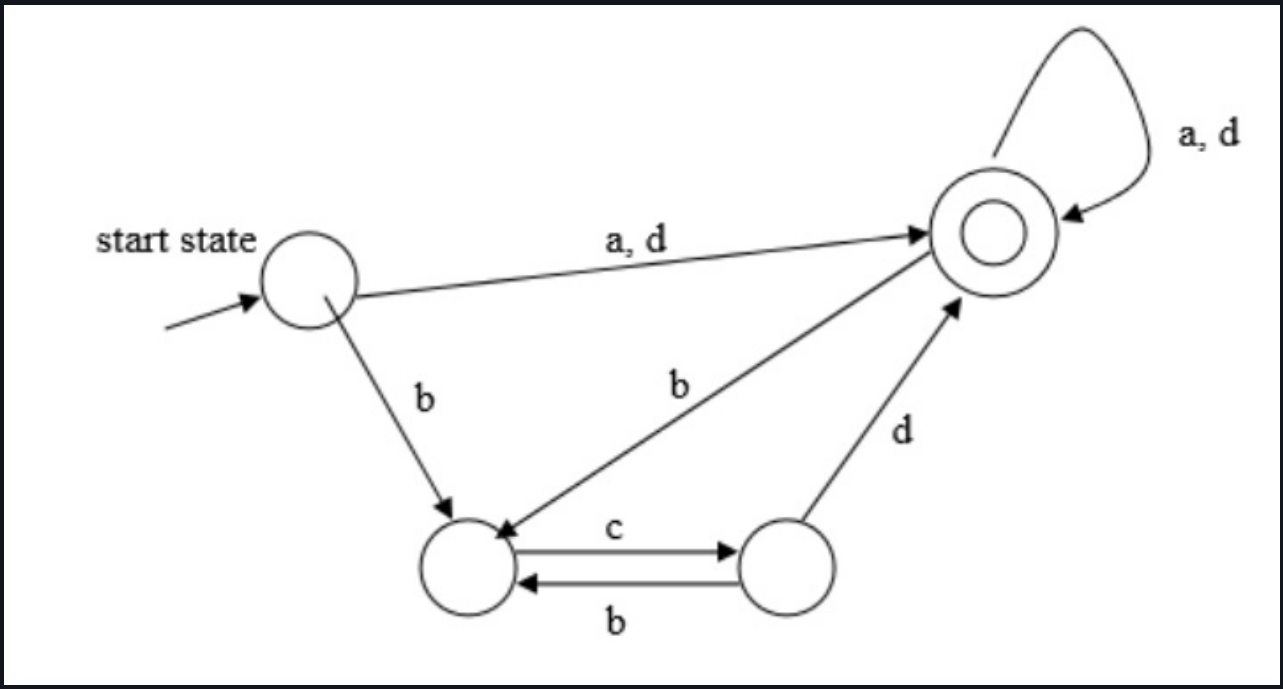
Questão 2 (10 pontos)

Desenhe um autômato finito determinístico (DFA) que reconhece strings geradas por (a | (bc)* d)+. Você não precisa usar os algoritmos formais para a conversão de expressões regulares para um NFA e em seguida um DFA (embora esses algoritmos pode fornecer algumas dicas sobre o que é necessário). Basta desenhar um diagrama adequado.

Resolucão

```
DFA que reconhece (a | (bc)* d)+ (figura abaixo):
```

- Estado inicial `q0` (não final): com `a` ou `d` vai para o estado final `qf` (um bloco completo foi lido — `d` sozinho é o caso `(bc)*d` com zero `bc`); com `b` vai para `qb`.
- `qf` (final, entre blocos): com `a` ou `d` permanece em `qf`; com `b` vai para `qb`.
- `qb` (esperando `c` após `b`): com `c` vai para `qbc`.
- `qbc` (zero ou mais `bc` lidos): com `b` volta para `qb`; com `d` vai para `qf`.
- Qualquer outra entrada vai para o estado de erro (implícito).



Questão 3 (10 pontos)

Uma nova linguagem de programação está sendo projetada para incluir os números decimais sem expoente. Um número decimal tem um período para separar o número inteiro e partes fraccionárias do número e deve ter, pelo menos, um dígito, tanto antes como após o ponto decimal. Além disso, um número decimal não deve ter qualquer líder supérfluo ou final 0 da. Alguns exemplos de números decimais legais são 3.14, 0.01, 17.0, 0.0, 123.00321. Alguns exemplos de números decimais ilegais são 00.0, 0.5, 17, 17., 003.01, 0.0012300 (este último exemplo é ilegal por causa dos 0's à direita; é necessário o 0 antes do ponto decimal). Escrever uma expressão regular (ou conjunto de expressões regulares) para gerar números decimais legais.

Resolucão

```
( 0 | [1-9][0-9]* ) \. ( 0 | [0-9]*[1-9] )
```

- Parte inteira: `0` isolado, ou um dígito inicial 1-9 seguido de zero ou mais dígitos (sem zeros à esquerda).
- Parte fracionária: exatamente `0`, ou zero ou mais dígitos terminando em um dígito não-zero (sem zeros à direita).

```
Exemplos: 3.14, 0.01, 17.0, 0.0, 123.00321 ✓; 00.0, 0.5, 17, 17., 003.01, 0.0012300 ✗.
```

Questão 4 (10 pontos)

Considere o seguinte fragmento de um programa Java (incluindo alguns operadores que não existem no JFLAP):

```
while (xyzzy >= thing) // repeat until thing is big enough
{ x += y; thing++; }
```

Liste, em ordem, os tokens que o scanner produziria ao ler essa entrada. Use nomes de tokens razoáveis — basta que sejam claros e descritivos.

Resolucão

```
WHILE LPAREN ID(xyzzy) GEQ ID(thing) RPAREN LBRACE ID(x)
PLUSASSIGN ID(y) SCOLON ID(thing) PLUSPLUS SCOLON RBRACE
```

Notas: += e ++ são tokens individuais (princípio do match mais longo); os tokens de identificador carregam o lexema como atributo; comentários, quebras de linha e espaços em branco não são tokens — o scanner os filtra.

Questão 5 (12 pontos)

A gramática da linguagem para declarações abaixo é ambígua por causa do problema do `else` pendente:

```
stmt ::= if ( expr ) stmt | if ( expr ) stmt else stmt | while ( expr ) stmt | S
```

Dê uma gramática não ambígua que gere exatamente as mesmas declarações e trate corretamente o `else` pendente. (Dica: introduza várias produções para declarações e divida-as em duas categorias: as que podem terminar em um `if` sem `else` — um "if curto" — e as que definitivamente não terminam assim.)

Resolução

A ideia-chave: a instrução que vem *depois* de um `if` e *antes* de outro (ou de um `else`) não pode ser ela própria um `if` sem `else`.

```
stmt ::= stmtNoTrailingSubstmt | ifStmt | ifElseStmt | whileStmt

stmtNoTrailingSubstmt ::= S

stmtNoShortIf ::= whileStmtNoShortIf | ifElseStmtNoShortIf | stmtNoTrailingSubstmt

ifStmt ::= if ( expr ) stmt
ifElseStmt ::= if ( expr ) stmtNoShortIf else stmt

ifElseStmtNoShortIf ::= if ( expr ) stmtNoShortIf else stmtNoShortIf
whileStmt ::= while ( expr ) stmt
whileStmtNoShortIf ::= while ( expr ) stmtNoShortIf
```

Notas: fatorar a gramática em

```
stmt ::= if ( expr ) stmt ifTail
ifTail ::= else stmt | ε
```

não resolve o `else` pendente — ainda é possível gerar duas derivações distintas para `if (expr) if (expr) stmt else stmt`. O erro mais sutil é esquecer o `while`: ele também precisa da versão "no short if", senão a gramática continua ambígua em `if (expr) while (expr) if (expr) stmt else stmt`.

Questão 6 (20 pontos)

Gramáticas e LR parsing. Considere a gramática

```
s ::= expr $
expr ::= a | a subs
subs ::= [ expr ] | [ expr ] subs
```

(Na primeira produção, `$` representa o fim de arquivo.)

(a) (3 pontos) Quais são os símbolos terminais e não terminais dessa gramática?

(b) (3 pontos) Descreva em prosa o conjunto de palavras gerado pela gramática.

(c) (Gramática repetida para referência.)

```
s ::= expr $
expr ::= a | a subs
subs ::= [ expr ] | [ expr ] subs
```

(d) (12 pontos) Construa (desenhe) a máquina de estados LR(0) dessa gramática. Você não precisa escrever as tabelas ACTION e GOTO, nem calcular FIRST e FOLLOW — basta desenhar a máquina, mostrando os conjuntos de itens em cada estado.

(e) (2 pontos) Essa gramática é LR(0)? Por que ou por que não?

Resolução

(a) Terminais: `a`, `[`, `]`, `$`. Não terminais: `s`, `expr`, `subs`.

(b) Expressões formadas por um identificador `a` seguido de zero ou mais expressões entre colchetes — ou seja, escalares e vetores de elementos em C/C++/Java (como `a`, `a[1]`, `a[1][b+2]` ...).

```
(d) Máquina LR(0) (produções: 0: s -> expr $, 1: expr -> a, 2: expr -> a subs, 3: subs -> [ expr
], 4: subs -> [ expr ] subs):

estado 1 (inicial): { s -> . expr $, expr -> . a, expr -> . a subs }
 a -> 2 expr -> 4

estado 2: { expr -> a ., expr -> a . subs, subs -> . [ expr ], subs -> . [ expr ] subs }
 [ -> 5 subs -> 3 reduz expr -> a

estado 3: { expr -> a subs . }
 reduz expr -> a subs

estado 4: { s -> expr . $ }
 $ -> aceita

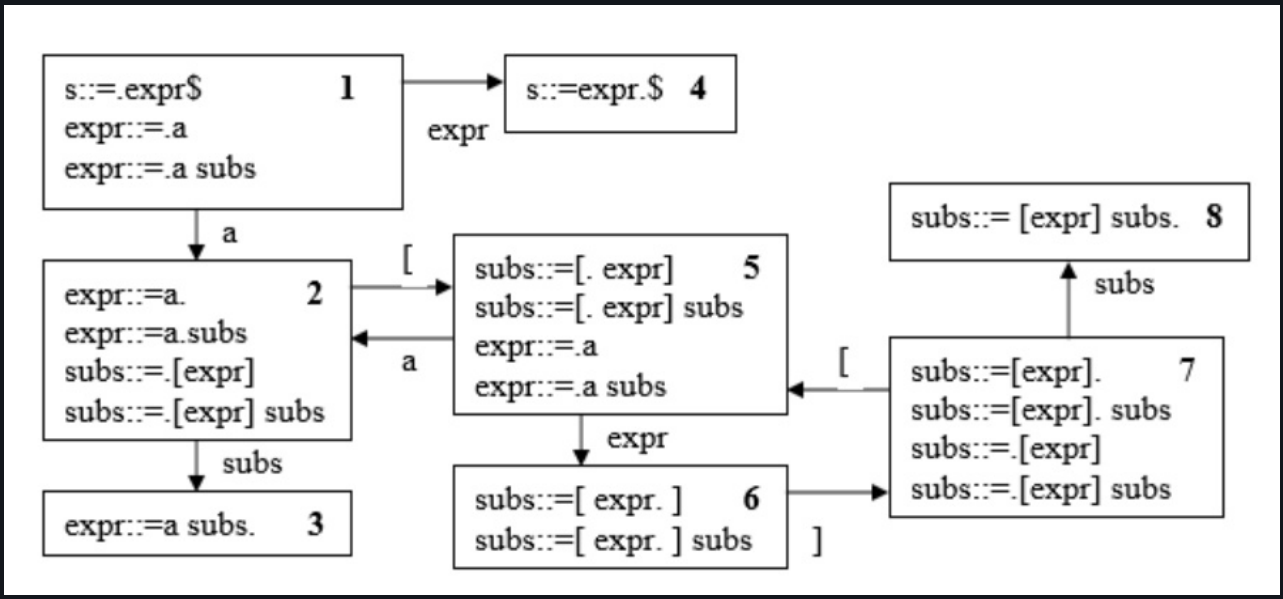
estado 5: { subs -> [ . expr ], subs -> [ . expr ] subs, expr -> . a, expr -> . a subs }
 a -> 2 expr -> 6

estado 6: { subs -> [ expr . ], subs -> [ expr . ] subs }
 ] -> 7

estado 7: { subs -> [ expr ] ., subs -> [ expr ] . subs, subs -> . [ expr ],
 subs -> . [ expr ] subs, expr -> . a, expr -> . a subs }
 [ -> 5 subs -> 8 reduz subs -> [ expr ]

estado 8: { subs -> [ expr ] subs . }
 reduz subs -> [ expr ] subs
```

Diagrama de estados oficial e tabela ACTION/GOTO (esta última é apenas informativa — o enunciado dispensa a tabela):



state	a	[	]	$	s	expr	subs
1	s2					g4	
2	r2,	r2,s5	r2	r2			g3
3	r3	r3	r3	r3			
4				acc			
5	s2					g6	
6			s7				
7	r4	r4, s5	r4	r4			g8
8	r5	r5	r5	r5			

(e) **Não.** Nos estados 2 e 7 há conflito deslocar/reduzir na entrada `[`: o estado teria que reduzir (`expr -> a` e `subs -> [ expr ]`) e ao mesmo tempo deslocar o `[`. LR(0) decide sem olhar o lookahead, então a gramática não é LR(0). Ela é, porém, **SLR(1)**: com lookaheads `FOLLOW(expr) = { $, ] }` e `FOLLOW(subs) = { $, ] }`, a redução e a deslocamento sobre `[` não colidem mais.

O2 — Prova au05 (outono de 2005) · prova do meio

Questão 1 (8 pontos)

Escreva uma expressão regular (ou conjunto de expressões regulares) que gere os seguintes conjuntos de strings. Você pode usar abreviaturas (`name = expressão regular`) se isso deixar a resposta mais clara.

(a) (4 pontos) Todas as strings de `a`'s, `b`'s e `c`'s que contêm um número par de `a`'s.

(b) (4 pontos) Todas as strings de `a`'s, `b`'s, `c`'s, `A`'s, `B`'s e `C`'s que seguem a convenção de Java para identificadores: o identificador deve começar com uma letra minúscula (`abc`), seguida de zero ou mais letras, com a restrição de que duas letras maiúsculas (`ABC`) não podem ficar lado a lado. `a`, `aA`, `aAaB`, `aaCabcBa` e `abc` são exemplos corretos; `A`, `Aa`, `aBC` e `aBBa` não são.

Resolução

(a)

```
((b|c)* | ((b|c)* a (b|c)* a (b|c)*)*)
```

Ou seja: ou não há nenhum `a`, ou os `a`'s aparecem em pares, com blocos de `b`/`c` entre eles. Uma forma equivalente mais curta: `((b|c)* a (b|c)* a (b|c)*)*`.

(b)

```
(a|b|c)+ ( (A|B|C) (a|b|c)+ )* (A|B|C)?
```

Começa com uma ou mais letras minúsculas; cada `A`/`B`/`C` (exceto uma eventual última) é seguido de uma ou mais letras minúsculas — assim duas maiúsculas nunca ficam adjacentes; pode terminar com uma única maiúscula.

Questão 2 (8 pontos)

Para cada uma das expressões regulares abaixo, dê uma breve descrição do conjunto de strings gerado. (Lembre-se de que `*` tem precedência maior — liga-se com mais força — do que a concatenação.)

(a) (4 pontos) `a*ba*ba*ba*`

```
(b) (4 pontos) (x*y*)* xx (x|y)*
```

Resolução

(a) Todas as strings de `a`'s e `b`'s com **exatamente três** `b`'s.

(b) Todas as strings de `x`'s e `y`'s com **pelo menos um par de `x`'s adjacentes** (`xx`).

Questão 3 (5 pontos)

A família C de linguagens inclui prefixos e operadores binários + e –, bem como os operadores de incremento e decremento ++ e --, que podem aparecer antes ou depois de uma expressão. Recordando que um scanner usa um princípio de palavra mais longa quando está lendo a entrada e parseando em tokens, quais são os tokens na seguinte declaração? a++=++thing+++other--;

Resolucão

```
ID(a) PLUSPLUS EQUALS PLUSPLUS ID(thing) PLUSPLUS PLUS ID(other) MINUSMINUS SCOLON
```

Pelo princípio do match mais longo: `++` vira `PLUSPLUS`, `+++` vira `PLUSPLUS` + `PLUS`, e `--` vira `MINUSMINUS`.

Questão 4 (8 pontos)

A expressão regular a seguir é acusada de gerar todas as strings de 0's e 1's que contêm exatamente um par 1's adjacentes:

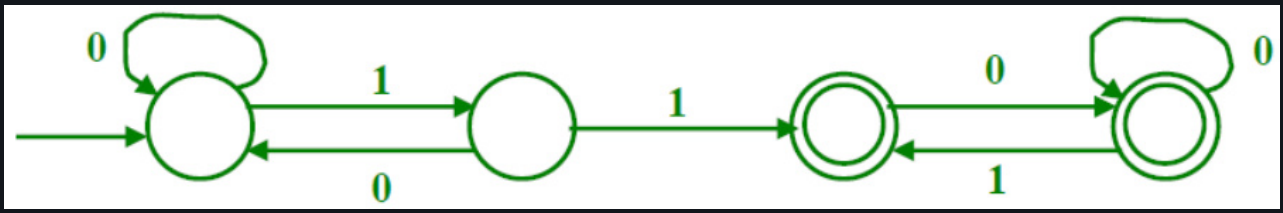
(0 | 10)* 11 (0 | 01)* Desenhe um DFA que aceita strings geradas por esta expressão regular.

Certifique-se de que o seu diagrama diferencie entre estados finais e não finais (use um círculo duplo para estados finais e um único círculo de estados não-finais). Você não precisa usar a construção formal de um NFA para DFA, apenas desenhar um DFA que aceita este conjunto de strings.

Resolução

DFA (figura abaixo; círculo duplo = estado final):

- `q0` (inicial, não final): prefixo `(0|10)*`. Com `0` → `q0`; com `1` → `q1`.
- `q1` (viu um `1` solto que pode virar `11` ou completar `10`): com `1` → `q2` (o par `11` foi encontrado); com `0` → `q0` (completou `10`).
- `q2` (**final**, já viu o único par `11`; está no sufixo `(0|01)*`): com `0` → `q2`; com `1` → `q3`.
- `q3` (no sufixo, viu um `1` que precisa de `0` para formar `01`): com `0` → `q2`; com `1` → erro (formaria um segundo par adjacente).
- Erro: tudo permanece em erro.



Questão 5 (3 pontos)

Questão repetida. Este enunciado e esta resolução são idênticos aos de **Questão 5 (8 pontos)**, de **M3 — CSE401 midterm inverno 2010** (página 33). Não há nada novo aqui — se você já copiou aquela, pode pular esta.

Questão 6 (8 pontos)

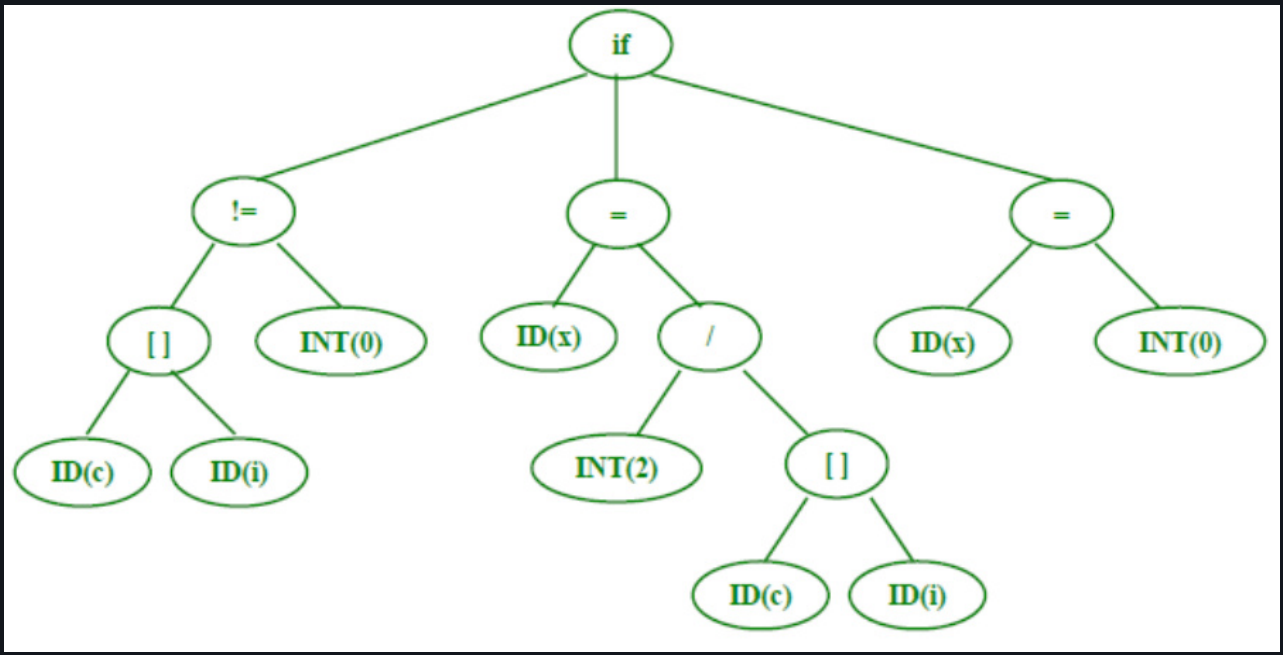
Considere o seguinte fragmento de progama C (ou Java, C ++, etc.). if (c[i] != 0) x = 2 / c[i]; else x = 0; Desenhe uma árvore de sintaxe abstrata (AST) para este fragmento de programa. A AST deve refletir a semântica do programa de origem, por exemplo, a sintaxe abstrata para c[i] deve ser mostrada como um vetor com índices em operação, em vez de dividi-lo em uma sequência de expressões endereço aritméticas ou, no outro extremo , apenas deixá-lo como um único nó que contém a label c[i].

Resolução

AST (figura abaixo):

```
If(cond: !=( ArrayIndex(c, i), 0 ),
 then: Assign(x, /( 2, ArrayIndex(c, i) )),
 else: Assign(x, 0))
```

O acesso `c[i]` é um nó de indexação de vetor (`ArrayIndex` com vetor e índice), não uma sequência de expressões aritméticas de endereço nem um nó literal `"c[i]"` — é isso que a AST deve refletir.



Questão 7 (8 pontos)

Considere a gramática:

```
exprs ::= exprs + expr | exprs * expr | expr
expr ::= x
```

(a) (4 pontos) Essa gramática é ambígua ou não? Se for ambígua, mostre duas árvores de análise diferentes para alguma string gerada por ela; se não for, dê um argumento informal.

(b) (4 pontos) A gramática captura adequadamente a precedência normal dos operadores `*` e `+` (`*` deve ter precedência maior que `+`)? Se sim, discuta brevemente por quê; se não, dê um exemplo que mostre a falha.

Resolução

(a) **Não é ambígua.** Informalmente: a única maneira de derivar uma sequência de operações é expandir o não-terminal `exprs` à esquerda repetidamente; como `exprs` só pode virar uma `expr` ou outro `exprs` (recursivo à esquerda), não há escolha livre de qual alternativa expandir em nenhum ponto — a derivação é única para cada string.

(b) **Não.** Considere `x + x * x`: a única derivação possível é

```
exprs => exprs * expr => (exprs + expr) * expr => ... => (x + x) * x
```

Como todas as derivações são associativas à esquerda, sem distinguir `+` de `*`, o `+` acaba ligando-se com mais força que o `*` — exatamente o contrário da precedência desejada.

Questão 8 (20 pontos)

(Questão inevitável de parsing LR.) O processamento de texto é um dos aplicativos mais duradouros do Unix, especialmente documentos com matemática. O pré-processador `eqn` traduz equações escritas em uma mini-linguagem para códigos de composição; a linguagem do `eqn` pode ser descrita por uma gramática livre de contexto e implementada com `lex` e `yacc`. Um fragmento da gramática — a sintaxe de expressões com sobrescritos — é:

```
0. eqns' ::= eqns $ ($ é o marcador de fim de arquivo)
1. eqns ::= eq sup eqns
2. eqns ::= eq
3. eq ::= x
```

Por exemplo, `x sup x sup x` descreve a equação x^(x^x). (Como na matemática, vários sobrescritos agrupam-se à direita.) Os símbolos `sup` e `x` são terminais que representam a si mesmos.

(a) (8 pontos) Construa uma máquina de estados LR(0) para essa gramática, mostrando o conjunto de itens de cada estado.

(b) (2 pontos) Essa gramática é LR(0)? Por que ou por que não?

(c) (4 pontos) Calcule FIRST, FOLLOW e Nullable de cada não-terminal da gramática.

(d) (4 pontos) Anote a tabela ACTION e GOTO SLR(1) para essa gramática, usando o diagrama LR(0) do item (a) e FIRST/FOLLOW/Nullable do item (c).

(e) (4 pontos) Essa gramática é LR(1)? Por quê ou por que não?

Resolução

(A) Máquina LR(0) (itens de cada estado; transições indicadas):

```
estado 0 (inicial): { eqns' -> . eqns $,
 eqns -> . eq sup eqns,
 eqns -> . eq,
 eq -> . x }
 eq -> 1 eqns -> 3 x -> 4

estado 1: { eqns -> eq . sup eqns,
 eqns -> eq . }
 sup -> 2

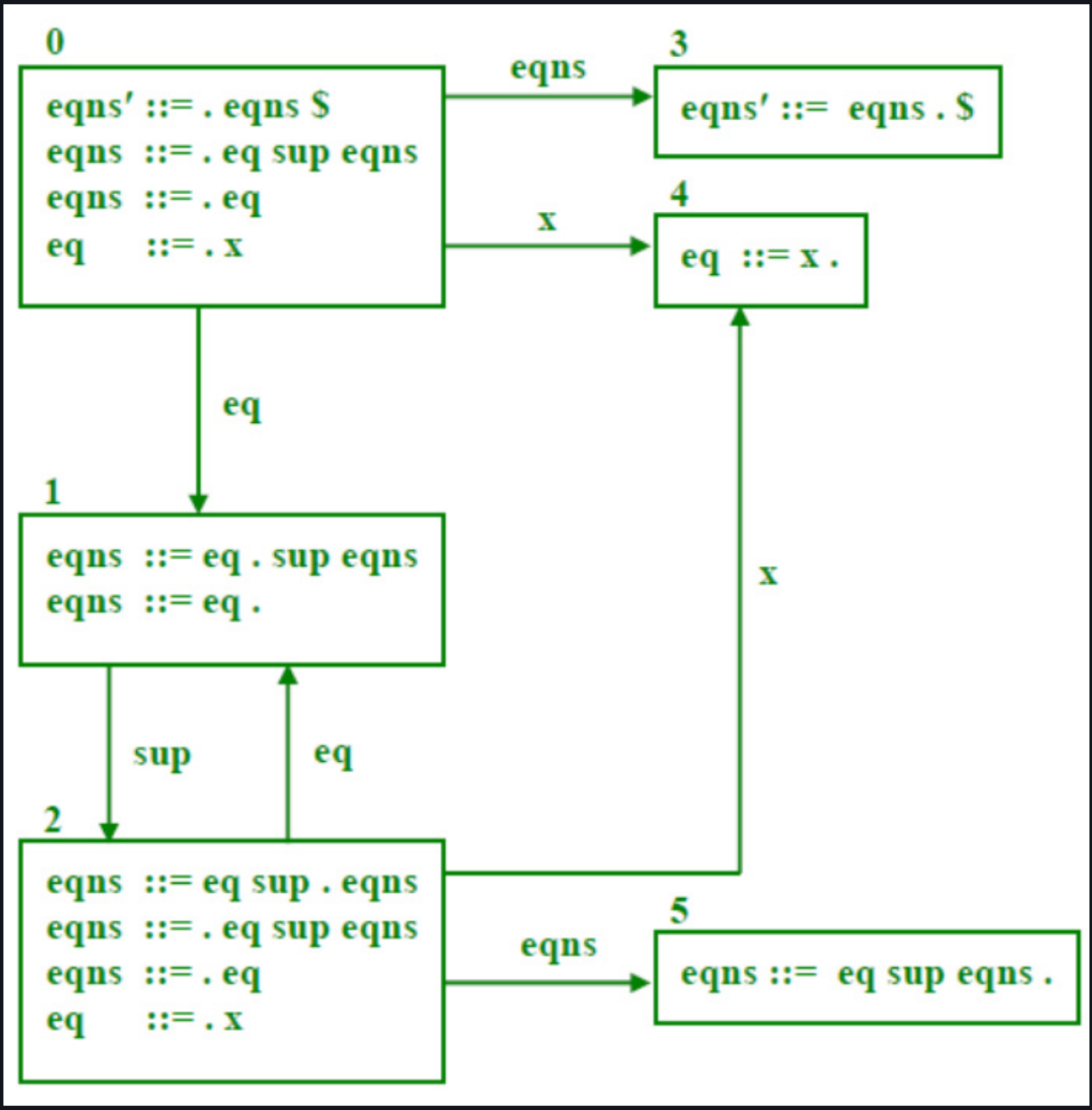
estado 2: { eqns -> eq sup . eqns,
 eqns -> . eq sup eqns,
 eqns -> . eq,
 eq -> . x }
 eq -> 5 x -> 4

estado 3: { eqns' -> eqns . $ }
 $ -> aceita

estado 4: { eq -> x . }

estado 5: { eqns -> eq sup eqns . }
```

Diagrama oficial:



(B) **Não.** No estado 1 há conflito deslocar/reduzir: em `sup` seria preciso deslocar (item `eqns -> eq . sup eqns`) e reduzir (`eqns -> eq .`); LR(0) decide sem olhar o lookahead.

(C)

Não-terminal	FIRST	FOLLOW	Nullable
`eqns`	`{ x }`	`{ $ }`	não
`eq`	`{ x }`	`{ sup, $ }`	não

(Nenhum não-terminal deriva ε.) Tabela oficial:

	First	Follow	Nullable
eqns'	x		no
eqns	x	$	no
eq	x	sup, $	no

(D) Tabela SLR(1) — reduções: R1 = `eqns -> eq sup eqns`, R2 = `eqns -> eq`, R3 = `eq -> x`:

Estado	ação em `x`	em `sup`	em `$`	GOTO
0	desloca 4	eq→1, eqns→3		
1	desloca 2	R2 (FOLLOW(eqns)={$})		
2	desloca 4	eq→5		
3	aceita			
4	R3 (FOLLOW(eq)={sup,$})	R3		
5	R1			

Tabela oficial:

state	x	sup	$	eqns	eq
0	s4			g3	g1
1		s2	r2		
2	s4			g5	g1
3			acc		
4		r3	r3		
5			r1		

(E) **Sim, é LR(1).** Com lookahead de um símbolo, o conflito do estado 1 desaparece: só se reduz `eqns -> eq` quando o lookahead é `$`, e só se desloca `sup` quando o lookahead é `sup`. Não há outros conflitos deslocar/reduzir nem reduzir/reduzir.

Questão 9 (16 pontos)

Considere a função C que devolve um inteiro:

```
cint fun(int a, int b, int c) {
 int ans;
 ans = max(a, c);
 ans = max(ans, b);
 if (ans < 0) {
 ans = - ans;
 }
 return ans;
}
```

(a) (4 pontos) Desenhe o layout do stack frame após o prólogo da função, mas antes da primeira instrução do corpo. Mostre:

- locais e offsets numéricos de todas as variáveis e parâmetros, e qualquer outra informação no frame;
- onde `esp` e `ebp` apontam no frame.

(b) (12 pontos) Na página seguinte, escreva uma versão em assembly x86 desta função. Observações:

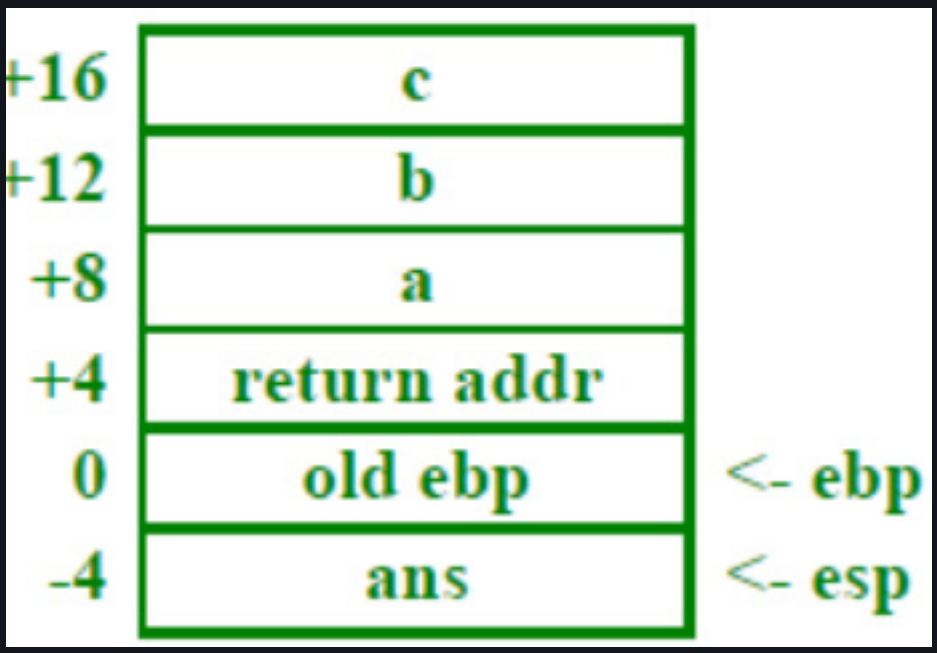
- sua resposta não precisa imitar exatamente a forma de código vista em aula — código x86 direto que reflita o código-fonte basta, mas:
- use as convenções de chamada corretas da linguagem C;
- o código deve incluir **todas** as instruções dos enunciados, ou seja, todos os stores e loads implícitos nas expressões, mesmo que não fossem estritamente necessários;
- variáveis e parâmetros devem ser endereçados com o registrador-base e o offset corretos do frame;
- `max` é uma função externa de valor inteiro, que pode ser chamada sem declarações adicionais;
- não inclua diretivas como `.386` ou `.model`.

Resolução

(A) Layout do frame (cdecl: `a` em `8(%ebp)`, `b` em `12(%ebp)`, `c` em `16(%ebp)`, `ans` em `-4(%ebp)`):

```
+----------------------+
| c 16 |
| b 12 |
| a 8 |
| endereço retorno 4 |
| ebp antigo 0 | <- %ebp
| ans -4 | <- %esp (após subl $4, %esp)
+----------------------+

%ebp aponta para o ebp antigo salvo; %esp aponta para ans (o local mais baixo do frame).
Diagrama oficial:
```



(B) Versão x86:

```
fun:
 pushl %ebp # prólogo
 movl %esp, %ebp
 subl $4, %esp # espaço para ans

 movl 16(%ebp), %eax # ans = max(a, c)
 pushl %eax # argumento c (seria possível: pushl 16(%ebp))
 movl 8(%ebp), %eax # argumento a
 pushl %eax
 call max
 addl $8, %esp
 movl %eax, -4(%ebp)
```

```
 movl 12(%ebp), %eax # ans = max(ans, b)
 pushl %eax # argumento b

movl -4(%ebp), %eax # argumento ans
 pushl %eax
 call max
 addl $8, %esp
 movl %eax, -4(%ebp)

 movl -4(%ebp), %eax # if (ans < 0)
 cmpl $0, %eax
 jnl skip
 movl -4(%ebp), %eax # ans = -ans
 negl %eax
 movl %eax, -4(%ebp)

skip:
 movl -4(%ebp), %eax # return ans
 movl %ebp, %esp
 popl %ebp
 ret
```

Convenção cdecl: argumentos empilhados da direita para a esquerda e o chamador limpa a pilha (`addl $8, %esp` após cada chamada).

```
fun:  push  ebp             ; prologue
      mov   ebp,esp
      sub   esp,4

      mov   eax,[ebp+16]    ; ans = max(a,c);
      push  eax             ;  (could use "push [ebp+16]" instead)
      mov   eax,[ebp+8]     ;  (etc.)
      push  eax
      call  max
      add   esp,8
      mov   [ebp-4],eax

      mov   eax,[ebp+12]    ; ans = max(ans,b);
      push  eax
      mov   eax,[ebp-4]
      push  eax
      call  max
      add   esp,8
      mov   [ebp-4],eax

      mov   eax,[ebp-4]     ; if (ans < 0)          *
      cmp   eax,0
      jnl   skip

      mov   eax,[ebp-4]     ;   ans = - ans;        *
      neg   eax
      mov   [ebp-4],eax

skip: mov   eax,[ebp-4]     ; return ans;           *
      mov   esp,ebp
      pop   ebp
      ret
```

PARTE II — PROVAS FINAIS

Todas as questões de prova final do material, reunidas num único bloco: as provas oficiais CSE401 de final e as provas complementares de conteúdo de final.

Assuntos típicos desta parte: estruturas de dados em tempo de execução, geração de código x86/x86-64, quadro de pilha, alocação de registradores, análise de fluxo, SSA, otimização e agendamento de instruções.

Nesta parte:

• F1 — CSE401 final inverno 2009 · F2 — CSE401 final inverno 2010 · F3 — CSE401 final outono 2010

• O3 — CSEP501 inverno 2008 · prova final · O4 — CSEP501 outono 2009 · prova final · O5 — CSEP501 outono 2011 · prova final

O sumário com a página de cada prova e de cada questão está logo após a página de organização.

1. Provas oficiais CSE401 — finais

Classificação herdada da pasta de origem “Provas finals” (inverno 2009, 2010 e outono 2010).

F1 — CSE401 final inverno 2009

Questão 1 (10 pontos)

A versão original 1.0 do Java usa um interpretador na Java VM para executar programas. Versões mais recentes do Java adicionaram um compilador "just-in-time", que traduz o código intermediário Java para código nativo de máquina (x86, PowerPC, ou outros), e executa o código traduzido em vez de interpretar o código intermediário diretamente.

(a) Dê uma possível razão pela qual o Java 1.0 incluiu apenas um interpretador, e não um compilador just-in-time. Por que isso foi uma decisão de engenharia razoável?

(b) Dê uma razão plausível pela qual versões posteriores do Java incluem um compilador just-in-time, além do interpretador. Por que este decisão tomada, em particular desde o interpretador já existia e já executou programas corretamente?

Resolução

(A) Havia ao menos duas boas razões:

- **Mais rápido de implementar:** não era preciso um back-end completo de compilador.
- **Portátil:** para mover Java para outra arquitetura, bastava reimplementar o interpretador e partes do runtime.

(B) **Velocidade de execução.** Programas interpretados são muito mais lentos que código nativo equivalente; com o interpretador já existindo e funcionando, o JIT foi um acréscimo opcional que trouxe desempenho sem abrir mão da compatibilidade.

Questão 2 (8 pontos)

Na classe nós vimos o visitor pattern, que era uma forma de organização de código para o verificador de tipo e outras operações de AST. Esta envolveu a adição de um método aceito com um parâmetro de objeto de visitante para cada tipo de nó AST, e cada objeto visitante tinha um conjunto de métodos de visita, um para cada tipo de nó AST. Embora este é um esquema um pouco complicado, é amplamente usado em compiladores e outras ferramentas implementadas em linguagens orientadas a objetos. Por que isso é uma maneira útil para organizar o código, e qual é o problema que está tentando resolver?

Resolução

Resolve um problema de modularidade. Se operações como verificação de tipo ficam espalhadas pelos métodos de cada nó da AST, adicionar ou alterar uma operação exige modificar várias classes. O visitor pattern concentra todos os métodos de uma mesma operação (ex.: typecheck) em um único módulo — um visitante por operação — trocando o custo de adicionar operações novas pelo custo de adicionar tipos de nós novos, o que é o trade-off certo para compiladores (poucos nós, muitas operações).

Questão 3 (8 pontos)

O projeto de compilador MiniJava inclui tabelas de símbolos separadas para diferentes escopos, como uma tabela de símbolos global para informações de classe e tabelas de símbolos separadas para métodos individuais. O compilador faz várias passagens sobre a AST para preencher essas tabelas. É realmente necessário fazer várias passagens sobre a AST para recolher esta informação? Existe alguma razão técnica que isto não poderia ser feito em uma única passagem?

Resolução

Sim, é necessária. Java permite usar classes e métodos antes de suas declarações aparecerem no programa. Uma primeira passagem coleta as declarações (nomes de classes, membros, escopos); só então uma passagem seguinte pode verificar os usos. Em uma única passagem seria preciso voltar atrás e reavaliar tudo que já foi checado quando a declaração fosse encontrada.

Questão 4 (10 pontos)

É certamente possível gerar código para uma máquina de destino, fazendo uma passagem de AST e gerar código diretamente. Isto é atualmente feito em alguns compiladores simples. Mas na maioria das produções de compiladores a AST é reduzida para criar um programa de baixo nível IL, e que é utilizado para gerar o código de destino, como no MiniJava. Dê duas razões diferentes do por que gerar uma versão IL de nível inferior do programa é uma coisa útil para fazer em um compilador (não apenas em MiniJava).

Resolução

1) O código de nível intermediário já está mais perto da máquina alvo (três endereços, temporários, desvios explícitos), o que torna a geração de código final muito mais simples do que direto da AST.

2) Muitas otimizações são mais fáceis ou mais eficazes no IL (propagação de constantes, dobragem, eliminação de código morto, alocação de registradores), que é um nível de abstração estável e independente do alvo.

Questão 5 (8 pontos)

O código IL de nível inferior em MiniJava e na maioria dos compiladores cria e usa um número ilimitado de variáveis temporárias (t1, t2, t3, ...) para valores intermediários, em vez de usar os registradores da máquina destino no código IL. Por quê? Por que não usar os registradores de máquina alvo diretamente? (Há pelo menos algumas razões - você só precisa dar uma).

Resolução

- É mais fácil gerar e otimizar o código IL quando podemos criar quantos temporários quisermos, sem limites de registradores.
- Evita amarrar o IL (e o otimizador, que é a maior parte do compilador que o processa) às peculiaridades do conjunto de registradores ou da arquitetura alvo. A alocação de registradores é feita depois, na geração de código.

Questão 6 (9 pontos)

x86 hacking, parte I. Suponha que x é uma variável inteira localizado no offset +12 do stack frame do registrador base ebp (ou seja, 12(%ebp), ou [ebp+12] na notação intel). Escrever três sequências de instruções x86 diferentes que implementam a atribuição x = x + 1 (isto é, três maneiras diferentes de incrementar o valor de x e armazenar o valor incrementado de volta na memória). Você pode usar quaisquer registradores que você desejar, e você pode usar ou a sintaxe Intel ou a sintaxe assembler gnu (mas não misture as duas sintaxes assembler juntas). As sequências devem utilizar instruções diferentes ou ter instruções em uma ordem diferente - não é ok para apenas repetir as mesmas instruções exatas, mas usando diferentes registros. Também não é correto adicionar instruções inúteis que não fazem nada, apenas para tornar o código diferente. Código para x=x+1, dado que x está localizado em ebp+12:

Resolução

Cinco sequências distintas (sintaxe Intel, `x` em `12(%ebp)`):

```
inc dword ptr [ebp+12]
```

```
mov eax, [ebp+12]
inc eax
```

```
mov [ebp+12], eax

mov eax, [ebp+12]
add eax, 1
mov [ebp+12], eax

mov eax, 1
add eax, [ebp+12]
mov [ebp+12], eax

mov eax, [ebp+12]
lea eax, [eax+1]
mov [ebp+12], eax
```

O que **não** funciona é `add [ebp+12], 1`: uma instrução x86 não admite imediato e deslocamento na mesma vez — ou o offset `12`, ou o valor `1`, mas não ambos.

Questão 7 (8 pontos)

x86 hacking, parte II. Considere a seguinte função C. No espaço abaixo, desenhar uma imagem do stack frame para a função foo direita antes da execução da instrução de atribuição no corpo da função. Sua imagem deve mostrar onde os parâmetros e variáveis estão localizados, bem como quaisquer itens adicionais que fazem parte do stack frame para foo, tal como o endereço de retorno. Você também deve chamar setas rotuladas mostrando onde no stack frame os registradores ebp (frame pointer) e esp (stack pointer) apontam.

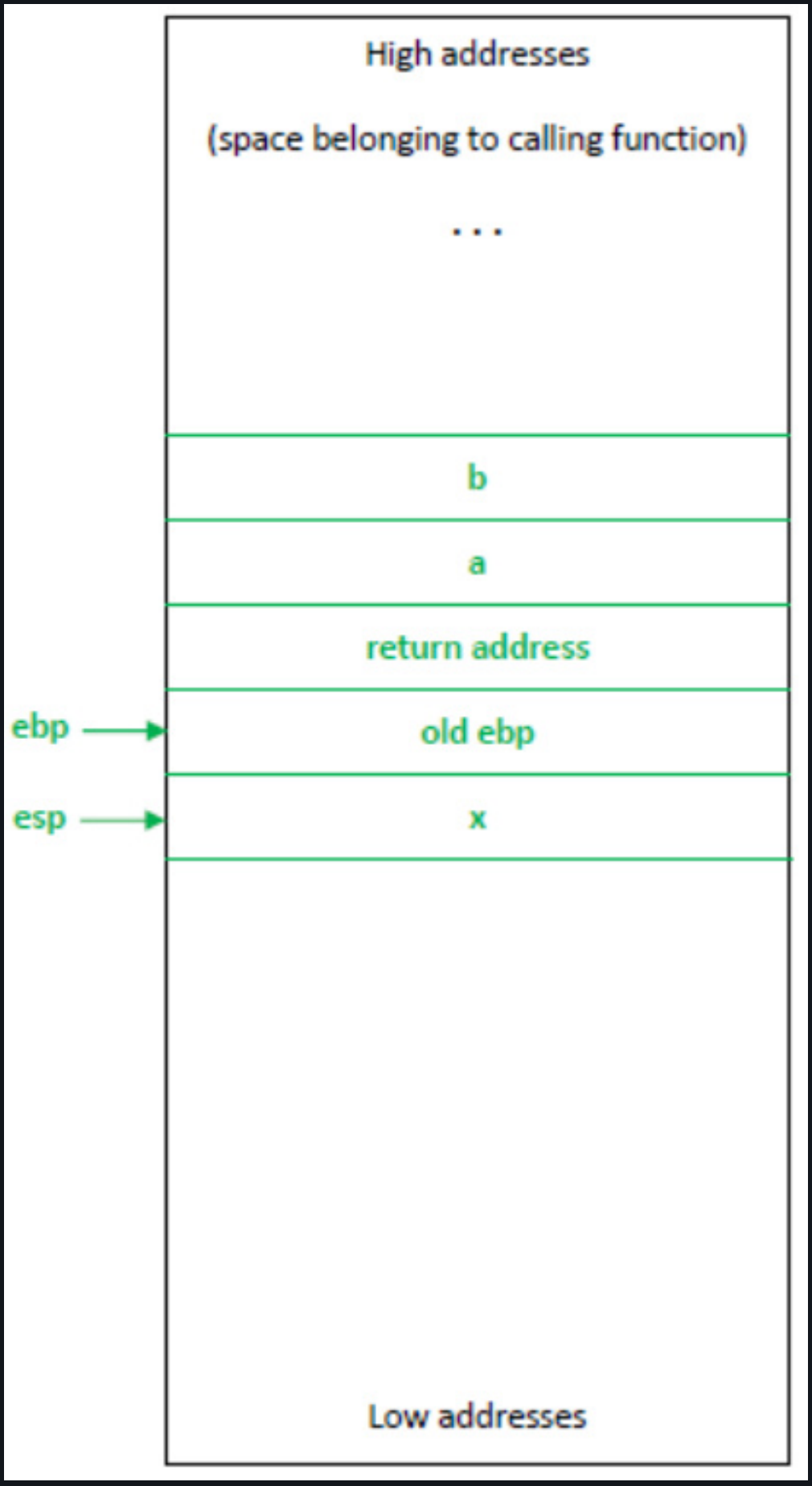
```
int foo(int a, int b) {
  int x;
  // draw frame at this point in execution
  x = a+b;
  return x;
}
```

Resolução

Stack frame de `foo` imediatamente antes de `x = a + b` (sintaxe cdecl):

```
altas -> +------------------+
 | a | 12(%ebp)
 | b | 8(%ebp)
 | endereço retorno | 4(%ebp)
 ebp ---- | ebp antigo | 0(%ebp) <- %ebp
 | x | -4(%ebp) <- %esp
baixas -> +------------------+
```

`%ebp` aponta para o ebp antigo guardado (base do frame); `%esp` aponta para o topo da pilha, logo abaixo dos locais; os parâmetros ficam acima do endereço de retorno.


High addresses
(space belonging to calling function)
. . .
b
a
return address
ebp
old ebp
esp
x
Low addresses

Questão 8 (15 pontos)

Tendo adicionado arrays ao MiniJava, queremos adicionar a instrução `for-each` do Java. A sintaxe é

```
for ( type loop_var : array_expr ) statement
```

A instrução executa repetidamente com `loop_var` recebendo os valores sucessivos de `array_expr`, começando no elemento 0. `loop_var` é declarado na instrução, com tipo `type`, que deve corresponder ao tipo dos elementos do array. Exemplo:

```
sum = 0; count = 0;
for (int x : v) { sum = sum + x; count = count + 1; }
```

(a) Desenhe o nó de AST necessário para suportar a nova instrução.

(b) Que mudanças são necessárias na análise semântica do compilador para lidar com `for-each`?

(c) Que código IL seria gerado por `for-each`? Use a notação da aula (ex.: `t0 = t1 + t2; iffalse t17 goto L;`), não as classes IL específicas do projeto.

Resolução

(A) Nó da AST (figura abaixo): um nó de loop com quatro filhos — a declaração da variável de loop, a expressão do array e o corpo (statement).

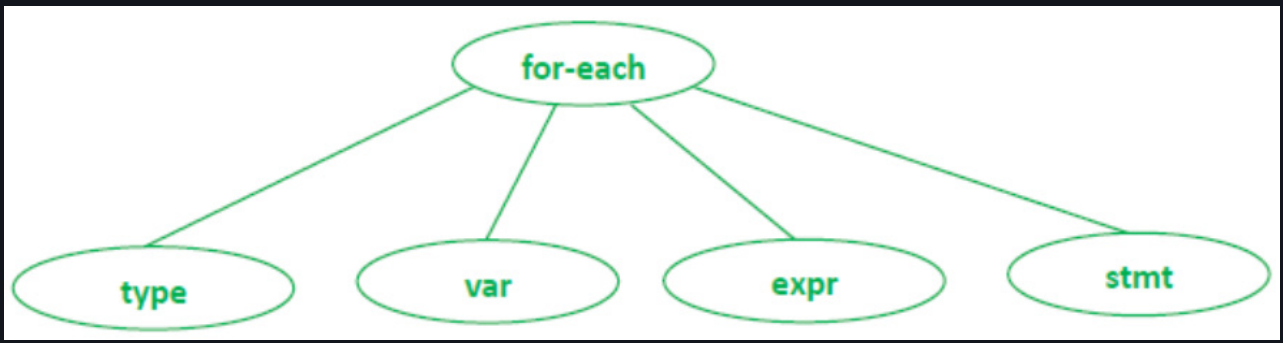
(B) Checagens:

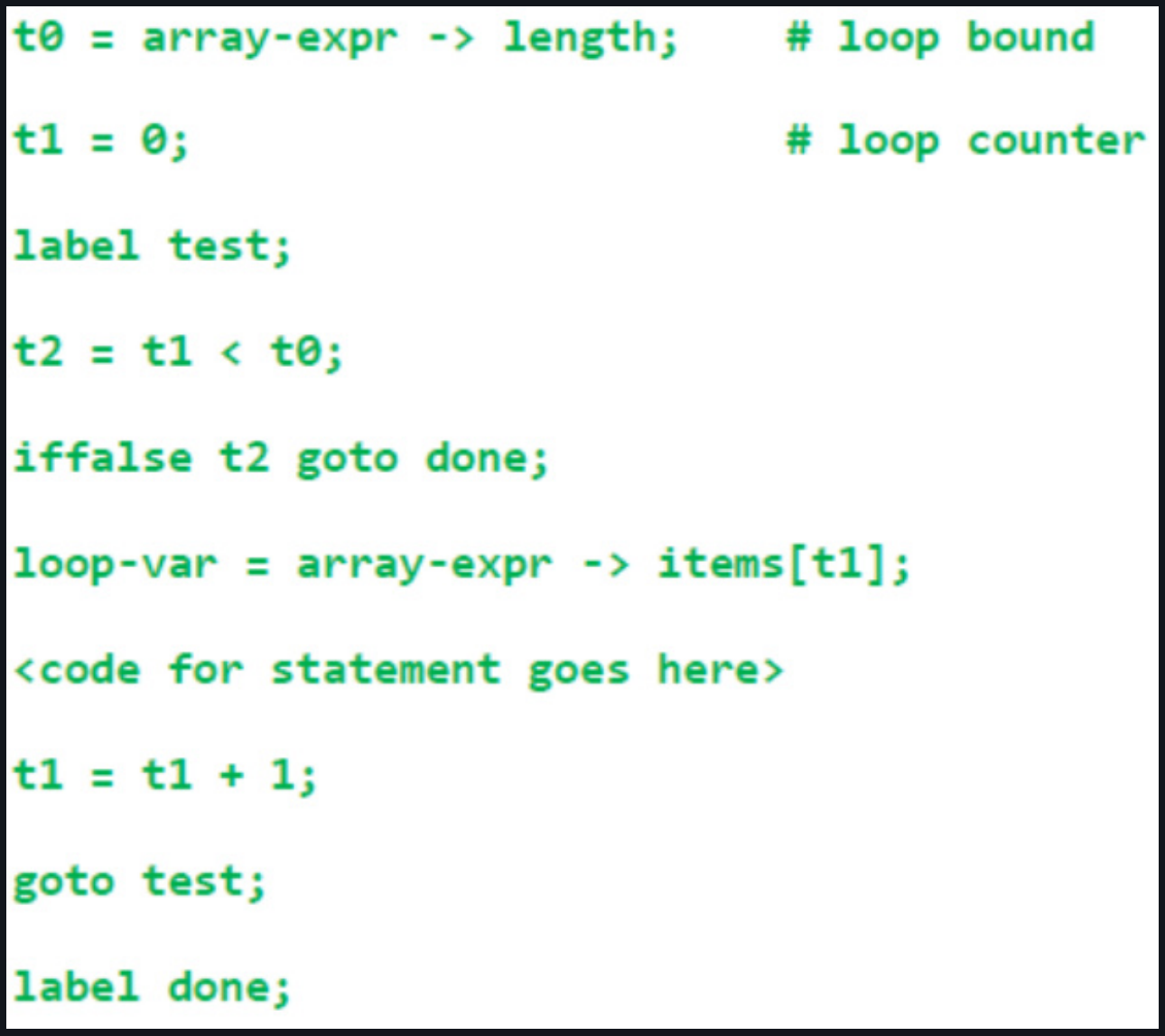
- A expressão deve ter tipo de array.
- O tipo dos elementos do array deve ser igual (ou compatível com) o tipo de `loop_var`.
- A variável de loop é inserida na tabela de símbolos do escopo do loop com seu tipo declarado (e é inacessível fora dele).

(C) Uma solução:

```
t0 = array_expr -> length # limite
t1 = 0 # contador
label test
t2 = t1 < t0
iffalse t2 goto done
loop_var = array_expr -> items[t1]
<corpo da instrução>
t1 = t1 + 1 # (ou +4, se o índice for deslocamento em bytes)
goto test
done:
```

Uma implementação completa também testaria se `array_expr` não é nulo antes do loop.





Questão 9 (12 pontos)

Para esta questão gostaríamos de realizar a propagação local, constante e dobrável, e a eliminação de atribuição morta no código IL gerado para as seguintes instruções:

```
x = 10;
```

i = 3; a [i] = x; Suponha que x, i, e o array a todos contém variáveis inteiras de 4 bytes. Na tabela abaixo, a primeira coluna mostra o código IL original é gerado por estas instruções.

(a) Preencha na segunda coluna com as declarações da primeira coluna após a propagação de constantes locais e dobre (em tempo de compilação aritmética) foram realizados. Você deve assumir que a propagação constante controla o conteúdo das variáveis armazenadas no stack frame, bem como temporárias como t2.

(b) Na terceira coluna, marque a caixa "delete" se a declaração seria eliminada pela eliminação da atribuição morta após executar as otimizações no item (a). Você deve assumir que a variável x e o array a estão vivos na saída do bloco de código, e a variável i não está viva na saída.

Original code	After constant propagation and folding	Deleted by dead code elimination? (mark X if so)
t1 = 10	t1 = 10	X
*(fp + xoffset) = t1	*(fp + xoffset) = 10	
t2 = 3	t2 = 3	X
*(fp + ioffset) = t2	*(fp + ioffset) = 3	X
t3 = *(fp + xoffset)	t3 = 10	X
t4 = *(fp + ioffset)	t4 = 3	X
t5 = t4 * 4	t5 = 12	X
t6 = t5 + fp	t6 = 12 + fp	
*(t6 + aoffset) = t3	*(t6 + aoffset) = 10	

Resolução

Após propagação de constantes e dobragem, e marcando com **X** o que a eliminação de atribuição morta remove (x e o array `a` vivos na saída; `i` não vivo):

Original	Depois da propagação/dobragem	Morta?
`t1 = 10`	`t1 = 10`	X
`*(fp + xoffset) = t1`	`*(fp + xoffset) = 10`	
`t2 = 3`	`t2 = 3`	X
`*(fp + ioffset) = t2`	`*(fp + ioffset) = 3`	X
`t3 = *(fp + xoffset)`	`t3 = 10`	X
`t4 = *(fp + ioffset)`	`t4 = 3`	X
`t5 = t4 * 4`	`t5 = 12`	X
`t6 = t5 + fp`	`t6 = 12 + fp`	
`*(t6 + aoffset) = t3`	`*(t6 + aoffset) = 10`	

Os stores de `x` e de `a` permanecem porque `x` e `a` estão vivos na saída; o store de `i` some porque `i` não está vivo. Observação: se o compilador também dobrasse `12 + fp` usando o deslocamento de `a`, a atribuição a `t6` também ficaria morta.

Questão 10 (12 pontos)

Um pouco de coloração de registradores. Considere o seguinte fragmento de código:

```
a = read();
b = a * a;
c = read();
```

```
d = b * c;
if (d > a) {
 e = d + 1;
} else {
 e = a + 1;
}
print(a);

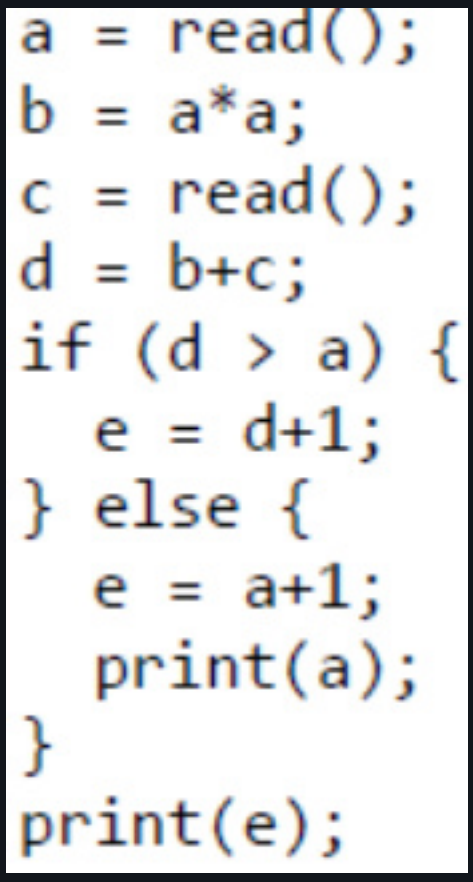
print(e);
```

(A) Desenhe o gráfico de fluxo de controle, mantendo o diagrama no lado esquerdo do papel.

(B) À direita do gráfico de controle, mostre nitidamente as faixas (live ranges) das variáveis.

(C) Abaixo, desenhe o grafo de interferência das variáveis, no lado esquerdo do papel.

(D) À direita do grafo de interferência, indique quais grupos de variáveis podem ocupar o mesmo registrador. Não é preciso seguir explicitamente o algoritmo de coloração; qualquer resposta que use o mínimo de registradores vale.



Resolução

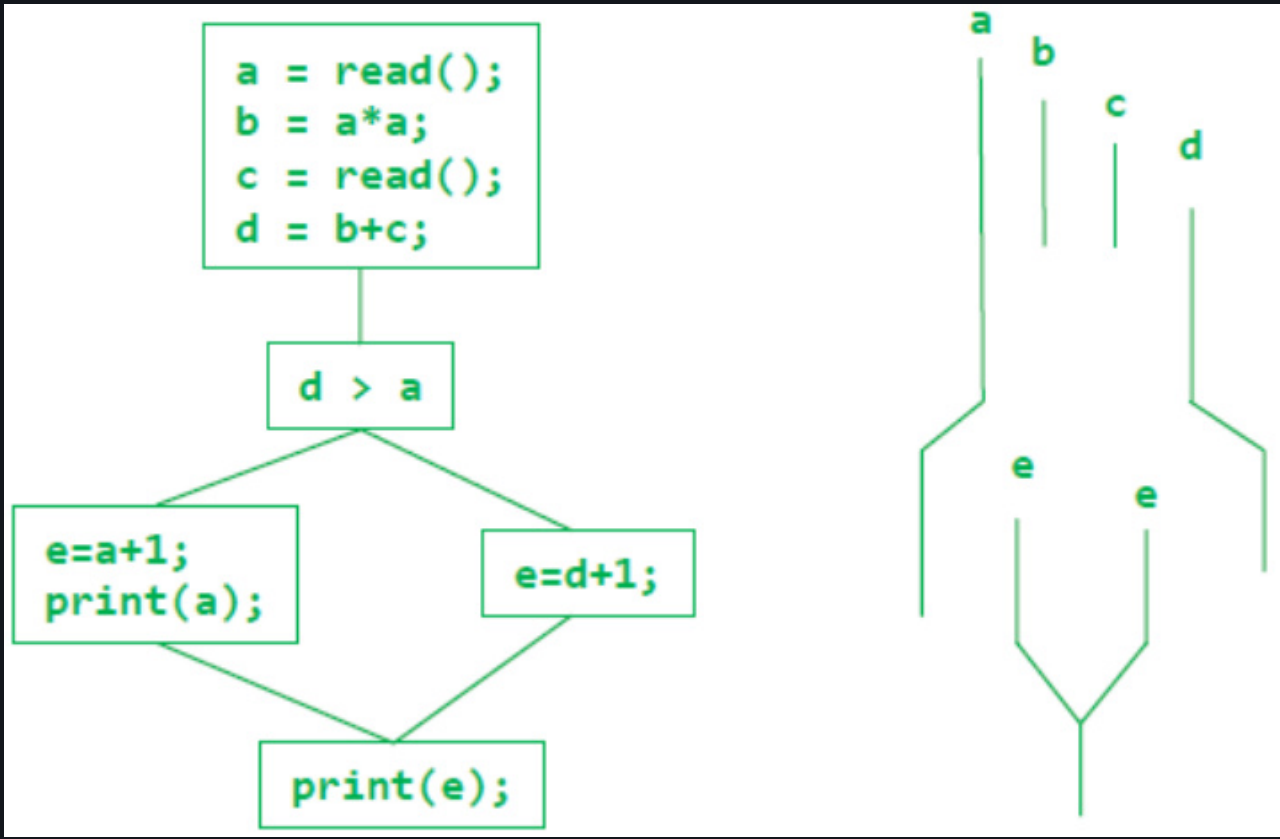
(A) e (B) Gráfico de controle e faixas vivas: figura abaixo. `b` e `c` só vivem até `d = b*c`; `d` vive até `e = d+1`; `a` vive até `print(a)`; `e` vive até `print(e)`.

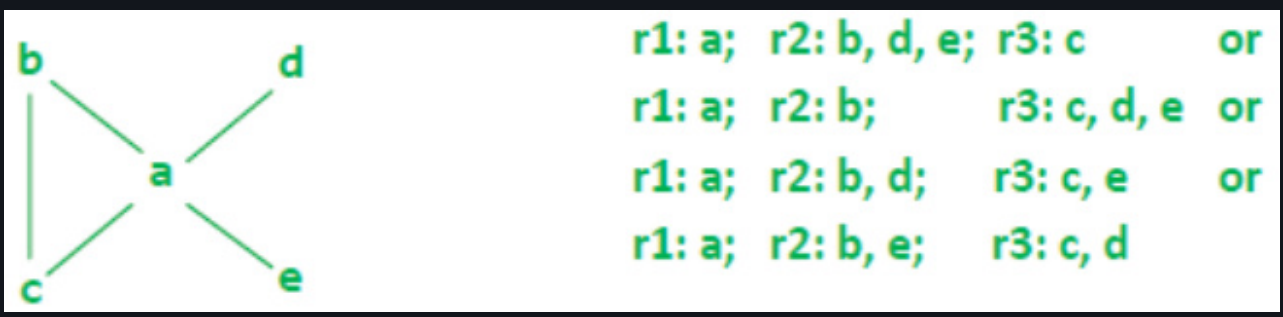
(C) Grafo de interferência (figura abaixo): arestas `a-b`, `a-c`, `a-d`, `a-e` e `b-c`. Não há aresta `b-d`/`c-d` (b e c morrem em `d = b*c`), nem `d-e` (d morre quando `e` nasce), nem `b-e`/`c-e`.

(D) Três registradores são necessários; uma atribuição possível:

```
r1 = {a}
r2 = {b, d, e}
r3 = {c}
```

Outras atribuições com 3 registradores também valem, ex.: `r2 = {b, d}` e `r3 = {c, e}`. O único grupo proibido é `{b, c}`, que interfere.


a = read();
b = a*a;
c = read();
d = b+c;
d > a
e=a+1;
print(a);
e=d+1;
print(e);
a
b
c
d
e
e


b
d
a
c
e
r1: a; r2: b, d, e; r3: c or
r1: a; r2: b; r3: c, d, e or
r1: a; r2: b, d; r3: c, e or
r1: a; r2: b, e; r3: c, d

F2 — CSE401 final inverno 2010

Pergunta 1 (14 pontos)

Questão repetida. Este enunciado e esta resolução são idênticos aos de **Questão 1 (14 pontos)**, de **I.2 — Segunda Prova do André · prova do meio (gabarito oficial, inglês)** (página 12). Não há nada novo aqui — se você já copiou aquela, pode pular esta.

Pergunta 2 (6 pontos)

O compilador com otimização que estamos desenvolvendo move computações para fora de loops quando elas sempre produzem o mesmo valor. Dado o código original

```
for (i = 0; i < n; i++)
 a[i] = sqrt(x/y);
```

o otimizador o reescreve para evitar recalcular dentro do loop:

```
temp = sqrt(x/y);
for (i = 0; i < n; i++)
 a[i] = temp;
```

(`sqrt` é função de biblioteca: sempre devolve o mesmo resultado para a mesma entrada e não tem efeitos colaterais; todos os valores são `double`.) Considerando execução em um único processador, sem concorrência, essa otimização é sempre segura e correta? Justifique brevemente.

Resolução

Não. Na versão otimizada, `sqrt(x/y)` é sempre avaliada, mas na versão original ela só é avaliada se o loop executar (ou seja, se `n > 0`). Com `n <= 0`, a avaliação fora do loop pode lançar erro de divisão por zero (`y = 0`) ou erro de domínio (`x/y < 0`), algo que nunca aconteceria no código original.

A versão segura é guardar a avaliação condicionada à execução do loop:

```
if (n > 0)
 temp = sqrt(x/y);
for (i = 0; i < n; i++)
 a[i] = temp;
```

Pergunta 3 (6 pontos)

Todos os compiladores otimizar executar a eliminação de código morto, o que elimina o código que nunca é executado , ou que calcula valores que nunca são usados . A maioria dos compiladores de otimização executar eliminação de código morto várias vezes durante os passes de otimização. Por quê? Por que não há qualquer vantagem para fazê-lo mais de uma vez ?

Resolução

Porque **outras otimizações geram novo código morto**. Ao eliminar uma computação, valores intermediários deixam de ser usados e se tornam mortos — então vale rodar a eliminação de novo, depois de cada passagem que possa tê-la criado.

Além disso, rodar cedo é barato e reduz a quantidade de código intermediário que as passagens seguintes precisam processar.

Pergunta 4 (10 pontos)

Considere o código à esquerda (variáveis globais/estáticas) e a otimização à direita, que calcula `a + b` uma única vez em um temporário (registrador/variável do compilador, invisível para o programa):

```
if ( a + b < 0 ) { if ( a + b < 0 ) {
 x = a + b; temp = a + b;
} else { if ( temp < 0 ) {
 y = a + b; x = temp;
} } else {
 y = temp;
 }
 }
```

(A) Essa otimização é sempre legal (segura e correta) quando o código roda em um único thread sem outros threads simultâneos? Argumente brevemente.

(B) E quando o código roda em um segmento de um programa multithreaded? Argumente brevemente.

Resolução

(A) **Sim.** Sem outros threads, `a` e `b` não mudam entre as avaliações; reavaliar `a + b` produziria o mesmo valor, então calcular uma única vez é seguro.

(B) **Não.** Como `a` e `b` são globais, outro thread pode modificá-las a qualquer momento. No original, `a + b` é avaliado duas vezes e pode dar valores diferentes; na versão otimizada, a mudança entre a primeira avaliação e as atribuições a `x`/`y` é ignorada — o comportamento observável muda.

Pergunta 7 (12 pontos)

Questão repetida. Este enunciado e esta resolução são idênticos aos de **Questão 7 (12 pontos) — A little coloring**, de **I.2 — Segunda Prova do André · prova do meio (gabarito oficial, inglês)** (página 13). Não há nada novo aqui — se você já copiou aquela, pode pular esta.

F3 — CSE401 final outono 2010

Questão 1 (10 pontos)

Para cada uma das tarefas abaixo, indique a fase inicial do compilador que sempre pode executar essa tarefa ou detectar a situação. Considere um compilador convencional que gera código nativo (x86, MIPS etc.) para uma linguagem como C++ ou Java.

Abreviações: `scan` = scanner (varredura); `parse` = parser (análise sintática); `sem` = semântica/verificação de tipo; `opt` = otimização; `instr` = seleção de instruções e scheduling; `reg` = alocação de registradores;

```
run = tempo de execução (o código compilado está rodando); can't = não pode ser feito durante a compilação nem durante a execução.
```

(a) Um comentário que começa com `/*` termina com `*/` antes do final do arquivo.

(b) Uma variável é declarada no máximo uma vez em um escopo (função, classe etc.).

(c) Acrescentar código para armazenar temporariamente um valor quando não há registradores suficientes para todos os valores vivos em um determinado ponto do programa.

(d) Substituir indexação de vetor (`a[i] = 0; i++`) e teste (`i < n`) por aritmética de ponteiros (`*p = 0; p++`) e teste de ponteiro (`p < &a[n]`).

(e) Avisar o programador que a declaração de uma variável em uma subclasse "shadows" (sombreia) a declaração da mesma variável em uma superclasse.

(f) Detectar que existe uma cláusula extra `else stmt` no programa que não está associada com o `if` anterior.

(g) Detectar que, em uma referência de matriz `a[k]`, o índice `k` está fora dos limites (em Java, onde os limites devem ser verificados).

(h) Garantir que um ponteiro (referência) `p` nunca será `NULL`.

(i) Garantir que, em uma referência `p.x` (`p->x` em C/C++), o valor referenciado por `p` terá um campo `x`, assumindo que `p` não é `NULL`.

(j) Usar os modos de endereçamento do x86 para calcular `x * 5` com `leal (%eax,%eax,4), %eax`.

(obs.: a lista de tarefas foi reconstruída a partir da resolução — o documento original não preservou a lista do enunciado.)

Resolução

(a) **scan** — o pareamento de comentários é feito pelo scanner.

(b) **sem** — checagem de declarações duplicadas no mesmo escopo.

(c) **reg** — spilling: armazenar temporariamente valores que não cabem em registradores.

(d) **opt** — otimização (substituição de indexação por aritmética de ponteiros).

(e) **sem** — aviso de sombreamento (shadowing) de variável entre subclasse e superclasse.

(f) **parse** — `else` extra sem `if` correspondente é erro de sintaxe.

(g) **run** — os limites de vetor em Java são verificados em tempo de execução.

(h) **can't** — não se pode garantir estaticamente que um ponteiro nunca será nulo.

(i) **sem** — verificação de tipo: `p.x` exige que o tipo de `p` tenha o campo `x` (assumindo `p` não nulo).

(j) **instr** — aproveitamento dos modos de endereçamento na seleção de instruções: `x * 5` vira `leal (%eax,%eax,4), %eax`.

Questão 2 (34 pontos)

Estruturas de dados de tempo de execução e código. Considere duas classes Java que representam personagens de uma simulação e uma terceira classe com um pequeno programa principal:

```
javapublic class Character {
 int x, y; // coordinates
 int age;
 void move(int dx, int dy) {
 x = x + dx; y = y + dy;
 }
 int getX() { return x; }
 int getY() { return y; }
 int getAge() { return age; }
}

public class Elf extends Character {
 int weight;
 int age;
 void setAge(int years) { age = years; }
 int getAge() { return age; }
}

public class Main {
 public static void main(String[] useless) {
 Character santa = new Character();
 Character bernie = new Elf();
 santa.move(12, 24);
 ((Elf) bernie).setAge(42);
 }
}
```

(A) (8 pontos) Estruturas de dados em tempo de execução. Na página seguinte, desenhe um diagrama das estruturas de dados em tempo de execução após a execução das declarações de variáveis do `main`. Mostre as variáveis do `main`, os objetos a que elas apontam e como objetos e campos são organizados na memória, com deslocamentos numéricos (0, 4, 8, ...) para cada campo, para uso nas partes seguintes. Não é preciso mostrar os detalhes do frame de `main` — basta mostrar os ponteiros para os objetos. Acrescente ao diagrama os mecanismos necessários para expedição dinâmica de métodos em Java (ex.:

vtables), com deslocamentos numéricos das entradas da tabela (0, 4, 8, ...). Ignore construtores.

(B) (7 pontos) Frames de pilha. Suponha que começamos a executar o `main` e avançamos até o corpo do método `move`, logo após `santa.move(12, 24)`, depois de executar a chamada e o prólogo da função (frame setup e ajuste do ponteiro-base), mas antes da primeira instrução de atribuição do corpo de `move`. Desenhe as setas indicando onde `ebp` (frame pointer) e `esp` (stack pointer) apontam após o prólogo. A imagem deve mostrar variáveis e parâmetros de ambos os métodos e seus deslocamentos numéricos em cada frame (haverá dois conjuntos de números de offset: um para o frame de `main` e outro para o frame de `move`).

(C) (9 pontos) Chamada de método. Agora suponha que a execução voltou ao `main`, depois que `move` terminou. A próxima linha é `((Elf) bernie).setAge(42);`. Traduza essa linha para assembly x86 como ela seria compilada no `main`. Assuma o frame de `main` organizado como na parte anterior e os layouts de objetos/vtables como na parte (A). Use as convenções de chamada descritas em aula, usando o registrador `%ecx` como ponteiro `this`. Você pode assumir que não é gerado código para verificar o cast

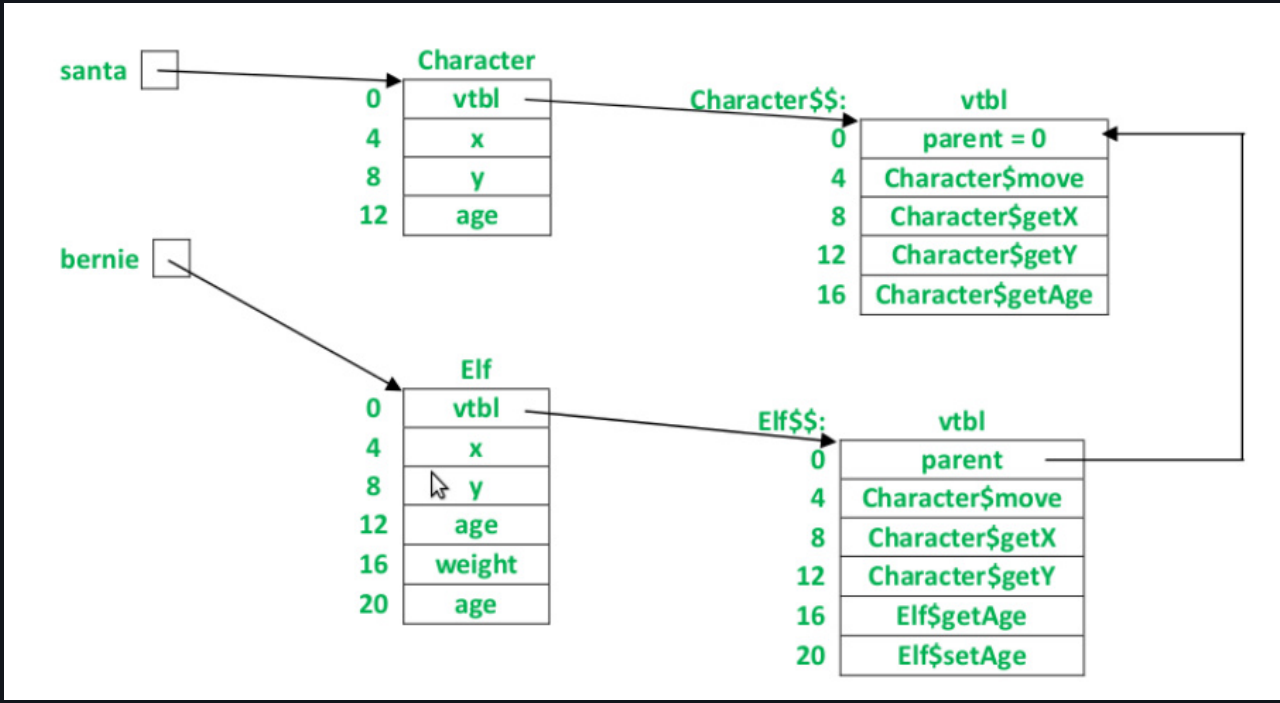
`(Elf)` em tempo de execução. Qualquer código x86 razoável que obedeça às convenções de chamada vale (você não precisa reproduzir exatamente o código que seu compilador geraria). Use sintaxe assembly x86 GNU/AT&T.

(D) (10 pontos) Implementação de método. Última parte! Deve uma tradução x86 do método `setAge` da classe `Elf`. Seu código deve incluir todo o código do método, com prólogo e retorno. Como o método é

`void`, não é preciso colocar valor de retorno em `%eax`. Inclua o prólogo e o epílogo habituais para salvar/restaurar o frame pointer (embora existam outras maneiras de escrever o código sem eles).

Resolução

De acordo com o diagrama oficial:



Os deslocamentos dos objetos e das vtables poderiam ser diferentes, embora a maioria das pessoas tenha chegado a estes layouts. O que é obrigatório: as partes comuns dos objetos `Character` e `Elf` e o início das duas vtables devem coincidir. Estritamente falando, as entradas da vtable são ponteiros para o código de máquina armazenado em outro lugar; na solução foram usados rótulos com os nomes dos métodos.

Layout do diagrama:

```
santa -> Character { 0: vtbl, 4: x, 8: y, 12: age }
bernie -> Elf { 0: vtbl, 4: x, 8: y, 12: age, 16: weight, 20: age }

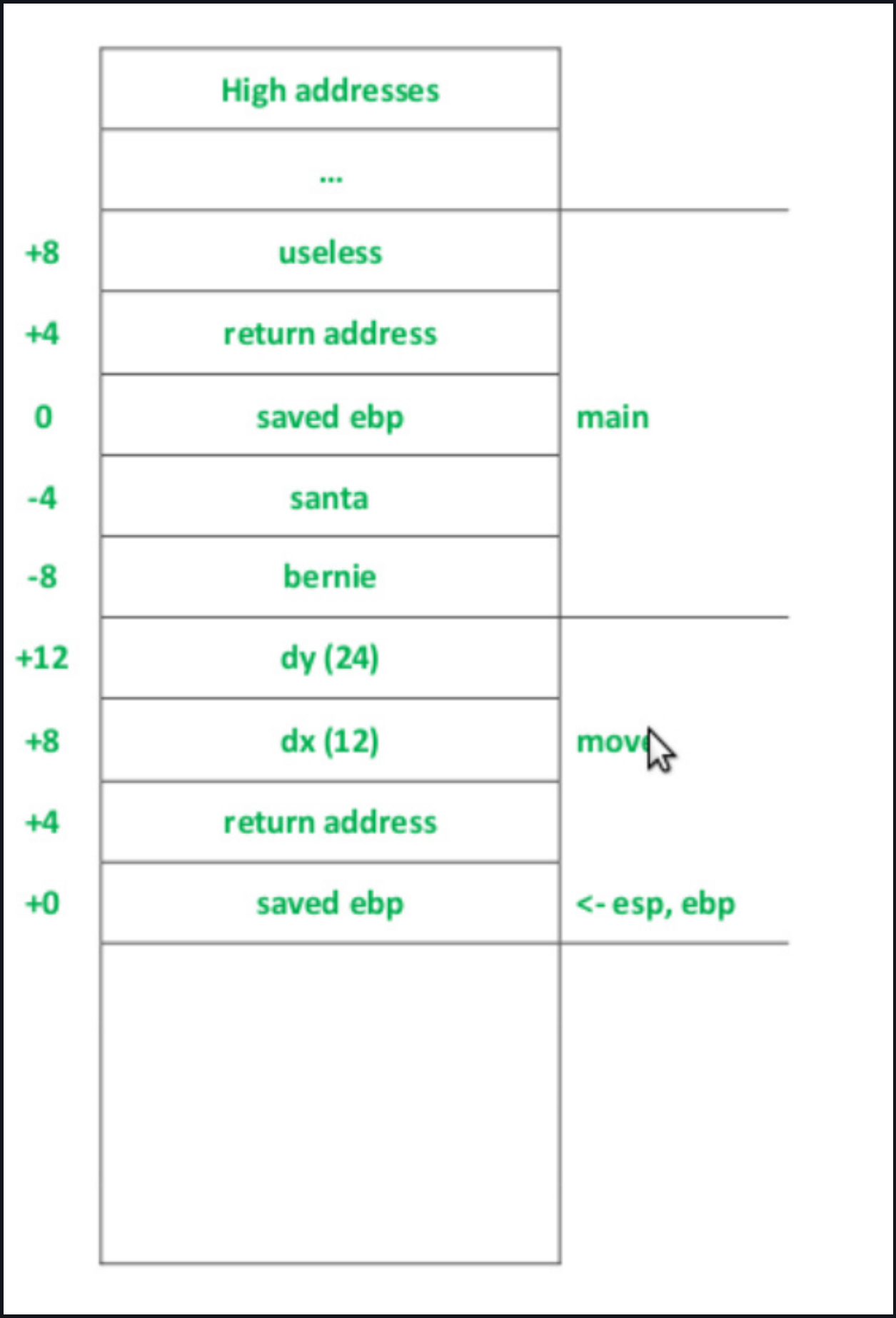
Character$$: vtbl { 0: parent = 0, 4: Character$move, 8: Character$getX,
 12: Character$getY, 16: Character$getAge }
Elf$$: vtbl { 0: parent -> vtbl de Character, 4: Character$move,
 8: Character$getX, 12: Character$getY,
 16: Elf$getAge, 20: Elf$setAge }
```

(B) Frames de pilha após o prólogo de `move`, antes da primeira atribuição do corpo (o `this` fica em `%ecx`):

```
+----------------------------+
| ... (chamador) | frame de main
| useless +8 |
| end. retorno +4 |
| ebp antigo 0(%ebp_main) |
| santa -4(%ebp_main) |
| bernie -8(%ebp_main) |
+----------------------------+
| dy (24) +12 | frame de move
| dx (12) 8 |
| end. retorno 4 |
| ebp antigo 0 | <- %ebp (aponta para o ebp antigo de main)
| %esp ------+ <- logo abaixo de ebp (sem locais)
+----------------------------+
```

Após `pushl %ebp; movl %esp, %ebp`, `%ebp` aponta para o ebp antigo salvo (que guarda o `%ebp` de

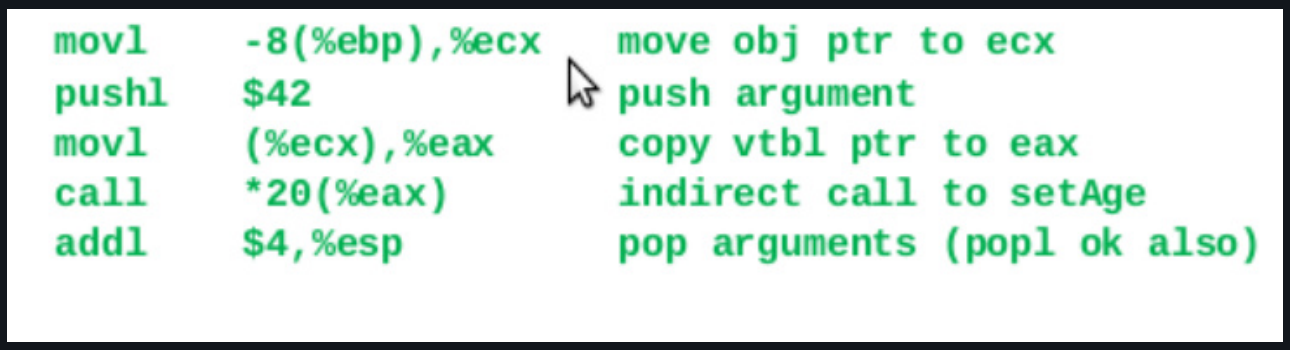
`main`) e `%esp` fica em `%ebp` (nenhuma variável local). Os parâmetros `dx` e `dy` ficam em `8(%ebp)` e `12(%ebp)`; o endereço de retorno em `4(%ebp)`. Nota do gabarito: os parâmetros de `move` não são anônimos — têm nomes além de valores (muitas pessoas omitiram os nomes na solução; isso não foi deduzido na correção).




```
(C) Chamada de ((Elf) bernie).setAge(42):
```

```
movl -8(%ebp), %ecx # ponteiro obj -> ecx (bernie está em -8(%ebp))
pushl $42 # empurra o argumento
movl (%ecx), %eax # copia o ponteiro da vtable para eax
call *20(%eax) # chamada indireta para setAge (offset 20 na vtable do Elf)
addl $4, %esp # retira o argumento da pilha (popl %ecx também serveia)
```

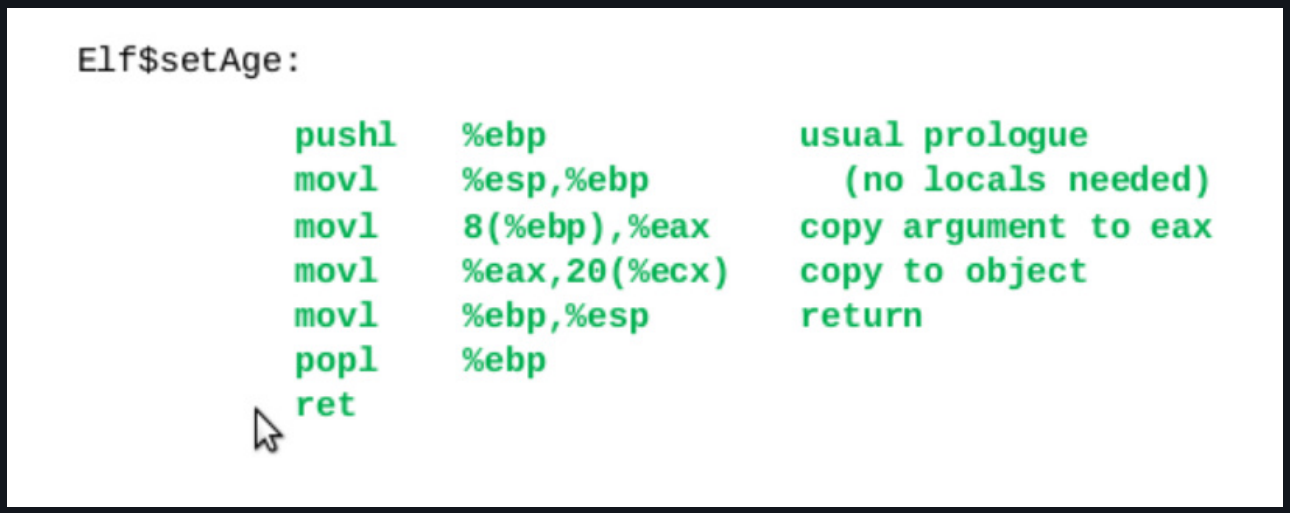
Achados no exame: como `bernie` está declarado como `Character`, o cast `(Elf)` é irrelevante em tempo de execução; o dispatch virtual resolve pelo valor do bit da vtable armazenado no próprio objeto.



(D) Método `Elf$setAge`:

```
Elf$setAge:
 pushl %ebp # prólogo usual
 movl %esp, %ebp # (sem variáveis locais)
 movl 8(%ebp), %eax # copia o argumento years para %eax
 movl %eax, 20(%ecx) # grava em age do objeto (offset 20 em Elf)
 movl %ebp, %esp # retorno
 popl %ebp
 ret
```

Notas da resolução: o `movl`/`popl` final poderia ser substituído por uma instrução `leave`. Várias soluções tentaram usar uma única instrução `movl 8(%ebp), 20(%ecx)` para copiar o dado; isso não pode ser feito, porque uma instrução x86 só admite uma especificação de endereço por vez.



Questão 3 (32 pontos)

Compilador "pirata": vamos adicionar um novo tipo de laço de contagem ao nosso compilador MiniJava. A sintaxe é:

```
id in exp1 to exp2 do stmt
```

A ideia é que `stmt` seja executado repetidamente, com a variável `id` assumindo os valores sucessivos `exp1`, `exp1 + 1`, `exp1 + 2`, ..., `exp2` a cada execução. Especificamente: primeiro `id` recebe `exp1`; em

seguida verifica-se se `id` é menor ou igual a `exp2`; se for, o corpo `stmt` é executado e então `id` é incrementado em 1. Esse ciclo teste-executa-incrementa repete até o teste ficar falso (o valor de `id` excede `exp2`).

A variável `id` deve estar declarada previamente na função e ter tipo `int`; deve ser variável local da função que contém o laço (não pode ser variável de instância, global ou de classe). As expressões `exp1` e `exp2` são avaliadas apenas uma vez, antes da atribuição inicial a `id`, e não são reavaliadas durante a execução do laço. Além disso, o corpo `stmt` não pode conter atribuições à variável `id`. Em outras palavras: depois de avaliados `exp1` e `exp2`, sabemos exatamente quantas vezes o laço executará e quais valores `id` assumirá — a execução do corpo não pode alterar essa sequência (embora possa retornar da função, lançar exceção ou terminar anormalmente).

(A) (3 pontos) Quais fichas (tokens) novas precisariam ser adicionadas ao scanner e ao parser do MiniJava para suportar a nova declaração `for` na gramática original? Basta listar as fichas; não é necessário dar especificação JFlex ou CUP.

(B) (7 pontos) Complete a classe abaixo para definir uma nova classe de nó AST para a nova instrução. Só é preciso definir variáveis de instância e o construtor. Assuma pacote e imports apropriados, e não se preocupe com o código do visitante:

```
public class ForLoop extends Declaration {
 // variáveis de instância abaixo
 ...
 // construtor abaixo
}
```

(C) (6 pontos) Complete a especificação CUP abaixo para definir uma nova produção para a instrução, com a(s) ação(ões) semântica(s) necessária(s) para analisar e inserir na AST um nó `ForLoop` adequado (como definido na parte (B)). A regra do parser para `Statement` já foi estendida como mostrado:

```
Statement ::= ... | ForStatement:s {: RESULT = s; :}
```

(D) (6 pontos) Descreva as verificações necessárias na parte semântica/verificação de tipo para checar se a instrução é legal. Não é preciso dar código de um método visitante — basta descrever as checagens.

(E) (10 pontos) Descreva o código que seria gerado para a nova declaração de laço. Mostre as instruções, rótulos e demais código de linguagem de montagem gerado para a própria declaração, e mostre onde entraria o código gerado para `exp1`, `exp2`, a(s) atribuição(ões) a `id` e `stmt`. Se precisar criar variáveis temporárias ou salvar valores em algum lugar, deixe claro o que você está fazendo e onde os valores são armazenados.

Resolução

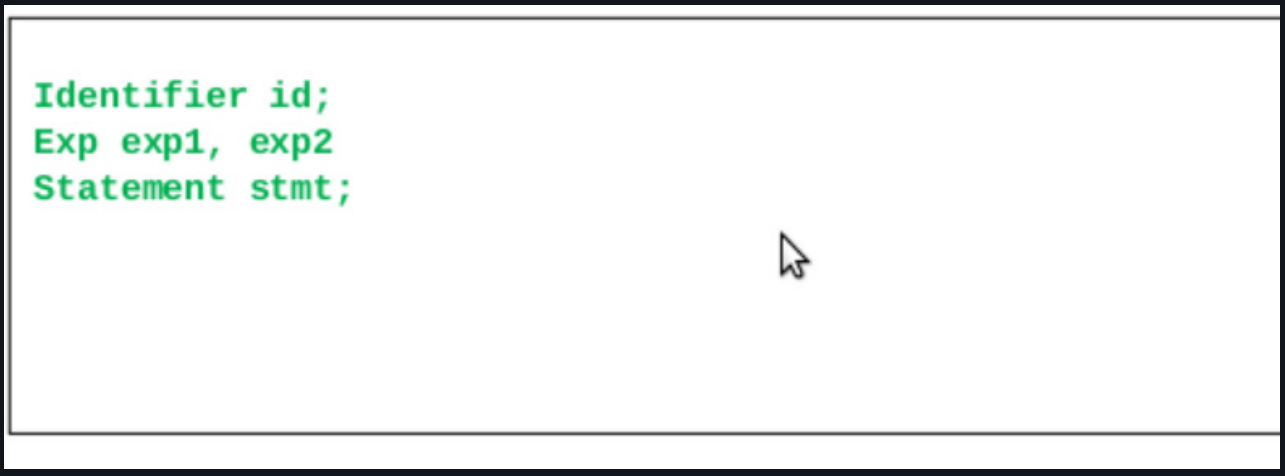
(A) Quatro fichas novas:

```
FOR IN TO DO
```

(`for` não é token no MiniJava original.)

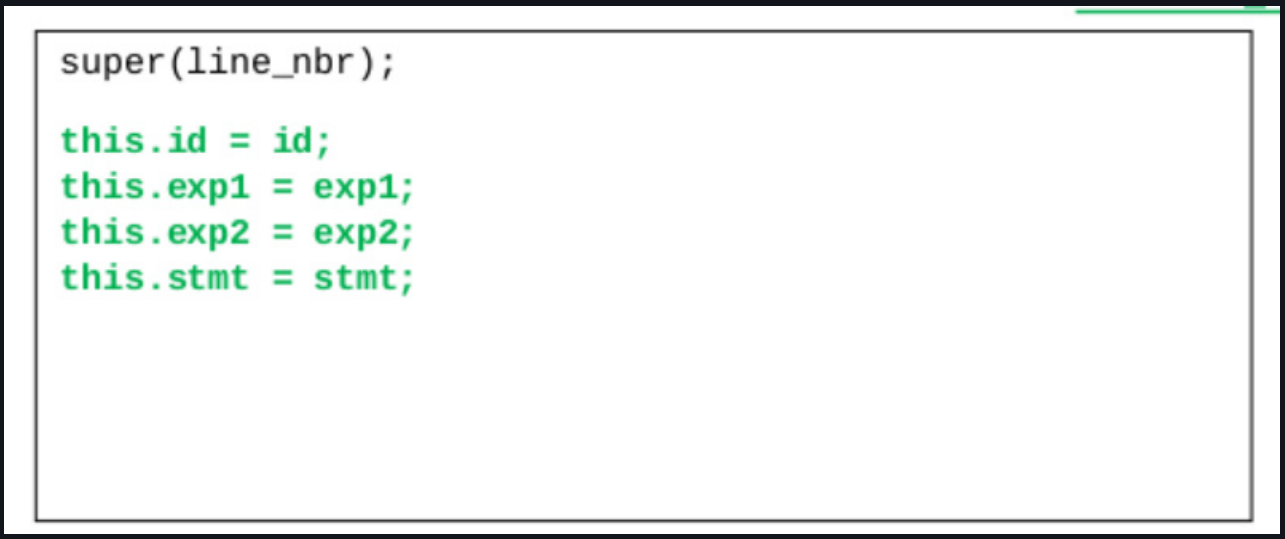
(B) Variáveis de instância:

```
Identifier id;
Exp exp1, exp2;
Statement stmt;
```



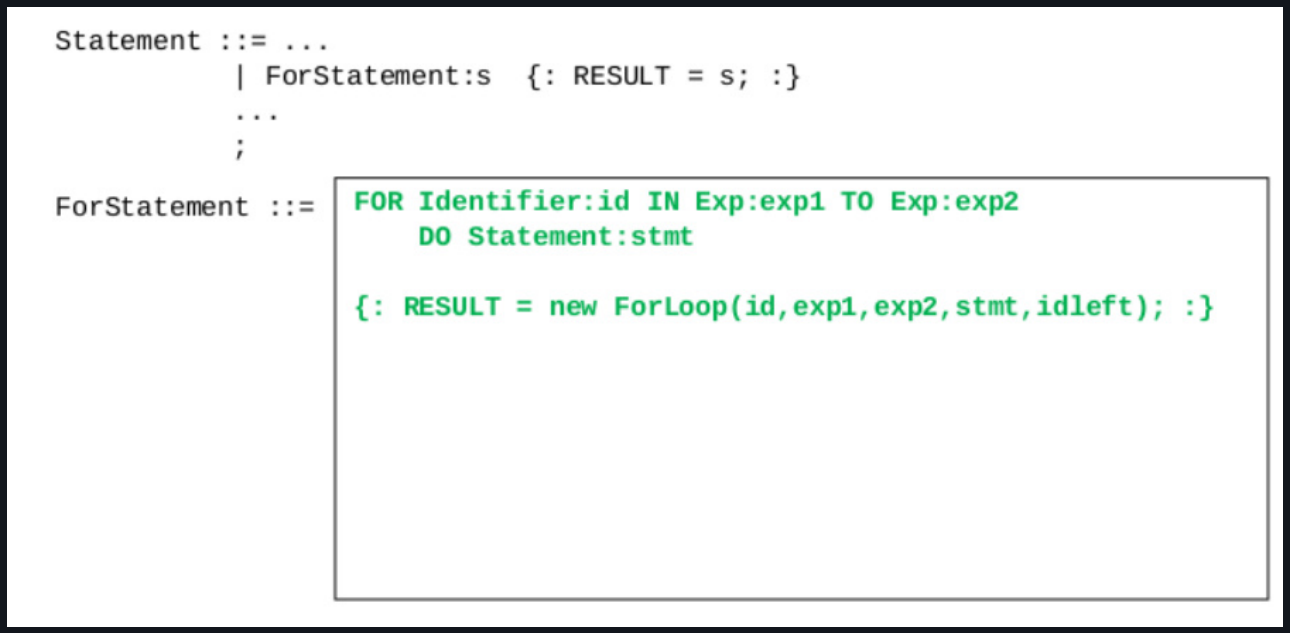
Construtor:

```
super(line_nbr);
this.id = id;
this.exp1 = exp1;
this.exp2 = exp2;
this.stmt = stmt;
```



(C) Especificação CUP:

```
ForStatement ::= FOR Identifier:id IN Exp:exp1 TO Exp:exp2 DO Statement:stmt
 {: RESULT = new ForLoop(id, exp1, exp2, stmt, idleft); :}
```



(D) Checagens da análise semântica:

1. `id` tem tipo `int`.
2. `id` é declarada localmente no escopo da função (não é global, de classe nem de instância).
3. `exp1` e `exp2` têm tipo `int`.
4. `stmt` não contém atribuições a `id`.

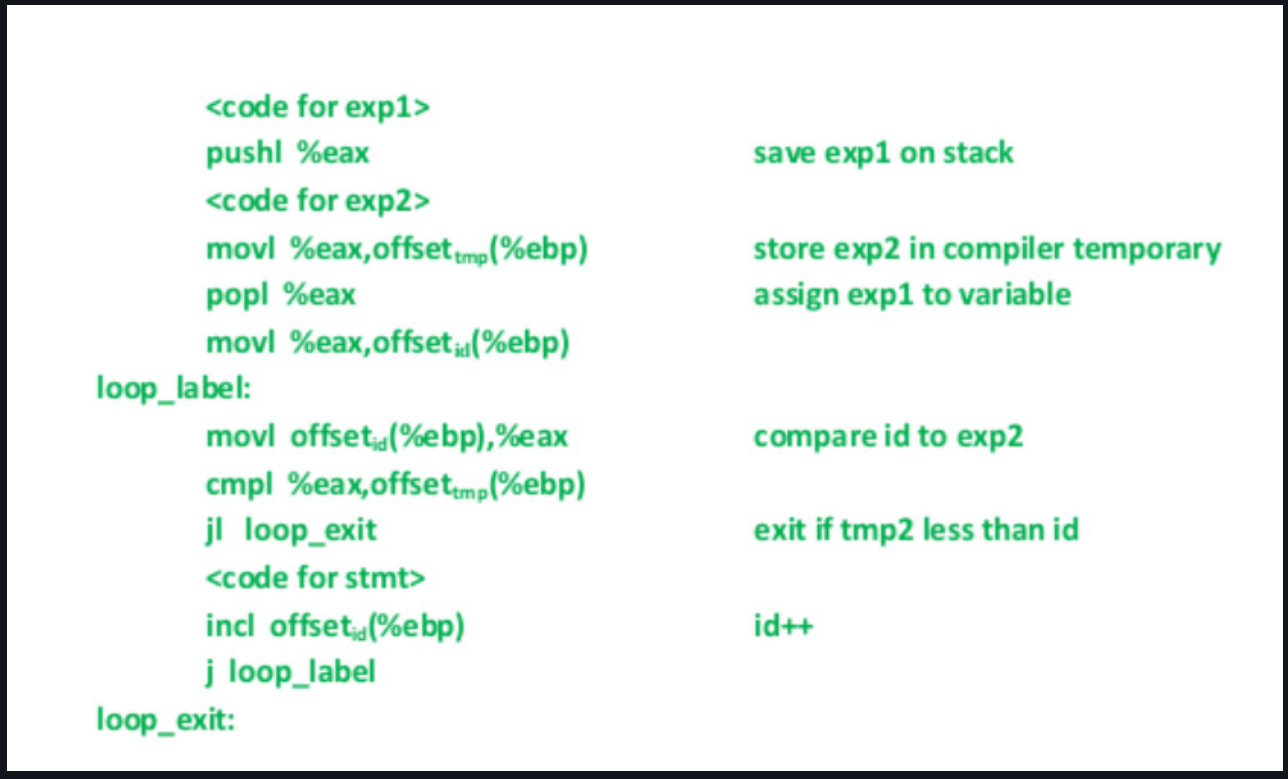
(E) Código gerado (solução simples, sem otimizações). É preciso espaço temporário para `exp1` enquanto `exp2` é avaliado, e um temporário do compilador para guardar `exp2` durante todo o laço (aqui: `exp1` é empurrado na pilha e `exp2` vai para uma variável local anônima):

```
<code for exp1>
pushl %eax # salva exp1 na pilha
<code for exp2>
movl %eax, offset2(%ebp) # guarda exp2 no temporário do compilador
popl %eax # atribui exp1 à variável
movl %eax, offset1(%ebp)

loop_label:
movl offset1(%ebp), %eax # compara id com exp2
cmpl %eax, offset2(%ebp)
jl loop_exit # sai se exp2 < id
<code for stmt>
incl offset1(%ebp) # id++
jmp loop_label

loop_exit:
```

Notas da resolução: `exp1` e `exp2` precisam ser avaliados — nessa ordem e antes da atribuição a `id` — para que algo como `for i in i+1 to i+10 do stmt` funcione. Além disso, `id` é uma variável comum e seu valor atual precisa ser armazenado em memória, pois pode ser referenciada pelo código do corpo; não pode ficar em um registrador durante todo o laço (o corpo pode usar registradores, então nem `id` nem `exp2` podem ficar, por exemplo, em `%edx`).



Questão 4 (12 pontos)

Alocação de registradores. Considere a função C:

```
int f(int a, int b) {
 int k, n;
 k = a;
 n = 0;
 while (k < b) {
 n = n + k;
 k = k + 1;
 }
 return n;
}
```

(A) Desenhe o grafo de interferência das variáveis e parâmetros desta função. Não é obrigatório desenhar o gráfico de fluxo de controle, mas pode ser útil esboçá-lo para ajudar na solução.

(B) Dê uma atribuição de (grupos de) variáveis a registradores usando o número mínimo de registradores possível, com base no grafo de interferência. Não é preciso seguir os passos do algoritmo de coloração explicitamente. Se houver mais de uma resposta com o número mínimo de registradores, qualquer uma vale. Use R1, R2, R3... para os nomes dos registradores.

Resolução

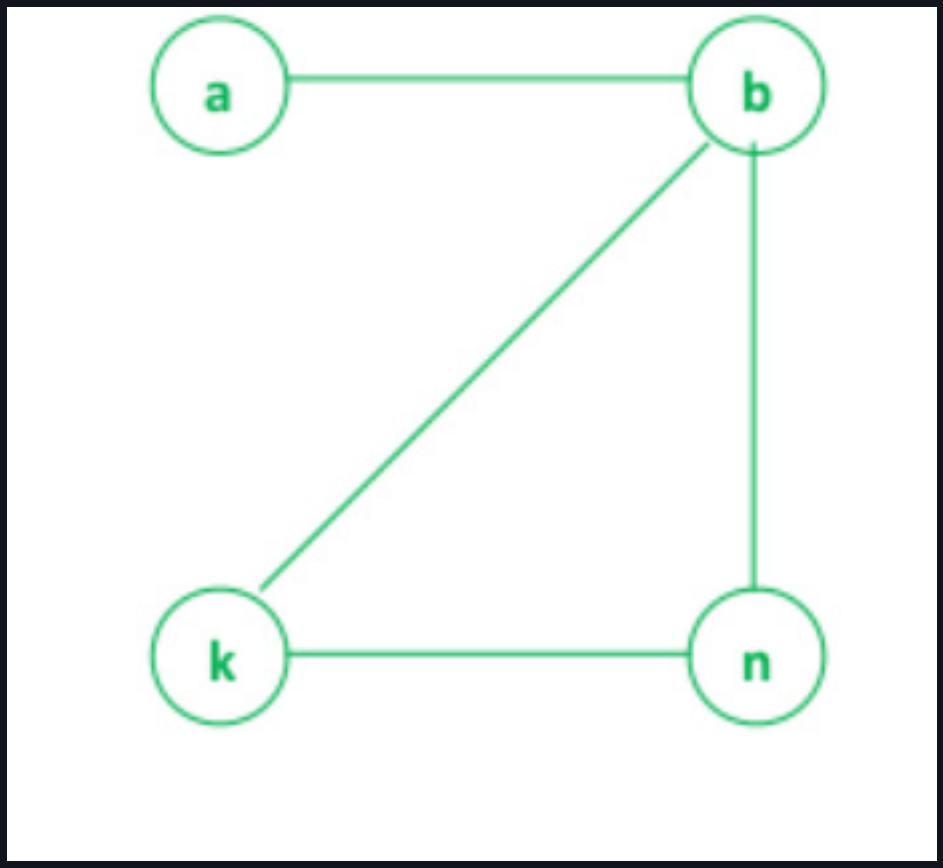
(A) Grafo de interferência (figura abaixo): triângulo entre `b`, `k` e `n` — cada par interfere, pois os três estão vivos dentro do laço. `a` não interfere com ninguém (`a` morre em `k = a`).

(B) São necessários **três** registradores:

```
R1 = {b}
R2 = {k}

R3 = {n}
```

Como `a` não interfere com `k` nem com `n`, ele pode ocupar o mesmo registrador de `k` (ou de `n`).



Questão 5 (12 pontos)

Um pouco de otimização. Vamos realizar propagação local de constantes e dobragem (aritmética em tempo de compilação), seguidas de eliminação de código morto. Temos uma função C com a seguinte declaração de matriz:

```
c
double A[10][10]; // 10x10 doubles de 8 bytes
```

A primeira coluna da tabela abaixo mostra o código de três endereços gerado para a atribuição `A[3][5] = A[3][5] + x;`:

(A) Preencha a segunda coluna com as declarações da primeira coluna depois de modificadas pela propagação de constantes e pela dobragem.

(B) Na terceira coluna, marque com um "X" a linha que será apagada pela eliminação de código morto depois das otimizações da parte (A).

Resolução

Após propagação/dobragem ("X" = eliminada como código morto):

Código original	Depois da propagação/dobragem	Morta?
`t1 = *(fp+xoffset)`	`t1 = *(fp+xoffset)`	
`t2 = 3`	`t2 = 3`	X
`t3 = t2*10`	`t3 = 30`	X
`t4 = t3+5`	`t4 = 35`	X
`t5 = t4*8`	`t5 = 280`	X

Código original	Depois da propagação/dobragem	Morta?
`t6 = fp+t5`	`t6 = fp+280`	
`t7 = *(t6+aoffset)`	`t7 = *(t6+aoffset)`	
`t8 = t1+t7`	`t8 = t1+t7`	
`t9 = 3`	`t9 = 3`	X
`t10 = t9*10`	`t10 = 30`	X
`t11 = t10+5`	`t11 = 35`	X
`t12 = t11*8`	`t12 = 280`	X
`t13 = fp+t12`	`t13 = fp+280`	
`*(t13+aoffset) = t8`	`*(t13+aoffset) = t8`	

Depois da dobragem, `t2..t5` e `t9..t12` não são mais usados (seus consumidores já recebem constantes), então todas essas atribuições são mortas. Sobram só o endereço base de `x` (`t1`), o endereço base de `A` (`t6`, `t13`) e o store final.

Original Code	After constant propagation & folding	"X" if deleted as dead code
t1 = *(fp+xoffset) // x	t1 = *(fp+xoffset)	
t2 = 3	t2 = 3	x
t3 = t2 * 10	t3 = 30	x
t4 = t3 + 5	t4 = 35	x
t5 = t4 * 8	t5 = 280	x
t6 = fp + t5	t6 = fp + 280	
t7 = *(t6+aoffset) // A[3][5]	t7 = *(t6+aoffset)	
t8 = t1 + t7 // A[3][5]+x	t8 = t1 + t7	
t9 = 3	t9 = 3	x
t10 = t9 * 10	t10 = 30	x
t11 = t10 + 5	t11 = 35	x
t12 = t11 * 8	t12 = 280	x
t13 = fp + t12	t13 = fp + 280	
*(t13+aoffset) = t8	*(t13+aoffset) = t8	

2. Provas complementares de final

wi08, au09 e au11 não trazem rótulo de meio/final na fonte; entraram aqui por cobrirem o curso inteiro — a prova começa em expressão regular e termina em x86, alocação de registradores, fluxo de dados e agendamento.

O3 — CSEP501 inverno 2008 · prova final

Questão 1 (12 pontos)

Escreva uma expressão regular (ou conjunto de expressões regulares) que gere o seguinte conjunto de strings. Você pode usar abreviaturas (`name = expressão regular`) se isso ajudar a deixar a resposta mais clara.

(A) (6 pontos) Todas as strings de `0`'s e `1`'s em que ou não há nenhum `1`, ou o número de `1`'s é ímpar (isto é, um `1`, três `1`'s, cinco `1`'s, etc.).

(B) (6 pontos) Todos os identificadores formados pelas regras:

- apenas letras minúsculas `a-z`, dígitos `0-9` e o underline _;
- deve começar com uma letra;
- não pode terminar com _;
- não pode haver dois lado a lado.

```
Exemplos: legais: hi, hi_there, only1idea, a_very_long_name. Ilegais: double__underscore,
1digit_at_the_beginning, ends_with_underscore_.
```

Resolução

(A) Existem muitas soluções possíveis. Aqui está uma:

```
0* (10* (10*10*)*)?
```

Leitura: `0*` cobre o caso sem nenhum `1`; a parte opcional tem um `1` seguido de blocos `10*10*` — ou seja, sempre um número ímpar de `1`'s, com `0`'s livres entre eles.

```
(B) Seja letter = a | b | ... | z e digit = 0 | 1 | ... | 9:

letter ( letter | digit | _ (letter | digit) )*
```

Cada repetição é uma letra, um dígito, ou um _ seguido de letra/dígitos — assim nunca há __ nem _ no fim, e o primeiro símbolo é sempre letra. Outra solução (talvez menos enigmática):

```
letter ( (letter | digit)* | _ (letter | digit)+ )*
```

Questão 2 (10 pontos)

A gramática simples

```
expr ::= expr + expr | expr * expr | id | ( expr )
```

é conhecida por ser ambígua. Um estagiário propôs substituí-la pela gramática abaixo para resolver o problema:

```
expr ::= term + term | term
term ::= factor * factor | factor
factor ::= id | ( expr )
```

(A) (5 pontos) A nova gramática gera o mesmo conjunto de strings da antiga? Se gerar, explique por quê (não precisa ser prova formal, mas deve ser uma explicação convincente); se não gerar, dê um exemplo de expressão gerada por uma gramática e não pela outra.

(B) (5 pontos) A nova gramática é ambígua? (Ignore, nesta parte, se ela gera ou não as mesmas cadeias da original.) Se for ambígua, dê um exemplo; se não for, explique por quê.

Resolução

(A) **Não.** A gramática original gera sequências arbitrárias de adições e multiplicações, como `id+id+id`. A nova não pode gerar essa expressão: `expr` só admite `term + term` ou `term`, ou seja, no máximo dois operandos por nível, a menos que um deles esteja entre parênteses.

(B) **Não é ambígua.** Os não-terminais adicionais eliminam a ambiguidade de precedência entre operadores misturados. As regras restritas de `expr` e `term` não permitem adições ou multiplicações com mais de dois operadores, a menos que um deles seja uma expressão entre parênteses — para cada string existe uma única derivação.

Questão 3 (15 pontos)

A questão obrigatória de LR parsing. Considere a gramática:

```
0. expr' ::= expr $
1. expr ::= ( type ) expr
2. expr ::= id
3. type ::= id
```

(A) (12 pontos) Desenhe a máquina de estados LR(0) dessa gramática. Você não precisa escrever as tabelas do parser nem calcular FIRST/FOLLOW/Nullable.

(B) (3 pontos) Essa gramática é LR(0)? Por que ou por que não?

Resolução

(A) Máquina LR(0):

```
estado 0 (inicial): { expr' -> . expr $
 expr -> . ( type ) expr
 expr -> . id }
 ( -> 1 id -> 2 expr -> 5

estado 1: { expr -> ( . type ) expr
 type -> . id }
 id -> 3

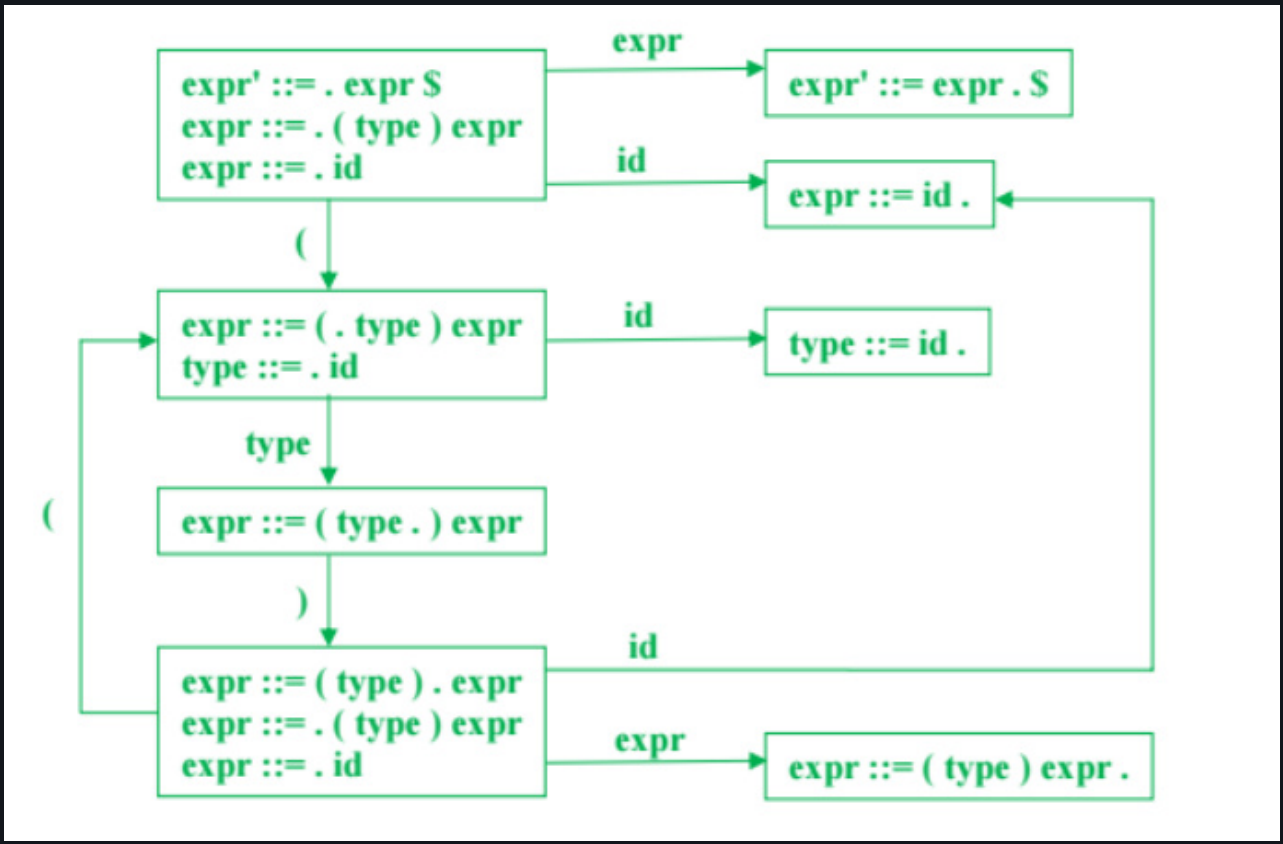
estado 2: { expr -> id . } (reduz expr -> id)

estado 3: { type -> id . } (reduz type -> id)

estado 4: { expr -> ( type ) . expr
 expr -> . ( type ) expr
 expr -> . id }
 ( -> 1 id -> 2

estado 5: { expr' -> expr . $ }
 $ -> aceita
```

Diagrama oficial:




```
Observe que type -> . id só aparece dentro dos parênteses (estado 1), por isso o estado 2
(saída de
id no topo) só contém expr -> id . — não há conflito.
```

(B) **Sim, é LR(0).** Não há conflitos deslocar/reduzir nem reduzir/reduzir em nenhum estado: os estados 2 e 3 contêm um único item completado, e os demais só deslocam ou fazem goto.

Questão 4 (15 pontos)

A outra questão (principalmente) obrigatória. Considere a função C:

```
cint thing(int p, int q) {
 int temp;
 temp = p + q;
 return fun(temp) + temp;
}
```

Assuma que `fun` é outra função declarada em outra parte do programa com o protótipo `int fun(int x);` (pode chamá-la sem declaração adicional, em C ou em assembly).

(A) (4 pontos) Desenhe o frame de pilha da função `thing` como ele apareceria em um programa x86 com as convenções de chamada do C padrão. A figura deve mostrar o layout dos parâmetros, das variáveis locais e os registradores `esp`/`ebp` como ficam após o prólogo executar (frame alocado), mas antes de qualquer instrução do corpo. Mostre os deslocamentos numéricos em relação ao registrador `ebp` para cada parâmetro e variável local.

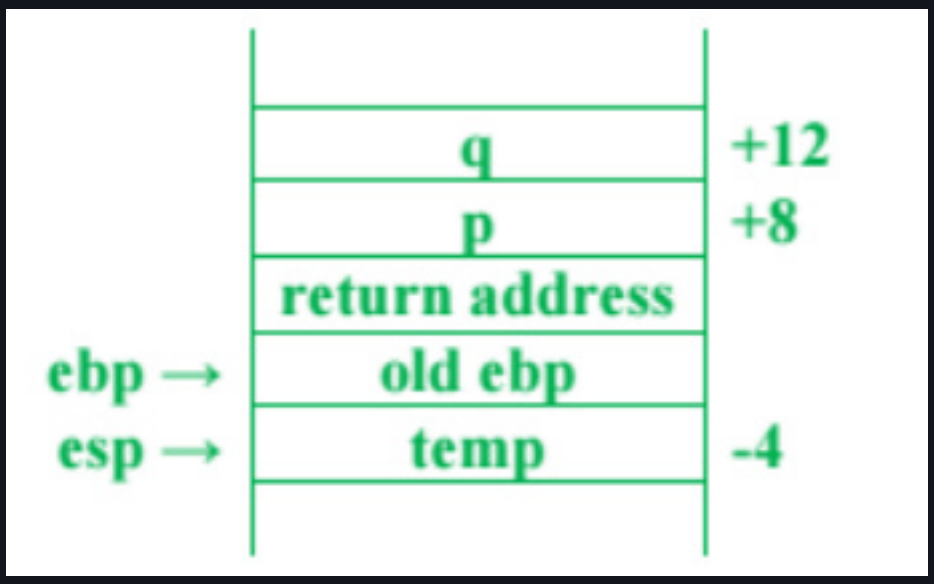
(B) (11 pontos) Traduza a função `thing` para assembly x86. A tradução não pode omitir nenhuma declaração — isto é, deve armazenar o resultado de `p + q` em `temp` — mas caso contrário pode ser qualquer código x86 razoável que siga o layout de pilha e as convenções de chamada da linguagem C. Você pode usar a sintaxe Intel/Microsoft **ou** GNU (só não misture as duas).

Resolução

(A) Frame (cdecl: `p` em `8(%ebp)`, `q` em `12(%ebp)`, `temp` em `-4(%ebp)`):

```
+----------------------+
| q 12 |
| p 8 |
| endereço retorno 4 |
| ebp antigo 0 | <- %ebp
| temp -4 | <- %esp
+----------------------+
```

Diagrama oficial:



(B) Versão x86 (sintaxe Intel):

```
thing:
 push ebp ; prólogo
 mov ebp, esp
 sub esp, 4 ; espaço para temp
 mov eax, [ebp+8] ; temp = p + q
 add eax, [ebp+12]
 mov [ebp-4], eax
 push eax ; argumento de fun(temp)
 call fun
 add esp, 4 ; limpa argumento (chamador)
 add eax, [ebp-4] ; fun(temp) + temp -> eax
 mov esp, ebp ; epílogo
 pop ebp
 ret
```

Questão 5 (15 pontos)

Suponha a máquina hipotética vista em aula, com as seguintes instruções e tempos de latência (tabela abaixo). Considere a sequência de instruções:

```
(a) LOAD r1 <- x
(b) LOAD r2 <- y
(c) MULT r3 <- r1 * r2
(d) LOAD r4 <- z
(e) ADD r5 <- r3 + r4
(f) STORE ans <- r5
```

(A) (6 pontos) Desenhe um grafo de precedência mostrando as dependências entre estas instruções.

Rotule cada nó (instrução) com a letra (a-f) e sua latência — o número de ciclos entre o início daquela instrução e o fim do grafo.

(B) (6 pontos) Reescreva as instruções na ordem em que a lista de scheduling as escolheria (ou seja, cada passo escolhe uma instrução que não depende de nenhuma ainda não emitida). Em caso de empate, escolha qualquer uma. Não é preciso mostrar as anotações nem traçar o algoritmo (mas alguma pista ajuda na correção).

(C) (3 pontos) Quantos ciclos foram necessários no modelo original? Quantos no novo calendário criado no item (B)?

Operation	Cycles
LOAD	3
STORE	3
ADD	1
MULT	2

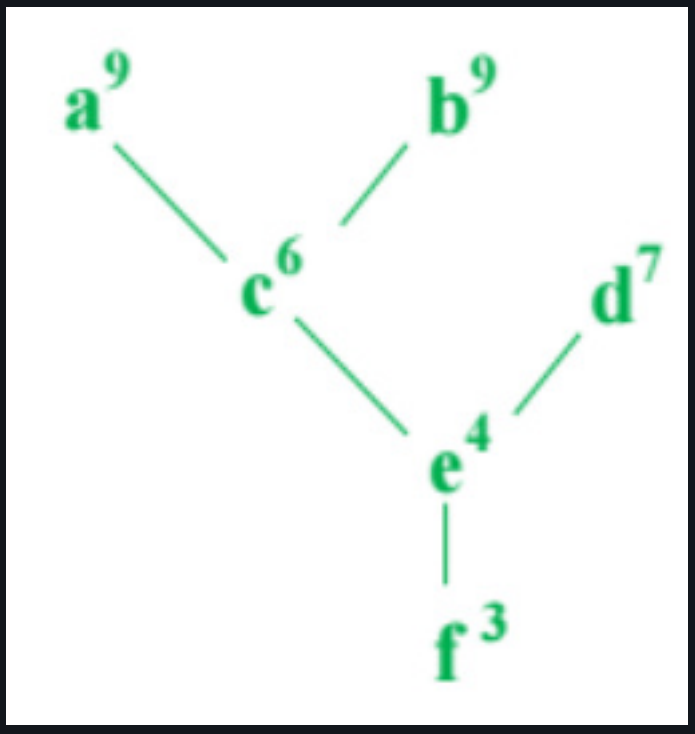
```
(a) LOAD    r1 ← x
(b) LOAD    r2 ← y
(c) MULT    r3 ← r1 * r2
(d) LOAD    r4 ← z
(e) ADD     r5 ← r3 + r4
(f) STORE   ans ← r5
```

Resolução

(A) Grafo de precedência (arestas: `a->c`, `b->c`, `c->e`, `d->e`, `e->f`; latências, da instrução até o fim do grafo):

```
a□ b□
 \ /
 c□ d□
 \ /
 \ /
 e□
 |
 f³
```

Latências: f = 3 (seu próprio tempo), e = 1 + 3 = 4, c = 2 + 4 = 6, d = 3 + 4 = 7, a = b = 3 + 6
= 9. Figura oficial:



(B) Ordem do scheduling:

```
(a) LOAD r1 <- x // os dois primeiros Loads podem estar em qualquer ordem
(b) LOAD r2 <- y
(d) LOAD r4 <- z
(c) MULT r3 <- r1 * r2
(e) ADD r5 <- r3 + r4
(f) STORE ans <- r5
```

(C) Original: **12 ciclos**. Novo calendário: **10 ciclos**.

Questão 6 (8 pontos)

Em vários problemas de análise de fluxo de dados iterativa trabalhamos com os conjuntos GEN, KILL, IN e OUT (variáveis vivas, definições que atingem, expressões disponíveis etc.). Agora queremos usar análise de fluxo de dados iterativa para relatar todas as variáveis que **podem estar não inicializadas** em algum ponto onde são usadas no programa. Qual dos problemas de análise padrão usaríamos para determinar essa informação? Como usar o resultado para decidir quais variáveis podem ser consideradas inicializadas? (Se nenhum dos problemas padrão servir, descreva um problema de análise de fluxo que funcione e como ele seria usado.)

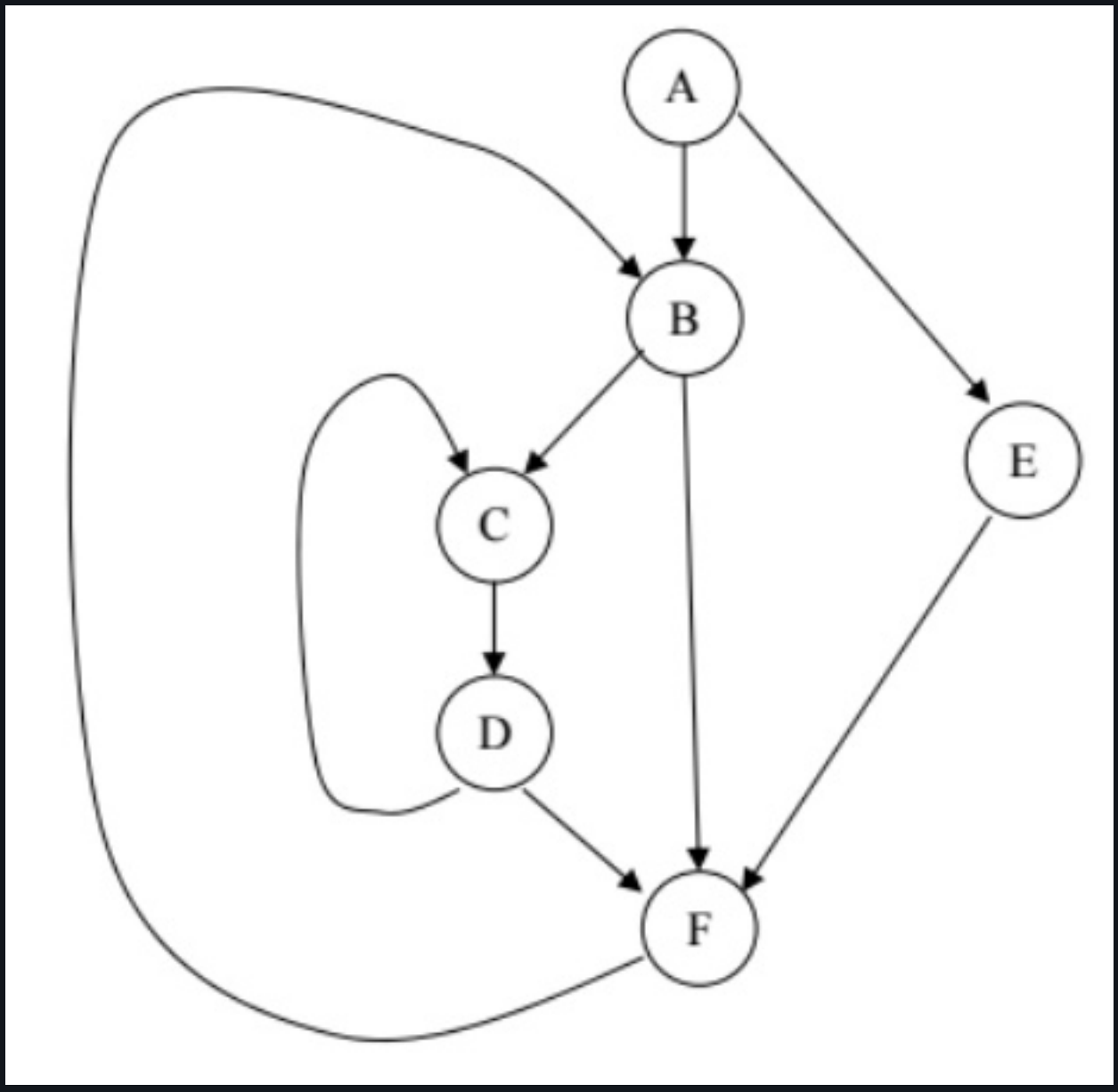
Resolução

Análise de variáveis vivas (liveness). Uma variável é viva em um ponto do programa se existe um caminho, a partir desse ponto até a saída, ao longo do qual seu valor pode ser usado antes de ser redefinido.

Aplicando a análise: as variáveis que estão vivas **no início** do programa (ou do procedimento) são exatamente aquelas que podem ser usadas sem ter sido inicializadas — existe um caminho até um uso sem nenhuma definição anterior. Já as variáveis que não estão vivas no início são aquelas em que todo caminho até qualquer uso passa por uma definição: são as que podem ser consideradas inicializadas.

Questão 7 (10 pontos)

Considere o gráfico de controle de fluxo abaixo (figura do enunciado). Para cada nó do fluxograma, liste os nós que o dominam e o nó correspondente ao seu dominador imediato.



Resolução

Grafo de controle (figura do enunciado). Resultado:

Nó	Dominadores	Dominador imediato
A	A	—
B	A, B	A
C	A, B, C	B
D	A, B, C, D	C
E	A, E	A
F	A, F	A

Figura oficial da resolução:

Node	Dominators	Immediate Dominator
A	A	–
B	A, B	A
C	A, B, C	B
D	A, B, C, D	C
E	A, E	A
F	A, F	A

Conferência: `pred(F) = {B, D, E}`; como todo caminho até F passa por A (e nenhum outro nó), `dom(F) = {A, F}` e `idom(F) = A`. O laço `C -> D -> C` faz `D` ser dominado por `C`.

Questão 8 (15 pontos)

Em aulas sobre fluxo de dados vimos o problema de liveness, que determina quais variáveis estão vivas na saída de cada bloco. Outro problema fundamental é o de **alcançar definições (reaching definitions)**:

para cada definição que aparece em um bloco, determinar em quais outros blocos o valor atribuído nela pode ser observado.

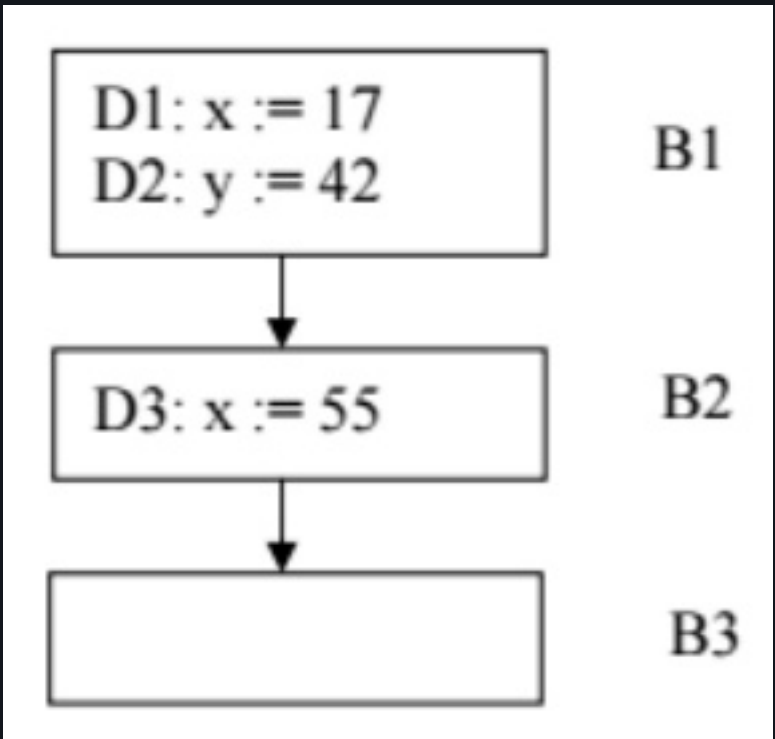
No exemplo do diagrama abaixo: as definições `D2` e `D3` alcançam `B3`, porque não há atribuições subsequentes a `x` ou `y` entre elas e `B3`; mas `D1` não alcança `B3`, porque a atribuição em `B2` mata `D1` (`D1` chega a `B2`, mas é morta lá).

Definição: a definição `d` do bloco `p` alcança o bloco `q` se existe pelo menos um caminho de `p` a `q` no qual `d` não é morta.

O problema é montado como fluxo de dados: para cada bloco, `GEN` = definições geradas no bloco (supondo que não sejam mortas depois no mesmo bloco) e `KILL` = todas as outras definições `d'` de variáveis definidas em `b`. No exemplo: `D3` pertence ao `GEN` de `B2`, e `D1` pertence ao `KILL` de `B2` (ambas definem `x`). Restringindo-nos a blocos com uma única instrução:

```
GEN[b] = { a definição d do bloco b, se houver }
KILL[b] = { todas as outras definições que atribuem uma variável definida em b }
IN[b] = ∪p∈pred[b] OUT[p]
OUT[b] = GEN[b] ∪ (IN[b] − KILL[b])
```

Calcule as definições que atingem os nós do fluxograma abaixo: primeiro preencha `GEN` e `KILL` na tabela, depois resolva `IN` e `OUT` iterativamente (para frente ou para trás, como preferir).



Resolução

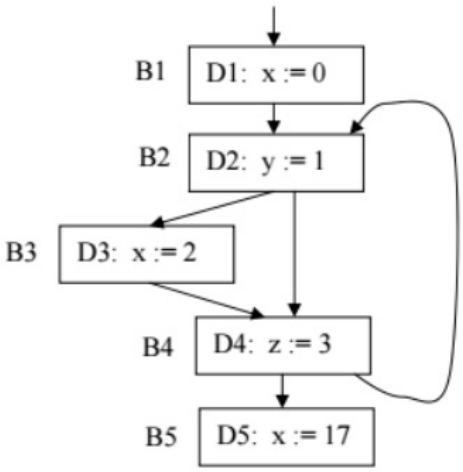
Exemplo do enunciado (figura no enunciado): em `B1 -> B2 -> B3`, as definições `D2` e `D3` atingem `B3`, mas `D1` não, porque `D3` (em `B2`) mata `D1` — a definição `D1` chega a `B2` mas é morta lá.

```
Para o fluxograma do exercício (B1: D1: x:=0, B2: D2: y:=1, B3: D3: x:=2, B4: D4: z:=3, B5: D5:
x:=17; arestas: entry->B1, B1->B2, B2->B3, B2->B4, B3->B4, B4->B5, B4->B2), a solução é:
```

Bloco	GEN	KILL	IN 1	OUT 1	IN 2	OUT 2	IN 3	OUT 3
B1	D1	D3, D5	∅	D1	∅	D1	igual	igual
B2	D2	∅	D1	D1, D2	D1-D4	D1-D4	igual	igual
B3	D3	D1, D5	D1, D2	D2, D3	D1-D4	D2, D3, D4	igual	igual
B4	D4	∅	D1, D2, D3	D1-D4	D1-D4	D1-D4	igual	igual
B5	D5	D1, D3	D1-D4	D2, D4, D5	D1-D4	D2, D4, D5	igual	igual

(`D1-D4` = {D1, D2, D3, D4}; as colunas IN/OUT 1 e 2 são as duas primeiras varreduras; na terceira nada muda — ponto fixo foi atingido. `B1` não tem predecessor, por isso seu IN é vazio.)

Figura oficial da resolução (fluxograma + tabela):



Block	GEN	KILL	IN 1	OUT 1	IN 2	OUT 2	IN 3	OUT 3
B1	D1	D3,D5	--	D1	--	D1	same	same
B2	D2	--	D1	D1,D2	D1,2,3,4	D1,2,3,4	same	same
B3	D3	D1,D5	D1,D2	D2,D3	D1,2,3,4	D2,3,4	same	same
B4	D4	--	D1,2,3	D1,2,3,4	D1,2,3,4	D1,2,3,4	same	same
B5	D5	D1,D3	D1,2,3,4	D2,4,5	D1,2,3,4	D2,4,5	same	same

O4 — CSEP501 outono 2009 · prova final

Questão 1 (12 pontos)

Para cada uma das tarefas abaixo, identifique o estágio do compilador que executa a tarefa ou detecta a situação. Considere um compilador convencional que gera código natico (x86, MIPS etc.) para uma linguagem como C++ ou Java. Se mais de um estágio puder sempre executar a verificação dentro de seu processamento normal, escolha o primeiro deles. Abreviações: `scan` = scanner; `parse` = parser; `sem` = semântica estática; `opt` = otimização; `instr` = seleção de instruções/scheduling; `reg` = alocação de registradores; `run` = tempo de execução (o programa compilado está rodando).

(a) O operador `+` é associativo à esquerda.

(b) Não existe o operador `<=>` na linguagem.

(c) Mover o cálculo `x + y` para fora de um laço, uma vez que nem `x` nem `y` mudam dentro do laço.

(d) Otimizar uma sequência de adições para usar a instrução `lea` do x86 (load effective address) em vez de várias `add`.

(e) Garantir que o resultado da função seja devolvido no registrador correto (`%eax` no x86).

(f) `int` seguido de um identificador que não foi declarado anteriormente neste escopo.

(g) Chaves `{ }` devidamente equilibradas.

(h) Em um acesso de array `a[i]`, o subscrito `i` está dentro dos limites do array.

(i) Inserir instruções de load/store para mover valores da e para a memória quando não há registradores suficientes para mantê-los.

(j) Determinar se um método desta classe sobrescreve (override) um método de alguma superclasse.

(k) Instrução `break` não marcada aparecendo dentro de uma instrução de laço ou `switch`.

(l) Na declaração de atribuição `p = (t) q`, o objeto referenciado por `q` tem tipo `t`.

(obs.: a lista de tarefas foi reconstruída a partir da resolução — o documento original não preservou a lista do enunciado.)

Resolução

(a) **parse** — precedência/associatividade é propriedade da gramática.

(b) **parse** — o símbolo não pertence ao alfabeto de tokens da linguagem.

(c) **opt** — lifting de código invariante de laço.

(d) **instr** — seleção de instruções aproveitando o modo de endereçamento `lea`.

(e) **reg** — convenção de chamada: o valor de retorno vai em `%eax`.

(f) **sem** — declaração usada antes de ser vista (escopo).

(g) **parse** — delimitadores equilibrados são checagem sintática.

(h) **run** — em Java (e C com checagem) os limites de array são verificados em tempo de execução.

(i) **reg** — spilling: inserir accesses à memória quando faltam registradores.

(j) **sem** — checagem de tipos/herança (override válido).

(k) **sem** — `break` fora de laço/`switch` é erro de contexto; crédito extra também para `parse` (um parser com contexto suficiente também poderia detectar).

(l) **run** — a checagem do cast acontece em tempo de execução.

Questão 2 (14 pontos)

Em XML (e HTML), um comentário tem a forma `<!-- Qualquer coisa que não contenha dois traços adjacentes -->`. Ou seja, o comentário começa com a sequência de quatro caracteres `<!--` e termina com a sequência de três caracteres `-->`. Exceto no início e no final, a sequência `--` não pode aparecer. Assim, nenhuma das linhas abaixo é um comentário válido:

```
<!--Não um comentário válido --não em xml-->
<!--Não serve, dois muitas -'s no final---->
```

(A) Dê uma expressão regular (ou um conjunto) que gere comentários XML legais como descritos acima. Para simplificar, assuma que o conjunto de caracteres consiste apenas em letras `a-z`, os caracteres `<`, `>`, `!`, `-` e espaços em branco. Você não precisa se preocupar com outros caracteres nem com comentários que se estendem por várias linhas. Use ␣ para indicar espaço em branco se precisar dele.

(B) Desenhe um DFA que aceite comentários XML como descacritos acima e como gerados pela(s) expressão(ões) regular(es) do item (A). Basta dar um DFA que corresponda à sua expressão regular — não é preciso usar algoritmos formais de conversão.

Resolução

(A)

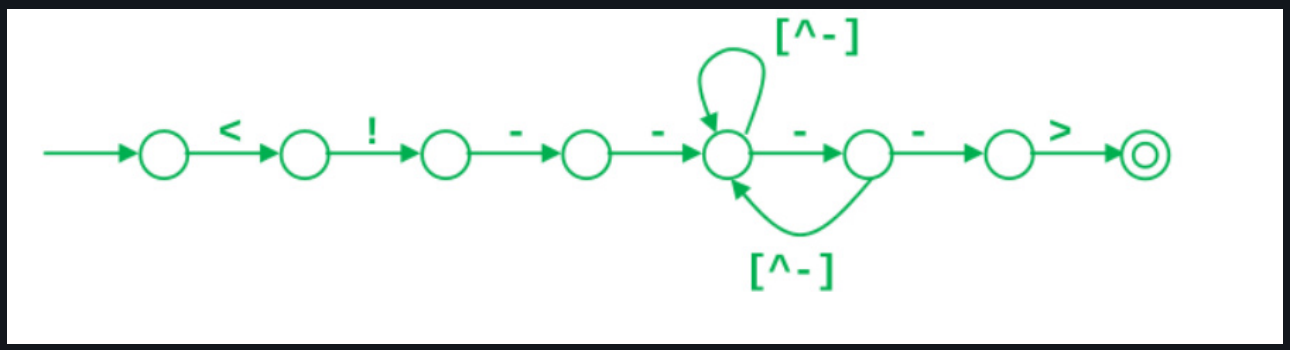
```
<!-- ( -? [^-] )* -->
```

Cada repetição consome um caractere não-traco, ou um traco único seguido de um caractere não-traco — assim nunca aparecem dois tracos adjacentes, e o corpo nunca termina com traco (o que criaria `--` na fronteira com o `-->` final).

(B) DFA sobre o alfabeto `{a-z, espaço, <, >, !, -}`:

```
q0 (inicial): '<' -> q1 (qualquer outra coisa -> erro)
q1: '!' -> q2 (senão erro)
q2: '-' -> q3 (senão erro)
q3 (corpo, último caractere != '-'):
 '-' -> q4
 {a-z, espaço, <, >, !} -> q3
q4 (corpo terminou com '-'):
 '-' -> q5 (pode ser o início do fechamento)
 {a-z, espaço, <, >, !} -> q3 (o '-' era do corpo)
q5 (vimos '--' final):
 '>' -> q6 (fecha o comentário)
 senão -> erro
q6: FIM (estado final)
```

Figura oficial do DFA:



Questão 3 (15 pontos)

A (quase) obrigatória questão de parsing LR. Na linguagem C, o nome de uma variável numa declaração pode ser precedido por um ou mais `*` para indicar que a variável contém um apontador para um valor, em vez do valor em si. Gramática desse fragmento (`ID` é um terminal que denota um identificador):

```
0. decl' ::= decl $
1. decl ::= ptr id
2. decl ::= id
3. ptr ::= * ptr
4. ptr ::= *
```

(A) (12 pontos) Desenhe a máquina de estados LR(0) dessa gramática. Você não precisa escrever as tabelas do parser ou FIRST/FOLLOW/Nullable (mas pode, se ajudar na parte (B)).

(B) (3 pontos) Essa gramática é LR(0)? Por quê ou por que não?

Resolução

(A) Máquina LR(0):

```
estado 0 (inicial): { decl' -> . decl $
 decl -> . ptr id
 decl -> . id
 ptr -> . * ptr
 ptr -> . * }
 * -> 1 id -> 2 ptr -> 3 decl -> 6

estado 1 (marcado ⊕): { ptr -> * .
 ptr -> * . ptr
 ptr -> . * ptr
 ptr -> . * }
 ptr -> 4 (e conflito em '*': ver (B))

estado 2: { decl -> id . } (reduz decl -> id)

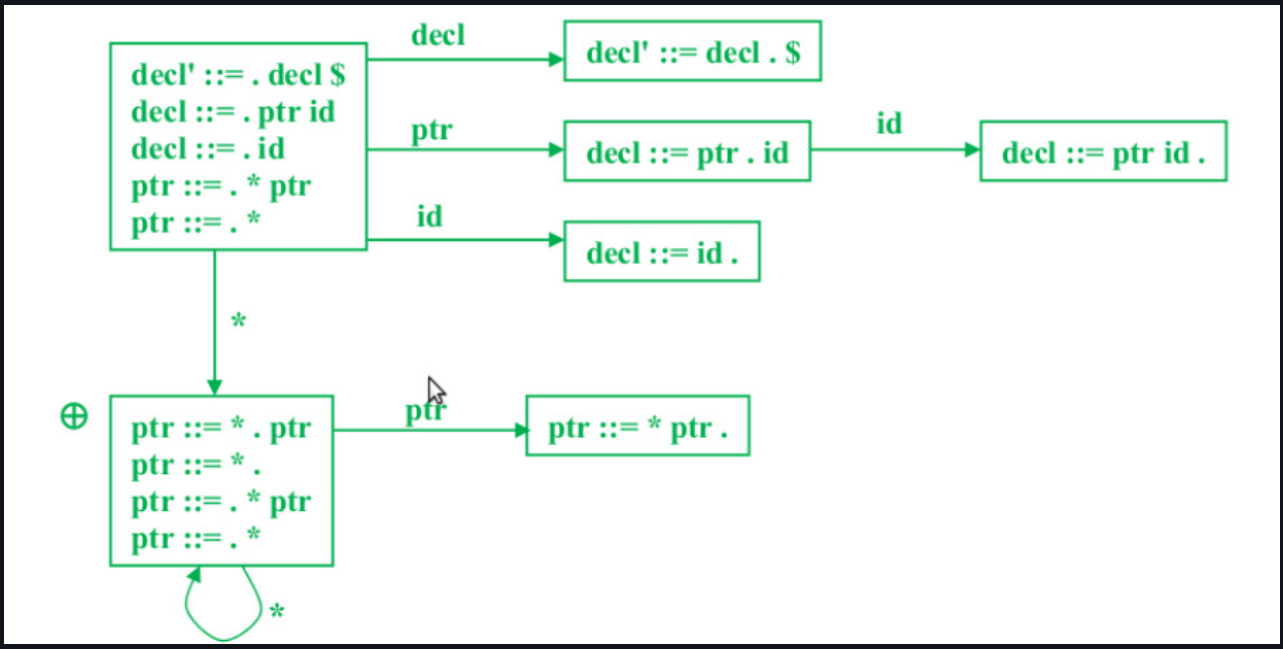
estado 3: { decl -> ptr . id }
 id -> 5

estado 4: { ptr -> * ptr . } (reduz ptr -> * ptr)

estado 5: { decl -> ptr id . } (reduz decl -> ptr id)

estado 6: { decl' -> decl . $ }
 $ -> aceita
```

Diagrama oficial:



(B) **Não.** O estado 1 (o marcado ⊕) tem conflito deslocar/reduzir: com o ponto antes de `*` (em `ptr -> * .` ptr e no closure `ptr -> . *`) precisa deslocar `*`, enquanto o item completado `ptr -> * .` pede redução. LR(0) decide sem lookahead, então há conflito.

Questão 4 (15 pontos)

A outra (principalmente) obrigatória questão. Considere a função C:

```
cint xyzzy(int x, int y) {
 int n;
 n = x + 1;
 if (y < 0)
 return n;
 else
 return xyzzy(y, n);
}
```

(A) (4 pontos) Desenhe o frame de pilha da função `xyzzy` como ele apareceria em um programa x86 usando as convenções de chamada C padrão. A figura deve mostrar o layout de parâmetros, variáveis locais e os registradores `%esp`/`%ebp` como ficam após o prólogo executar e alocar o frame, mas antes de qualquer instrução do corpo. Mostre os deslocamentos numéricos em relação a `%ebp` para cada parâmetro e variável local.

(B) (11 pontos) Traduza a função `xyzzy` para assembly x86. A tradução não pode omitir nenhuma declaração (por exemplo, não pode otimizar fora a atribuição à variável `n`), mas caso contrário pode ser qualquer código x86 razoável que siga o layout de pilha e as convenções de chamada do C. Você pode usar a sintaxe Intel/Microsoft **ou** GNU (só não misture as duas). Definição da função, por conveniência:

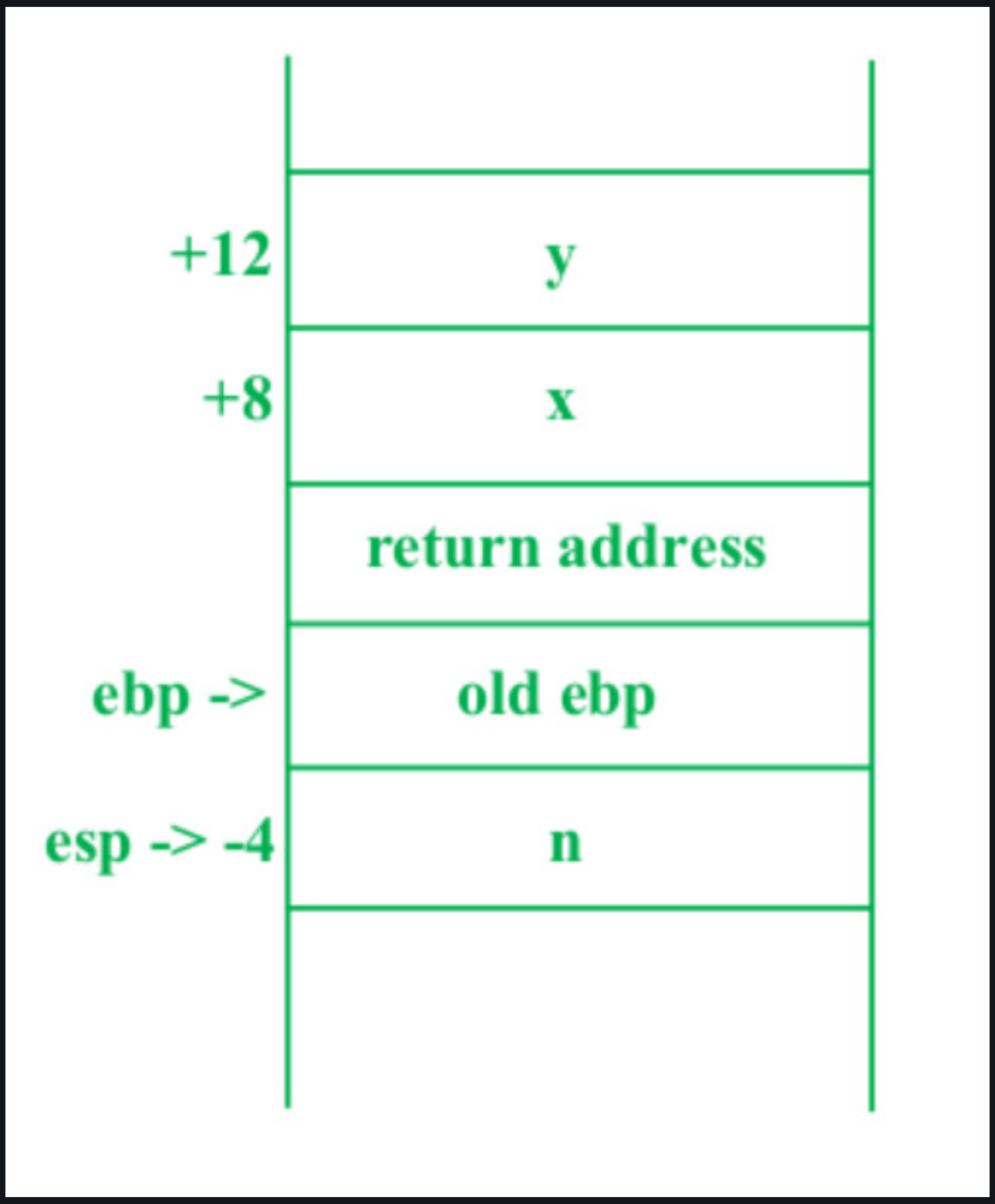
```
cint xyzzy(int x, int y) {
 int n;
 n = x + 1;
 if (y < 0)
 return n;
 else
 return xyzzy(y, n);
}
```

Resolução

(A) Frame (cdecl: `x` em `8(%ebp)`, `y` em `12(%ebp)`, `n` em `-4(%ebp)`):

```
+----------------------+
| y 12 |
| x 8 |
| endereço retorno 4 |
| ebp antigo 0 | <- %ebp
| n -4 | <- %esp
+----------------------+
```

Diagrama oficial:



(B) Versão x86 (sintaxe Intel):

```
xyzzy:
 push ebp ; prólogo
 mov ebp, esp
 sub esp, 4
 mov eax, [ebp+8] ; n = x + 1
 inc eax
 mov [ebp-4], eax
 mov eax, [ebp+12] ; compara y < 0
 cmp eax, 0

jnl else ; salta se falso
 mov eax, [ebp-4] ; return n (em eax)
 jmp exit
else:
 mov eax, [ebp-4] ; chamada xyzzy(y, n)
 push eax ; empurra n
 mov eax, [ebp+12]
 push eax ; empurra y
 call xyzzy
 add esp, 8 ; retira argumentos; resultado em eax
exit:
 mov esp, ebp ; desfaz locais
 pop ebp ; restaura ebp antigo
 ret
```

Obviamente há muitas maneiras de fazer isso; esta é uma versão simples que traduz cada parte da função de forma isolada.

```
xyzzy:  push ebp                ; function prologue
        mov  ebp,esp
        sub  esp,4
        mov  eax,[ebp+8]        ; n = x+1
        inc  eax
        mov  [ebp-4],eax;
        mov  eax,[ebp+12]       ; compare y < 0
        cmp  eax,0
        jnl  else               ; jump false
        mov  eax,[ebp-4]        ; return n in eax
        jmp  exit
else:   mov  eax,[ebp-4]        ; call xyzzy(y,n)
        push eax                ; push n
        mov  eax,[ebp+12]
        push eax                ; push y
        call xyzzy
        add  esp,8              ; pop args, result in eax
exit:   mov  esp,ebp            ; pop locals
        pop  ebp                ; restore old ebp
        ret                     ; return
```

Questão 5 (15 pontos)

Por questões de segurança, várias linguagens têm a noção de dados "contaminados" (tainted): qualquer valor lido do ambiente externo é marcado como contaminado — potencialmente perigoso. O resultado de qualquer operação que use dados contaminados também é contaminado. Há ainda um modo de marcar um valor como "não contaminado" (`clean`), usado após uma verificação de que é seguro.

Operações do código de baixo nível:

- `x = y ⊕ z` (binária): `x` fica contaminado se `y` ou `z` (ou ambos) forem contaminados.
- `x = y` (atribuição): `x` fica contaminado se `y` for contaminado.
- `x = read()` (entrada): o resultado `x` fica contaminado.
- `x = clean(y)`: atribui `y` a `x`, mas marca `x` como não contaminado.

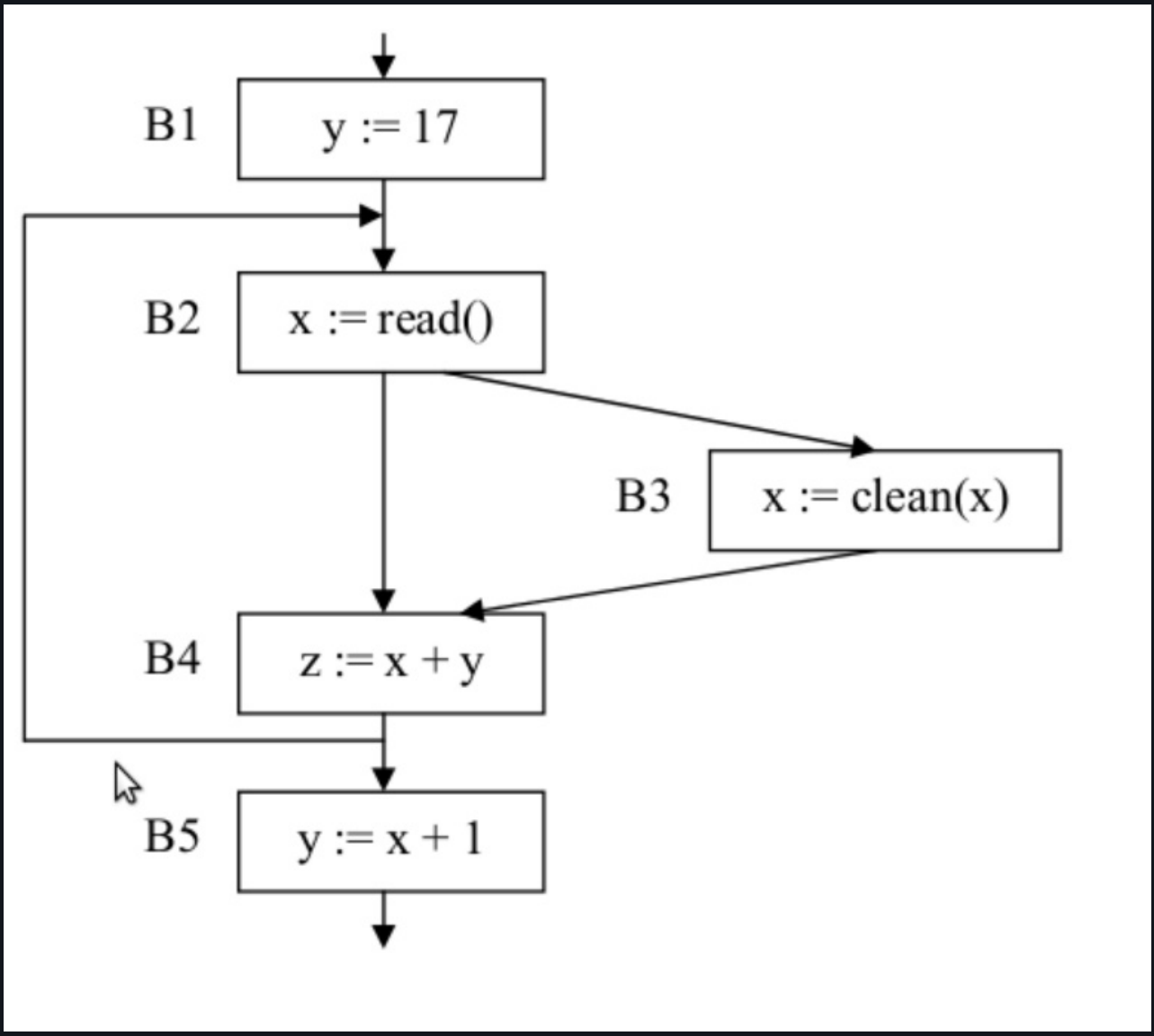
Queremos usar análise de fluxo de dados para descobrir variáveis contaminadas: uma variável que **pode** conter valor contaminado é marcada como contaminada; só marcamos como não-contaminada se o valor for garantidamente seguro. Defina, para cada bloco básico `b`:

```
IN[b] = variáveis possivelmente contaminadas na entrada de b
OUT[b] = variáveis possivelmente contaminadas na saída de b
GEN[b] = variáveis marcadas contaminadas em b e não limpas antes de sair
CLEAN[b]= variáveis limpas em b e não contaminadas depois, antes de sair
```

`GEN[b]` e `CLEAN[b]` podem ser calculados estaticamente a partir do conteúdo de `b`; `IN` e `OUT` são calculados iterativamente.

(A) Dê as equações de fluxo de dados apropriadas para `IN` e `OUT` de um bloco `b`, em termos de `IN`, `OUT`, `GEN` e `CLEAN` — combinando informações locais do bloco com informações sobre predecessores.

(B) Considere o fluxograma abaixo. Complete a tabela usando análise de fluxo de dados iterativa para identificar as variáveis contaminadas em `IN` e `OUT` de cada bloco: primeiro preencha `GEN` e `CLEAN`, depois resolva `IN`/`OUT` iterativamente (para frente ou para trás). Assuma que não há variáveis contaminadas no `IN` do bloco B1.



Resolução

(A) As equações:

```
IN[b] = ∪x∈preds(b) OUT[x]
OUT[b] = GEN[b] ∪ (IN[b] − CLEAN[b])
```

(B) Fluxograma: B1: y := 17 → B2: x := read() → (B3: x := clean(x) | —) → B4: z := x + y → B5: y := x + 1, com arestas entry→B1, B1→B2, B2→B3, B2→B4, B3→B4, B4→B5, B4→B2 (laço) e B5→exit.

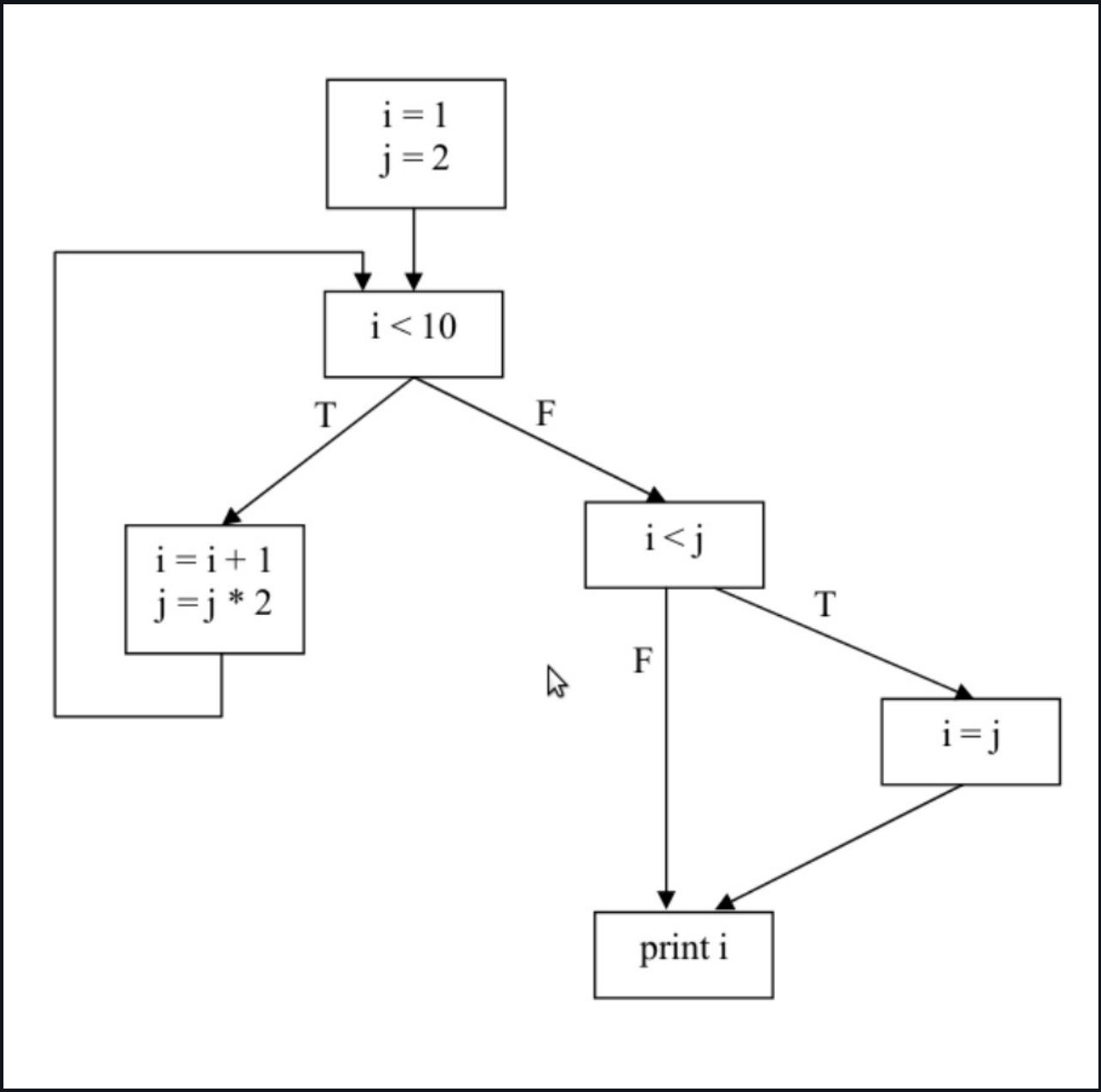
Bloco	GEN	CLEAN	IN 1	OUT 1	IN 2	OUT 2	IN 3	OUT 3
B1	—	—	—	—	—	—	igual	igual
B2	x	—	—	x	x, z	x, z	igual	igual
B3	—	x	x	—	x, z	z	igual	igual
B4	z (se x ou y)	—	x	x, z	x, z	x, z	igual	igual
B5	y (se x)	—	x, z	x, y, z	x, z	x, y, z	igual	igual

Notas: `B1` não tem predecessor, então seu `IN` é vazio; o ponto fixo é atingido após a segunda varredura. O `GEN` de `B4`/`B5` é **condicional** (o `z` só fica contaminado se `x` ou `y` estiverem), o que é o que o enunciado quer dizer com "calculável estaticamente". O gabarito também aceitava deixar esses `GEN` vazios. Tabela oficial:

Block	GEN	CLEAN	IN 1	OUT 1	IN 2	OUT 2	IN 3	OUT 3
B1	--	--	--	--	--	--	same	same
B2	x	--	--	x	x, z	x, z	same	same
B3	--	x	x	--	x, z	z	same	same
B4	z* (if x or y)	--	x	x, z	x, z	x, z	same	same
B5	y* (if x)	--	x, z	x, y, z	x, z	x, y, z	same	same

Questão 6 (14 pontos)

SSA. Considere o fluxograma abaixo. Na página seguinte, redesenhe esse fluxograma em forma SSA (single static assignment), inserindo funções Φ quando necessário para mesclar versões de variáveis nos pontos de junção. Você não precisa calcular dominadores nem usar um algoritmo específico para posicionar as Φ, mas não se esqueça das Φ onde elas são necessárias. Não há crédito extra por Φ desnecessárias, embora isso não seja penalizado se feito corretamente.



Resolução

Com subscritos SSA (cada definição recebe um número de versão):

```
Bloco 1 (entrada): i1 = 1
 j1 = 2
 -> Bloco 2

Bloco 2 (topo do laço): i2 = Φ(i1, i3)
 j2 = Φ(j1, j3)
 i2 < 10
 true -> Bloco 3 false -> Bloco 4

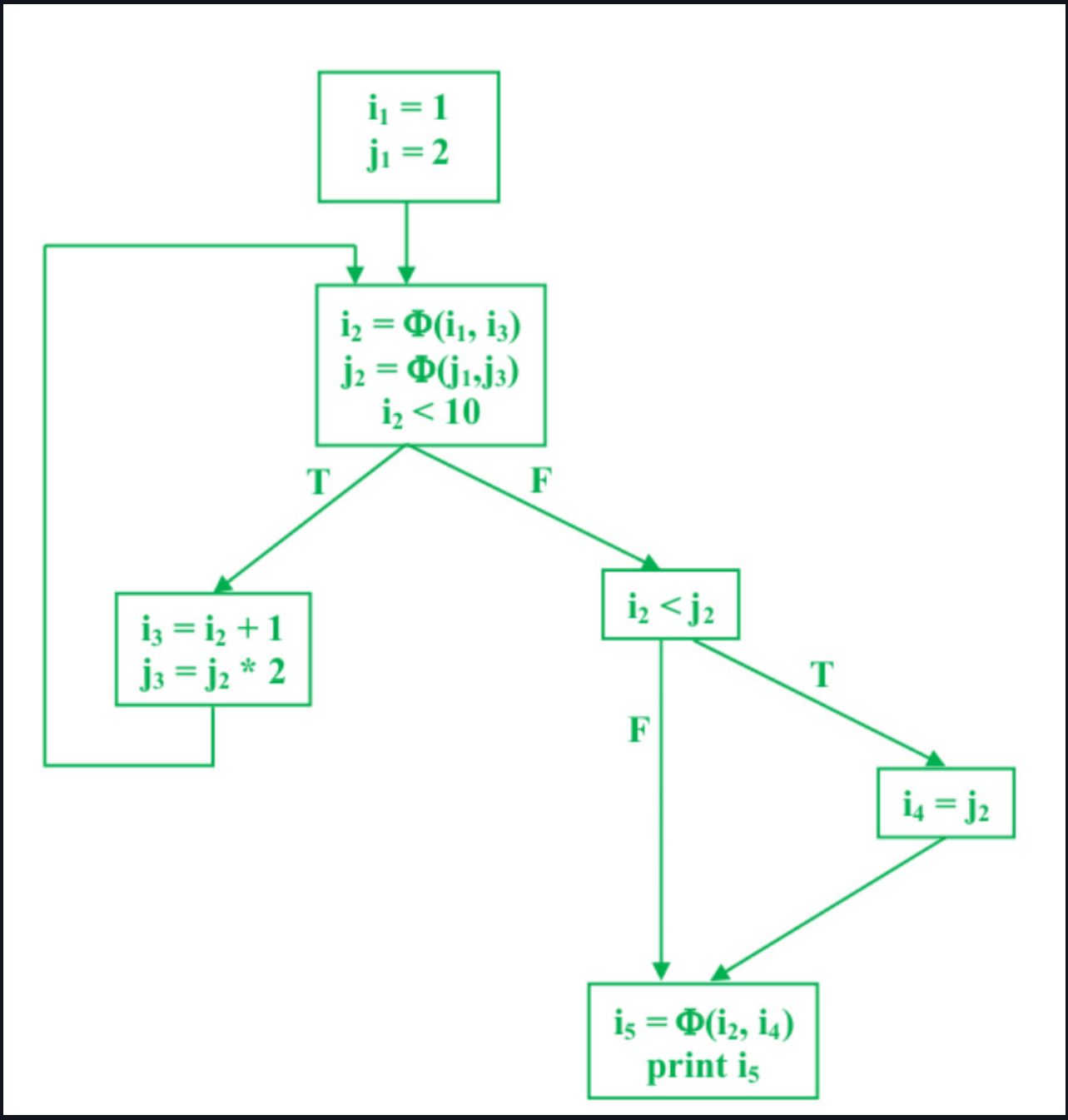
Bloco 3 (corpo): i3 = i2 + 1
 j3 = j2 * 2
 -> Bloco 2 (back edge)


Bloco 4: i2 < j2
 false -> Bloco 6 true -> Bloco 5
```

```
Bloco 5: i4 = j2
 -> Bloco 6

Bloco 6 (saída): i5 = Φ(i2, i4)
 print i5
```

As Φ necessárias: no topo do laço, `i` recebe `i1` (da entrada) ou `i3` (do corpo) e `j` recebe `j1` ou `j3`; na saída, `i` recebe `i2` (quando o laço termina normalmente) ou `i4` (quando `i = j` executa). Diagrama oficial:



Questão 7 (15 pontos)

Suponha a máquina hipotética da aula com as seguintes instruções e tempos de latência:

Operação	Ciclos
LOAD	3
STORE	3

Operação	Ciclos
ADD	1

Operation	Cycles
LOAD	3
STORE	3
ADD	1

Agora considere as instruções que copiam duas palavras consecutivas de memória:

```
(a) LOAD r3 <- *r1
(b) STORE *r2 <- r3
(c) ADD r1 <- r1 + 1
(d) ADD r2 <- r2 + 1
(e) LOAD r4 <- *r1
(f) STORE *r2 <- r4
```

(a) LOAD r3 ← *r1
(b) STORE *r2 ← r3
(c) ADD r1 ← r1 + 1
(d) ADD r2 ← r2 + 1
(e) LOAD r4 ← *r1
(f) STORE *r2 ← r4

A notação `*rn` numa LOAD ou no endereço de uma STORE significa usar o valor em `rn` como endereço da operação de memória. Em um load/store, o registrador de endereço (e, no store, o registrador de

origem) fica livre depois de um ciclo e pode ser alterado por instruções seguintes sem interferir no load/store. Porém, o load/store em si requer 3 ciclos para terminar, e o valor lido por um load só fica disponível depois disso.

(A) (6 pontos) Desenhe um grafo de precedência com as dependências entre essas instruções. Rotule cada nó com a letra (a-f) e sua latência — o número de ciclos entre o início daquela instrução e o fim do grafo.

(B) (6 pontos) Reescreva as instruções na ordem escolhida pela lista de scheduling (a cada passo, escolhe-se uma instrução que não depende de nenhuma ainda não emitida ou ainda em execução). Em empate, escolha arbitrariamente. Rotule cada instrução com sua letra na sequência original e o número do ciclo em que inicia (a primeira instrução começa no ciclo 1).

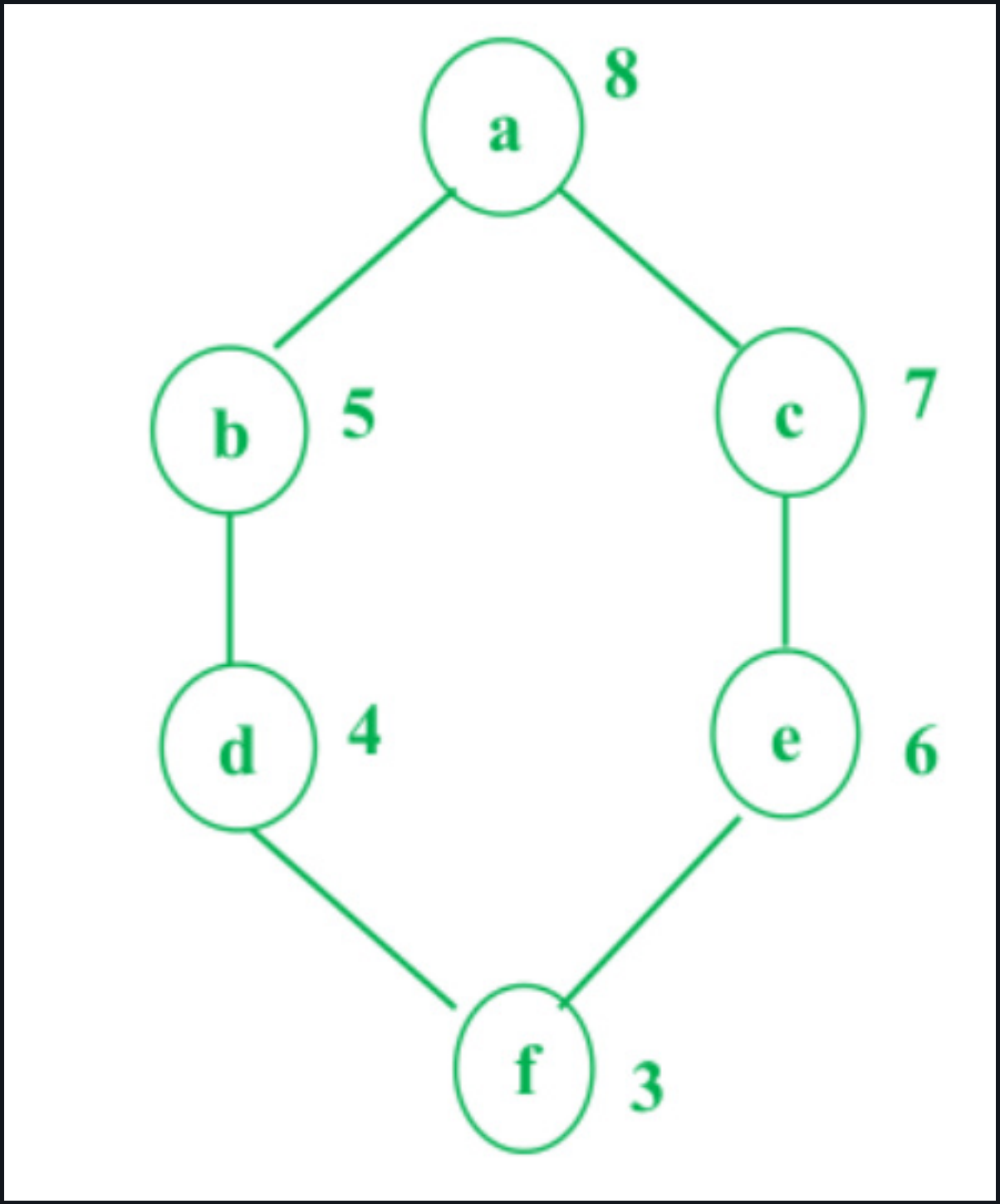
(C) (3 pontos) Quantos ciclos foram necessários no modelo original? E no novo calendário do item (B)?

Resolução

(A) Grafo de precedência — arestas: `a–b` (r3), `a–c` (r1), `b–d` (r2), `c–e` (r1), `d–f` (r2), `e–f` (r4). Latências (ciclos do início da instrução até o fim do grafo):

```
a□
 / \
b□ c□
|  |
d□ e□
 \ /
 f³
```

Figura oficial:



(B) Novo calendário (ciclo de início em parênteses):

```
1 (a) LOAD r3 <- *r1
2 (c) ADD r1 <- r1 + 1
3 (e) LOAD r4 <- *r1
4 (b) STORE *r2 <- r3
5 (d) ADD r2 <- r2 + 1
6 (f) STORE *r2 <- r4
```

(C) Original: **12 ciclos**. Novo calendário: **8 ciclos**.

O5 — CSEP501 outono 2011 · prova final

Questão 1 (10 pontos — 5 cada)

Expressões regulares. Dê expressões regulares que geram os seguintes conjuntos de strings. Você só pode usar as operações básicas de concatenação, escolha (|) e repetição (*) mais os operadores derivados ? e + e classes de caracteres simples como [abc0-9] e [^a-z]. Você pode usar abreviações como vogais = [aeiou]. Você não pode usar os operadores mais complexos encontrados em várias ferramentas de software que lidam com expressões regulares estendidas.

(a) Todas as sequências de 0's e 1's que contêm um número ímpar de 1's e contêm pelo menos um ou mais 1's.

(b) Todas as strings que contêm a's, b's e c's com pelo menos um a e pelo menos um b.

Resolução

(a) Uma solução bastante simples:

```
0*1(0|(10*1))*
```

Leitura: zeros iniciais opcionais, depois um `1` (garante pelo menos um e deixa a contagem ímpar); cada repetição seguinte acrescenta ou nenhum `1` (`0`) ou exatamente dois (`10*1`), mantendo a contagem ímpar.

(b) Com a abreviação `abcs = (a|b|c)*` (ou seja, `[abc]*`):

```
abcs a abcs b abcs | abcs b abcs a abcs

Ou, sem abreviação: (a|b|c)* a (a|b|c)* b (a|b|c)* | (a|b|c)* b (a|b|c)* a (a|b|c)*.
```

Questão 2 (10 pontos)

Gramáticas livres de contexto. Considere a seguinte sintaxe para expressões envolvendo adição e seleção de campo:

```
expr ::= expr + field
expr ::= field
field ::= expr . id
field ::= id
```

(a) Mostre que esta gramática é ambígua. Aqui estão duas derivações de `id+id.id`.

(b) (4 pontos) Dê uma gramática livre de contexto não ambígua que corrija o(s) problema(s) com a gramática do item (a) e gere expressões com id, campo de seleção e adição. Como em Java, seleção de campo deve ter precedência maior do que a adição e tanto seleção de campo quanto adição devem ser associativos à esquerda (ou seja, a + b + c significa (a + b) + c).

Resolução

(A) Duas derivações diferentes para `id + id . id` (mesma cadeia, duas árvores):

Derivação 1 — `id + (id . id)` (seleção pega só o campo da direita):

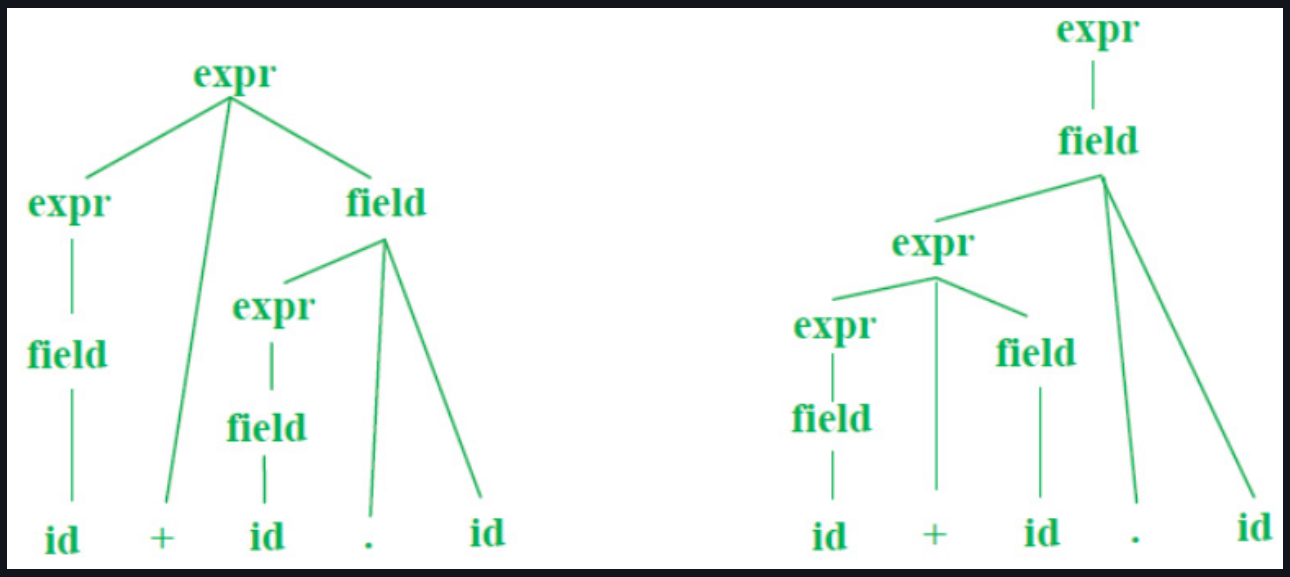
```
expr
+---+-----------+
expr + field
| +---+---+
field expr . id
| |
id field
 |
 id
```

Derivação 2 — `(id + id) . id` (a seleção pega a soma inteira como operando esquerdo):

```
expr
 |
field
```

```
+-------+-------+
expr . id
+------+------+
expr + field
| |
field id
|
id
```

Como a mesma cadeia tem duas árvores sintáticas, a gramática é ambígua.



(B) A mudança é na regra de `field`: a seleção deve encadear campos, não expressões inteiras:

```
expr ::= expr + field
expr ::= field
field ::= field . id
field ::= id
```

Com isso, `+` fica com precedência menor (seu operando direito é um `field`), a seleção encadeia à esquerda (`a.b.c` é `((a.b).c)`) e a adição também é à esquerda (o operando esquerdo de `+` é um `expr` completo, mas o direito não contém `+`), atendendo ao enunciado.

Questão 3 (18 pontos)

Questão (quase) obrigatória de parsing-LR. Considere a seguinte gramática que pretende capturar expressões envolvendo adição (+) e de exponenciação (**), em que a adição é associativa à esquerda e a exponenciação é associativa à direita:

```
0. e' ::= e $
1. e ::= e + t
2. e ::= t
3. t ::= x ** t
4. t ::= x
```

(a) (10 pontos) Desenhe a máquina de estados LR(0) para essa gramática. (Você não precisa incluir a tabela com shift/reduce e ações goto, embora possa escrevê-la mais tarde se achar que é útil para responder a outras partes da questão.)

(b) (2 pontos) Esta gramática é LR(0)? Por que ou por que não?

(c) (4 pontos) Compute First, Follow e Nullable para cada um dos não terminais desta gramática.

(d) Esta gramática é SLR? Por que ou por que não?

Resolução

(a) Nota do gabarito: o operador `**` foi destinado a ser um único token. Uma série de soluções o tratou como dois tokens adjacentes `*`; se feito corretamente, nenhum ponto foi deduzido.

Máquina de estados LR(0):

```
S1 (inicial): { e' ::= . e $
 e ::= . e + t
 e ::= . t
 t ::= . x ** t
 t ::= . x }
 t -> S2, e -> S3, x -> S4

S2: { e ::= t . } (reduz e ::= t)

S3: { e' ::= e . $
 e ::= e . + t }
 + -> S5 ($ = aceita)

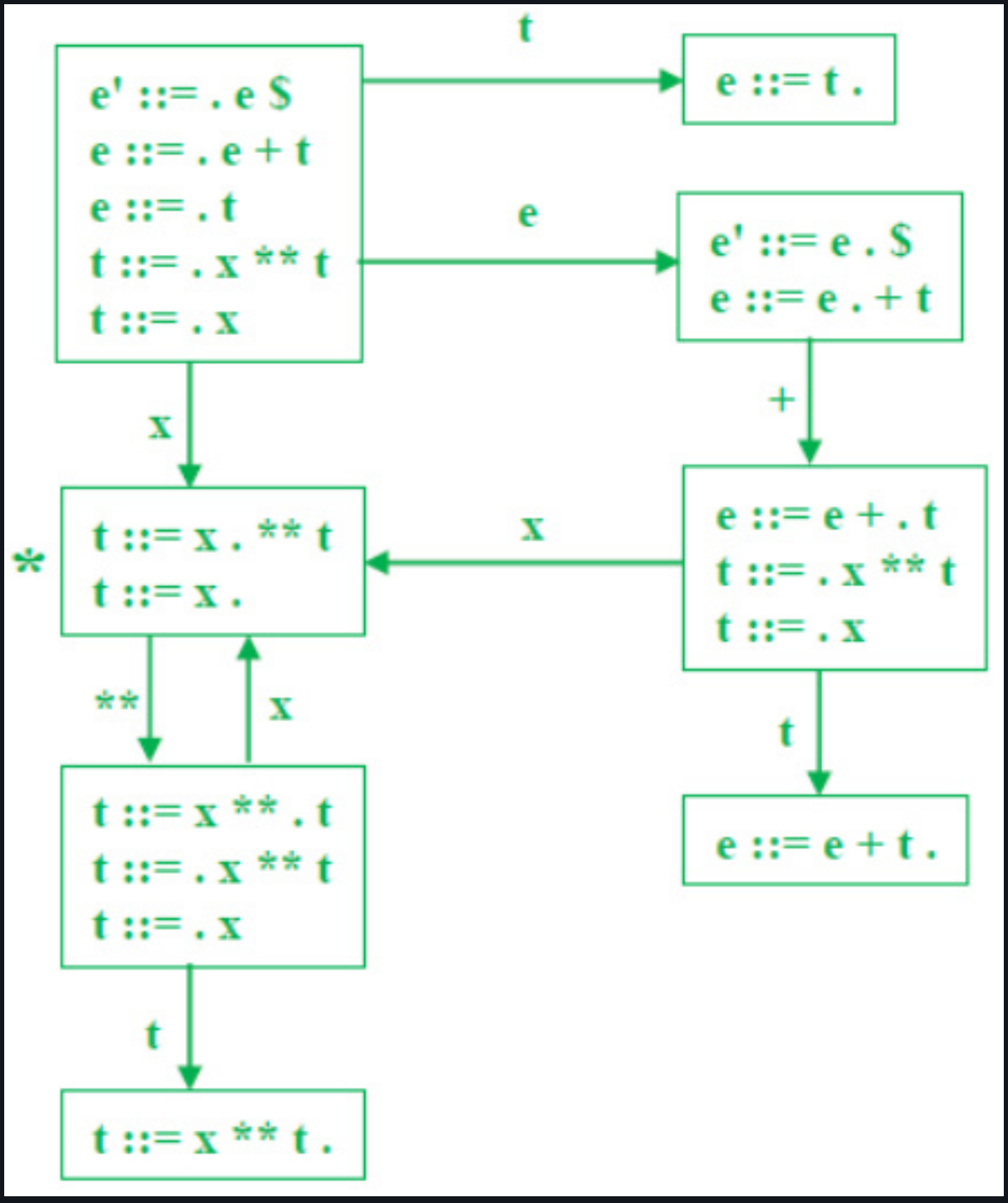
S4 (conflito, marcado *): { t ::= x . ** t
 t ::= x . }
 ** -> S6

S5: { e ::= e + . t
 t ::= . x ** t
 t ::= . x }
 t -> S7, x -> S4

S6: { t ::= x ** . t
 t ::= . x ** t
 t ::= . x }
 t -> S8, x -> S4

S7: { e ::= e + t . } (reduz)

S8: { t ::= x ** t . } (reduz)
```



(b) **Não.** O estado rotulado com `*` (S4) tem um conflito deslocar/reduzir: com a entrada `**` ele tanto

```
poderia deslocar (t ::= x . ** t) quanto reduzir (t ::= x .), e LR(0) decide sem lookahead.

(c)
| | FIRST | FOLLOW | Nullable |
| --- | --- | --- | --- |
| e' | x | — | não |
| e | x | +, $ | não |
| t | x | +, $ | não |
```

	first	follow	nullable
e'	x		no
e	x	+, $	no
t	x	+, $	no

(d) **Sim** (é SLR). O conflito de S4 é resolvível porque `**` não está em `FOLLOW(t) = {+, $}`: na entrada `**` escolhe-se deslocar; nos terminais de FOLLOW(t) escolhe-se reduzir.

Questão 4 (22 pontos)

Gostaríamos de adicionar um novo loop para o MiniJava que é semelhante a um loop while, exceto que ele pode ter várias condições e sequências de declarações em um loop. O caso simples tem uma

```
condição e uma sequência de instruções. Por exemplo, o loop do i<n => sum=sum+a[i]; i++; od
é equivalente ao loop tradicional:

while (i<n) { sum=sum+a[i]; i++; }
```

A diferença é que os parênteses não são necessários em torno da condição, o símbolo terminal => separa a condição do corpo do laço, o fim do laço está indicado pela palavra-chave `od` sem vírgula ou outra pontuação depois dela, e o corpo do laço pode ter várias declarações sem a necessidade de chaves,

porque os símbolos => e `od` indicam a extensão do corpo do loop. Estes laços podem ser aninhados no caminho óbvio, já que o novo laço é apenas outro tipo de declaração. Por exemplo, se tivéssemos vetores 2-D, um array n x n poderia ser inicializado com:

```
i=0;
do i<n => j=0;
 do j<n => a[i][j]=0; j++;
 od
i++;
od
```

Mais interessante, um loop pode conter mais de uma condição mais sequência de declaração agrupada. O loop abaixo altera inteiros positivos x e y de modo a que, em caso de cessação, ambos contêm o maior divisor comum dos seus valores originais:

```
do x>y => x=x-y;
[ ] y>x => y=y-x;
od
```

Se um loop contém um grupo-sequência de mais de uma condição/declaração, eles são separados por uma caixa feita de colchete esquerdo e direito sem espaço entre ([ ]). Para executar o loop, as condições são avaliadas na ordem a partir do início. Se a condição for avaliada como verdadeira, a(s)

declaração(ões) correspondente(s) seguindo o símbolo => até a próxima `[ ]` ou `od` são executadas, e, em seguida, a iteração do loop é concluída. A execução da próxima iteração do loop começa de volta ao topo depois de `do`. Se nenhuma condição for verdadeira, então a execução do loop termina. Em outras palavras: em cada iteração do loop, no máximo uma sequência de instruções é executada — as instruções seguem a primeira condição verdadeira na lista — e as últimas condições e instruções no loop não são executadas nessa iteração. Responda as seguintes perguntas sobre a nova instrução loop.

(a) (6 pontos) Dê uma regra ou um conjunto de regras para gramática livre de contexto sem ambiguidades para adicionar este novo tipo de loop ao Statement da gramática do MiniJava. Sua resposta deve incluir novos terminais adicionais e não-terminais, conforme necessário, e pode incluir regras gramaticais

adicionais para não-terminais se for o caso. Você só precisa dar as adições e mudanças na gramática MiniJava; você não precisa escrever CUP ou outro código-fonte de gerador de parser.

(b) (3 pontos) Que mudanças precisam ser feitas no scanner do MiniJava para adicionar esta nova instrução de loop à linguagem? Novamente, basta descrever as mudanças. Você não precisa escrever JFlex/CUP ou outro código-fonte.

(c) (4 pontos) Que mudanças ou adições precisam ser feitas para as classes de árvore de sintaxe abstrata do MiniJava ou definições de nó para incluir esta nova instrução de laço na gramática abstrata MiniJava? Sua resposta deve dar uma descrição dos tipos de nós que precisam ser adicionados ou alterados e seus conteúdos. Não precisa ser detalhado em Java, C# ou outra linguagem.

(d) (3 pontos) Que verificações precisam ser adicionadas à fase semântica/análise estática de tipos do compilador para esta nova instrução de loop?

(e) (6 pontos) Delinear a forma de código (essencialmente a linguagem pseudo-assembly) necessário para implementar esta nova instrução de loop. Sua resposta deve mostrar a forma do código necessário para um loop como o seguinte, com dois pares de sequência condição/instrução:

```
do cond1 => stmt1 [ ] cond2 => stmt2 od
```

A resposta deve ser semelhante à dos exemplos dados em sala de aula para outras construções de controle, mostrando onde os rótulos e saltos apareceriam e onde o código para as condições e instruções dentro do loop seriam colocados.

Resolução

(a) Uma gramática sem ambiguidades (terminais `'do'`, `'od'`, `'=>'` e `'[]'`):

```
Statement ::= 'do' Guards 'od'
Guards ::= Guard | Guards '[]' Guard
Guard ::= Expression '=>' Statement*
```

`Guards` é a lista de grupos (separados por `[]`) e cada `Guard` é uma condição seguida da sua sequência de declarações — a primeira condição verdadeira da lista define o que executa na iteração.

(b) Novos tokens no scanner: `do`, `od`, `=>` e `[]` (o scanner precisa reconhecer os lexemas; `do` e `od` viram palavras-chave novas).

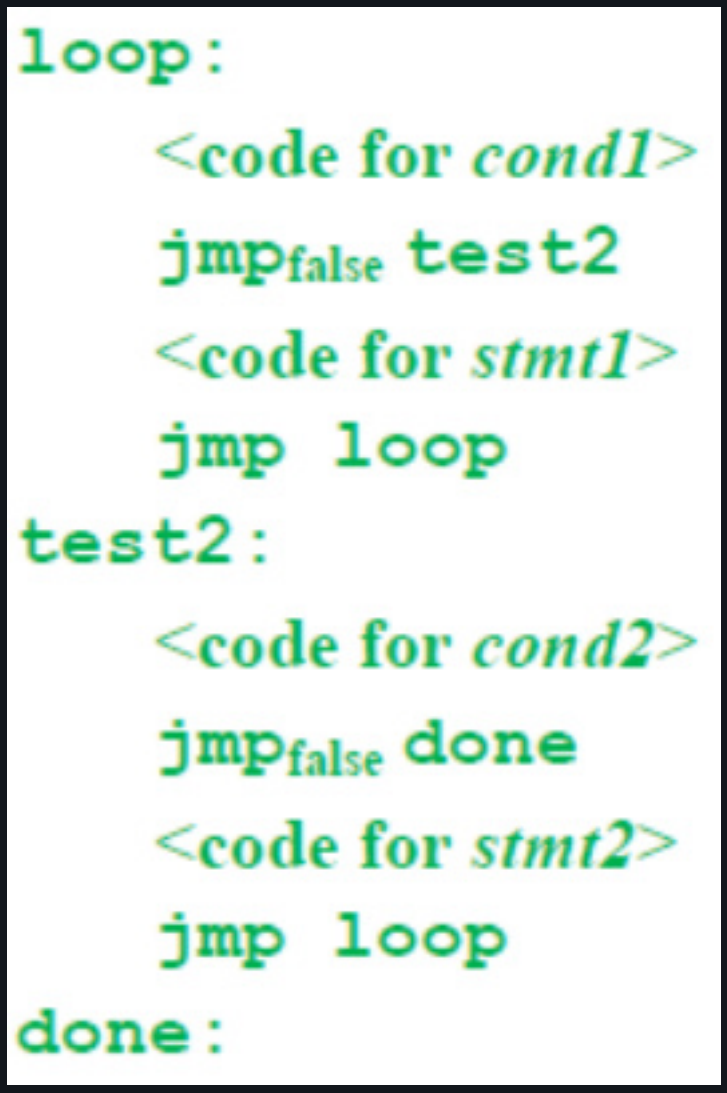
(c) Basta uma nova classe `DoStatement` estendendo `Statement`, contendo uma lista de pares (Expression, lista de Statement) como filhos.

(d) Verificar que as expressões à esquerda de cada `=>` têm tipo Boolean.

(e) Forma do código (rótulos e saltos) para `do cond1 => stmt1 [ ] cond2 => stmt2 od`:

```
loop:
 <code for cond1>
 jmp_false test2
 <code for stmt1>
 jmp loop
test2:
 <code for cond2>
 jmp_false done
 <code for stmt2>
 jmp loop
done:
```

`loop` é o topo (o início do `do`); se a condição cai para falso, testa a próxima; se nenhuma for verdadeira, salta para `done` (o `od`); ao terminar um corpo, volta para o topo na próxima iteração. Solução oficial:



Questão 5 (17 pontos)

Considere a seguinte função e definição de estrutura de dados em C. (Este código utiliza a convenção gcc em que `long` é um tipo inteiro de 64 bits):

```
struct node {     // node with 64-bit int and pointer
   long val;
   struct node * next;
};
long sum(struct node * p) {
   long first, rest;
   if (p == NULL) {
      return 0;
   } else {
      first = p->val;
      rest = sum(p->next);
      return first+rest;
   }
}
```

Esta questão envolve traduzir esta função para código assembler x86-64. Regras básicas:

- You may use either Linux/gcc or Microsoft/masm assembly language, and must follow the corresponding register linkage and stack frame conventions.
- Linux argument registers: rdi, rsi, rdx, rcx, r8, r9
- Linux: called function must save/restore rbx, rbp, r12-r15 if used.
- Microsoft argument registers: rcx, rdx, r8, r9; caller must provide a save area for these four parameter registers at the bottom of the caller's stack frame that the called function can use if desired.
- Microsoft: called function must save/restore rbx, rsi, rdi, rbp, r12-r15 if used.
- Both: function result returned in rax.
- Both: rsp must be aligned on a 16-byte boundary when a call instruction is executed to call the function.
- Pointers and ints are 64 bits (8 bytes) each.
- Your code should implement all of the statements in the original function. In particular, it should include the recursive function call and include store instructions for assignments to local variables, and may not rewrite the function into something different like a loop that produces the same result. Other than that, you can use any reasonable x86-64 code that follows the standard function call and register conventions. (In particular, if a previously computed value is still in a register when needed later in the code, you don't need to include an additional instruction to reload it from memory.)
- Assume the representation of a NULL pointer is an 8-byte binary zero (0) value.
- A C struct is a simple record type. It is not an object with a method table pointer or other hidden data fields.

(a) (0 pontos) Quais convenções você está usando (circule).

(b) (5 pontos) Desenhe o stack frame para uma função de soma como ela apareceria em um programa x86-64 usando as convenções de chamada de função que você indicou acima. Sua imagem deve mostrar as localizações dos parâmetros de função (se ocupam armazenamento), as variáveis locais, temporárias, e os registradores ponteiro de pilha (rsp) e ponteiro de quadro (rbp) como eles existem após o prólogo assinar e alocar o quadro de pilha, mas antes de qualquer uma das declarações no corpo da função ter sido executado. Certifique-se de mostrar os deslocamentos numéricos a partir do registrador ponteiro de quadro para cada variável local e outros locais de armazenamento atribuídos.

(c) (12 pontos) Traduzir a função `sum` para linguagem assembly x86-64. Certifique-se de seguir as regras dadas na primeira página da questão (dica: leia as regras novamente depois de terminar a sua resposta).

```
long sum(struct node * p) {
   long first, rest;
   if (p == NULL) {
      return 0;
   } else {
      first = p->val;
      rest = sum(p->next);
      return first+rest;
   }
}
```

```
struct node {
   long val;
   struct node * next;
};
```

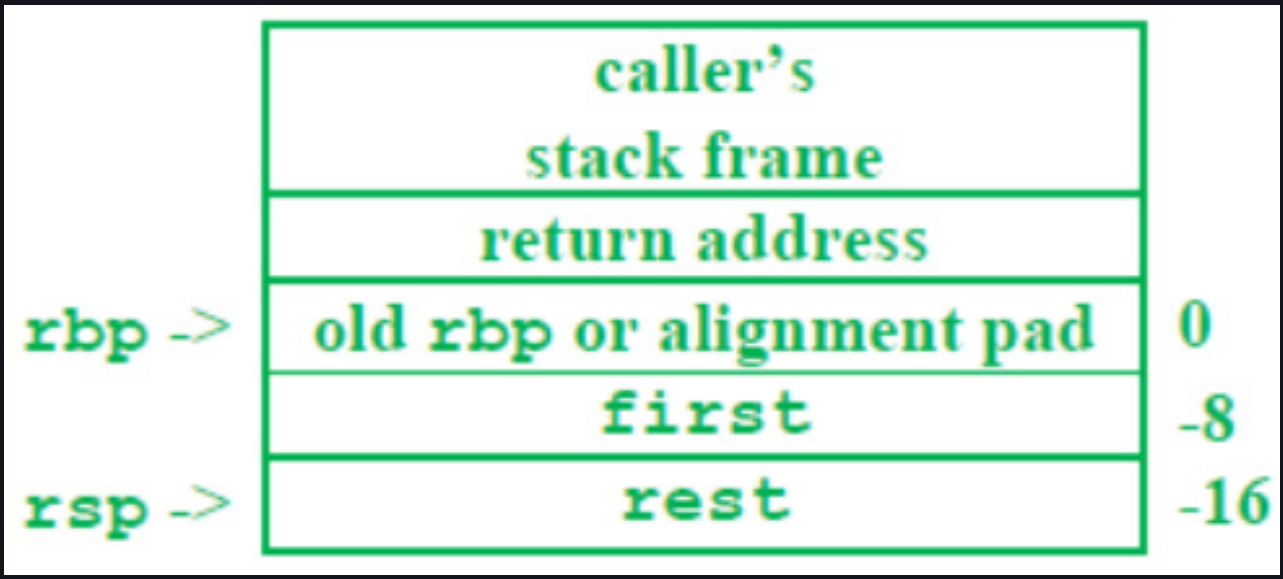
Resolução

(a) A resolução oficial usou a convenção **Linux/gcc** (`%rdi` como argumento, sintaxe GAS, prólogo padrão com `%rbp`).

```
(b) Após o prólogo (pushq %rbp; movq %rsp,%rbp; subq $16,%rsp), antes do corpo:

+-----------------------------+
 | caller's stack frame |
 +-----------------------------+
 | return address |
 rbp -> | old rbp or alignment pad | 0
 | first | -8
 rsp -> | rest | -16
 +-----------------------------+
```

O argumento `p` chega em `%rdi` (registrador, sem slot na pilha); `first` fica em `-8(%rbp)` e `rest` em `-16(%rbp)`.

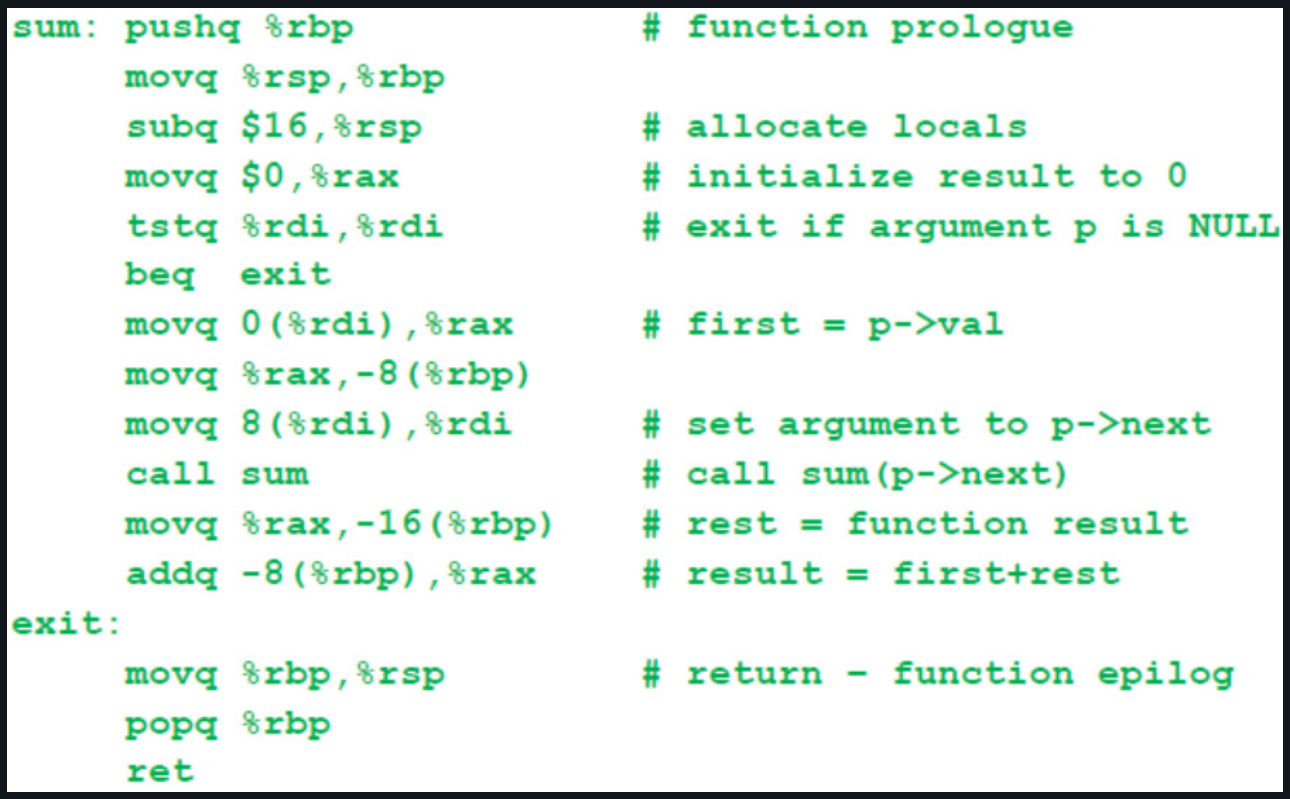


(c) Tradução (sintaxe GAS/Linux):

```
asm
sum: pushq %rbp # prólogo
 movq %rsp, %rbp
 subq $16, %rsp # aloca locais
 movq $0, %rax # resultado = 0
 tstq %rdi, %rdi # sai se p == NULL
 beq exit
 movq 0(%rdi), %rax # first = p->val
 movq %rax, -8(%rbp)
```

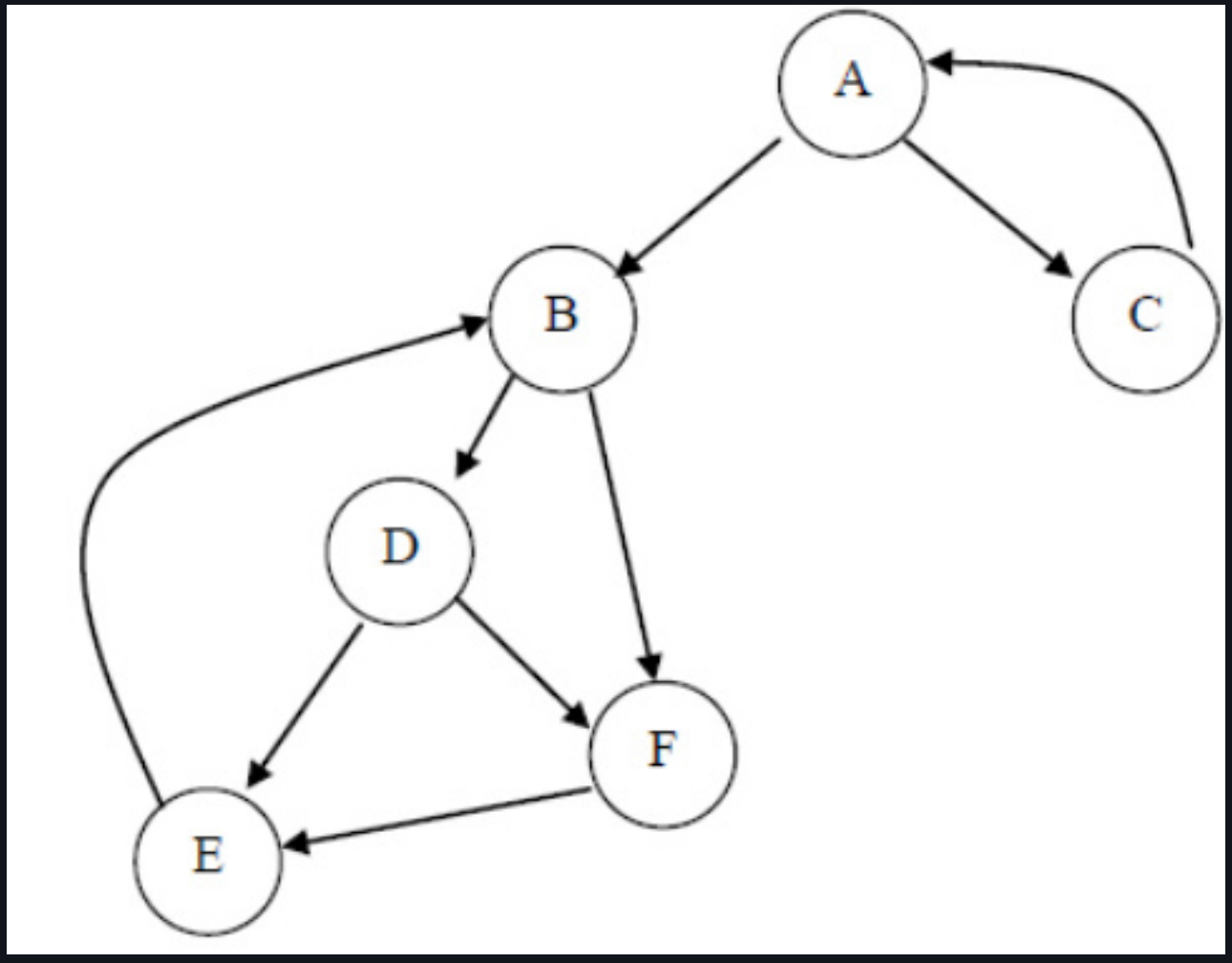
```
movq 8(%rdi), %rdi # argumento = p->next
 call sum # rest = sum(p->next)
 movq %rax, -16(%rbp)
 addq -8(%rbp), %rax # resultado = first + rest
exit:
 movq %rbp, %rsp
 popq %rbp
 ret
```

Obviamente existem muitas soluções possíveis; esta é uma tradução bastante simples do código original. O par de instruções `movq`/`popq` no final pode ser substituído com `leave`, assumindo que `%rbp` é utilizado como registrador base.



Questão 6 (13 pontos)

Dominators e loops. Considere o seguinte grafo de fluxo:



(a) (9 pontos) Para cada nó, listar os nós que são seus dominadores e seu dominador imediato. A é o nó inicial no grafo de fluxo.

(b) (4 pontos) Liste as back edges no grafo de fluxo (isto é, x→y em que a aresta x→y é uma back edge). Para cada back edge, listar o conjunto de nós que formam o loop natural associado com a back edge. (Há provavelmente mais linhas na tabela abaixo do que você precisa, mas adicione mais se precisar.)

Resolução

Grafo: nós A (inicial), B, C, D, E, F; arestas A→B, A→C, C→A, B→D, B→F, D→E, D→F, F→E, E→B.

```
(a) Dominadores:
| Nó | Dominadores | Dominador imediato |
| --- | --- | --- |
| A | A | — |
| B | A, B | A |
| C | A, C | A |
| D | A, B, D | B |
| E | A, B, E | B |
| F | A, B, F | B |
```

Node	Dominators	IDOM
A	A	–
B	A, B	A
C	A, C	A
D	A, B, D	B
E	A, B, E	B
F	A, B, F	B

(b) Back edges (a aresta volta para um dominador da origem) e seus loops naturais:

Back edge	Nós do loop natural
C → A	A, C
E → B	B, D, E, F

Back edge	Nodes in the associated natural loop
C -> A	A, C
E -> B	B, D, E, F

PARTE III — EXERCÍCIOS DOS SLIDES (COM SOLUÇÕES)

Exercícios marcados como “Exercise” nos slides da aula, com enunciado do slide e resolução passo a passo. Não são questões de prova, mas fecham o material e caem nos mesmos assuntos.

Nesta parte:

• Exercício 1 — Gramática ambígua de expressões aritméticas (slide C-grammars) · Exercício 2 — Dangling else (slide C-grammars) · Exercício 3 — Que linguagem esta gramática gera? (slide E-Ir-construction) · Exercício 4 — Arrays 2-D: Fortran × C (slide K-codeshape-1) · Exercício 5 — Grafo de interferência e coloração (slide N-backend)

O sumário com a página de cada prova e de cada questão está logo após a página de organização.

Exercícios dos slides

Exercício 1 — Gramática ambígua de expressões aritméticas (slide C-grammars)

Questão 1

Slide C-grammars (p.18) - Exemplo: gramática ambígua para expressões aritméticas:

```
expr ::= expr + expr | expr - expr | expr * expr | expr / expr | int
```

int ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 Exercise: show that this is ambiguous. How? Show two different leftmost or rightmost derivations for the same string. Equivalently, show two different parse trees for the same string.

Resolucão

```
Basta uma string com duas operações do mesmo nível, por exemplo "1+2+3" (derivação: expr =>
expr+expr =>
int+expr =>1+expr => ... ). Duas derivações leftmost DIFERENTES para a MESMA string:
```

Derivacao1 (soma associada à esquerda - arvore ((1+2)+3)):

expr

```
=> expr + expr
=> (expr + expr) + expr [expande o expr mais à esquerda com +]
=> (int + expr) + expr
=> (1 + expr) + expr
=> (1 + int) + expr
=> (1 +2) + expr
=> (1 +2) + int
=>1 +2 +3 Derivacao2 (soma associada à direita - arvore (1+(2+3))):
```

expr

```
=> expr + expr
=> int + expr [expande o expr da esquerda diretamente para int]
=>1 + expr
=>1 + (expr + expr)
=>1 + (int + expr)
=>1 + (2 + expr)
=>1 + (2 + int)
```

=>1 +2 +3 As duas sequências de formas sentenciais são distintas (na1ª, o terceiro passo expande o expr da esquerda de novo; na2ª, ele vira int e a expansão acontece do lado direito) e produzem árvores de sintaxe diferentes:

((1+2)+3) e (1+(2+3)). Portanto a gramática é ambígua.

Consequência prática: a ambiguidade gera conflitos deslocar/reduzir nos parsers LR (a decisão de precedência/associatividade ficaria indefinida). Por isso linguagens reais usam gramáticas estratificadas (term/factor para precedência) ou diretrizes %left/%right no gerador de parser.

Exercício 2 — Dangling else (slide C-grammars)

Questão 1

Slide C-grammars (p.27) - Outro exemplo clássico (dangling else):

```
ifStmt ::= if ( cond ) stmt | if ( cond ) stmt else stmt Exercise: show that this is
ambiguous. How?
```

Resolução

use o string com dois if's e um único else:

"if (c1) if (c2) a else b" Derivacao1 - o else pareia com o if EXTERNO (a raiz usa a segunda produção):

stmt

```
=> if ( cond ) stmt else stmt [producao2 na raiz]
=> if ( cond ) if ( cond ) stmt else stmt [stmt interno usa a producao1]
=> if ( cond ) if ( cond ) a else stmt
=> if ( cond ) if ( cond ) a else b Leitura: (if (c1) if (c2) a) else b Derivacao2 - o else
pareia com o if INTERNO (a raiz usa a primeira produção):
```

stmt

```
=> if ( cond ) stmt [producao1 na raiz]
=> if ( cond ) if ( cond ) stmt else stmt [stmt interno usa a producao2]
=> if ( cond ) if ( cond ) a else stmt
=> if ( cond ) if ( cond ) a else b Leitura: if (c1) (if (c2) a else b)
```

O string é idêntico nas duas derivações, mas as árvores associam o else a if's diferentes: gramática ambígua.

Consequência: parsers LL/LR encontram conflito no else pendente. A convenção universal (C, Java, etc.) é resolver o conflito de DESLOCAR, pareando o else com o if mais INTERNO - ou seja, a derivação2. Em ferramentas como yacc/bison isso é feito com %nonassoc/precedência (em geral, um %prec maior para a produção com

else) ou simplesmente pela regra padrão de conflito shift > reduce.

Exercício 3 — Que linguagem esta gramática gera? (slide E-lr-construction)

Questão 1

Slide E-lr-construction (p.5) - Construção dos estados LR(0), exemplo:

```
S' ::= S $
S ::= ( L )
S ::= x
L ::= S
L ::= L , S Question: What language does this grammar generate?
```

Resolução

Resolução - leitura recursiva da gramática:

- S' só garante o fim de entrada ($); a linguagem é a de S.
- S ::= x: um x isolado é uma S.
- S ::= (L): S também é um parêntese aberto contendo uma L.
- L ::= S: uma L é uma única S.
- L ::= L , S: uma L é uma L seguida de vírgula e mais uma S - recursivamente, isso gera uma OU MAIS S separadas por vírgula.

Lingua descrita (em PT): strings sobre {x, (,), ,} em que cada item é um x ou um parênteses contendo uma lista por vírgula de itens, recursivamente. Mais formalmente:

```
L(G) = { w | w é um item }, onde item ::= x | ( lista ) e lista ::= item ( , item )+
Exemplos PERTENCENTES: x ; (x) ; (x,x) ; ((x)) ; (x,(x)) ; (x,x,(x,x)).
```

Exemplos NAO PERTENCENTES: ε (vazio) ; x,x (no alto nível só pode haver UM S, sem vírgula solta) ; ( ) (lista vazia não existe) ; ,x ; x) ; (x.

Ou seja: "listas por vírgula de x's entre parênteses, aninháveis, nunca vazias - e um x solto no alto nível".

Exercício 4 — Arrays 2-D: Fortran × C (slide K-codeshape-1)

Questão 1

Slide K-codeshape-1 (p.34) - Arrays2-D:

- Subscripts começam em1 (default do Fortran).
- C etc. usam ordem row-major. Ex.: array com3 linhas e2 colunas armazenado na sequencia: a(1,1), a(1,2), a(2,1), a(2,2), a(3,1), a(3,2).
- Fortran usa ordem column-major.

Exercises: What is the layout? How do you calculate the location of a(i,j)? What happens when you pass array references between Fortran and C/etc. code?

Resolução

1) Layout (Fortran, column-major, subscripts iniciando em1): os elementos variam primeiro nas LINHAS. Para o exemplo de3 linhas x2 colunas, a ordem em memória e:

a(1,1), a(2,1), a(3,1), a(1,2), a(2,2), a(3,2) (ou seja, a coluna1 inteira e depois a coluna2 - transposta da ordem da C.)

2) Endereço de a(i,j), com n = numero de LINHAS, t = tamanho do elemento, base = endereco do primeiro elemento:

endereco(a(i,j)) = base + ((j-1) * n + (i-1)) * t Passo a passo: quantas colunas inteiras cabem antes da coluna j? (j-1) colunas, cada uma com n elementos; dentro da coluna j, falta andar (i-1) elementos ate a linha i.

3) Passagem entre Fortran e C: e a MESMA regiao de memoria lida com ordens opostas. O que Fortran enxerga como a(i,j), o codigo C enxerga como a_c[j][i] (com o numero de linhas e colunas trocados). Consequencias:

- a matriz "aparece transposta" para quem le na outra ordem;
- exemplos: um array Fortran1x3 e visto por C como3x1; loops que percorrem linhas em Fortran percorrem colunas em C;
- para passar corretamente: informar as dimensoes certas na interface (chamar de fora de Fortran/C com tamanhos transpostos) e tratar o buffer como a matriz transposta - ou copiar/reordenar explicitamente. (Nao ha "arranjo magico": quem trata o ponteiro sem trocar a semantica de dimensoes recebe os dados em ordem errada.)

Exercício 5 — Grafo de interferência e coloração (slide N-backend)

Exercício — Grafo de interferência e coloração (slide N-backend)

Slide N-backend (p.47) - Analise de vida e coloracao de grafos:

Exercise: given that a and b are live on entry and dead on exit, and that x and y are live on exit:

```
{ int tmp_2ab =2*a*b;
int tmp_aa = a*a;
int tmp_bb = b*b;
x := tmp_aa + tmp_2ab + tmp_bb;
y := tmp_aa - tmp_2ab + tmp_bb; }
```

(a) construct the register interference graph

(b) color the graph; how many registers are needed?

Resolução

Passo1 - Intervalos de vida (live ranges), instrucoes numeradas I1..I5:

```
I1: t1 :=2*a*b (t1 = tmp_2ab)
I2: t2 := a*a (t2 = tmp_aa)
I3: t3 := b*b (t3 = tmp_bb)
I4: x := t2+t1+t3
I5: y := t2-t1+t3
```

- a: viva da entrada ate I2 (usada em I1 e I2; morta depois de I2).
- b: viva da entrada ate I3 (usada em I1 e I3; morta depois de I3).
- t1: definida em I1, usada em I4 e I5 - viva de I1 a I5.
- t2: definida em I2, usada em I4 e I5 - viva de I2 a I5.
- t3: definida em I3, usada em I4 e I5 - viva de I3 a I5.
- x: definida em I4, viva na saida.
- y: definida em I5, viva na saida.

Passo2 - Arestas de interferencia (dois registradores interferem se os valores estao vivos ao mesmo tempo; equivalente: toda variavel viva quando um novo valor e definido interfere com ele):

- entrada: a e b vivas juntas -> a-b.
- I1 define t1 com {a,b} vivos -> t1-a, t1-b.
- I2 define t2 com {t1,b} vivos -> t2-t1, t2-b.
- I3 define t3 com {t1,t2} vivos -> t3-t1, t3-t2.
- I4 define x com {t1,t2,t3} vivos -> x-t1, x-t2, x-t3.
- I5 define y com {x} vivo -> x-y.

Grafo (11 arestas):

a - b, a - t1, b - t1, b - t2, t1 - t2, t1 - t3, t2 - t3, t1 - x, t2 - x, t3 - x, x - y.

Passo3 - Coloracao:

- {t1,t2,t3} forma um triangulo (clique de3) -> pelo menos3 cores.
- x interfere com t1, t2 e t3 -> uma quarta cor e obrigatoria: {t1,t2,t3,x} e um K4 -> o numero cromatico e >=4.
- Coloracao com4 cores (uma possibilidade):

t1=1, t2=2, t3=3, x=4

```
a: vizinhos {b, t1=1} -> a=2 (nao interfere com t2)
b: vizinhos {a=2, t1=1, t2=2} -> b=3 (nao interfere com t3)

y: vizinho {x=4} -> y=1 (nao interfere com t1)
```

Sao necessarios e suficientes 4 registradores. Uma atribuicao valida:

```
R1 = { t1 (tmp_2ab), y }
R2 = { t2 (tmp_aa), a }
R3 = { t3 (tmp_bb), b }
R4 = { x }
```

Grafo de interferência (item a): as 11 arestas listadas acima — {t1,t2,t3} forma um triângulo (clique de 3) e x interfere com os três, formando um K4 (daí 4 registradores).

Nota — fontes e critério de compilação

Nota — fontes e critério desta edição. Fontes: pasta `Compiladores` extraída de `Compiladores.zip` — `exames\Provas de Washington\Provas mid` e `Provas finals` (classificação oficial de meio/final), `exames\Provas Andre` (gabaritos das provas do Prof. André) e `Slides\*.pdf` (exercícios da Parte III). Critério de classificação: provas com rótulo de origem foram mantidas no grupo de origem; as cinco provas sem rótulo (au02, au05, wi08, au09, au11) foram classificadas pelo conteúdo predominante, como explicado na página de organização. Critério de repetições: quando a mesma questão aparece duas vezes, fica a primeira ocorrência e a segunda vira uma nota de referência (o enunciado e a resolução não foram duplicados). Critério de texto: fim de linha no meio da frase foi removido, listas e rótulos ganharam a quebra de linha que faltava, e código/figuras foram preservados como no original.